



„ComputerParts.hu” dokumentáció

Készítette: Magyar Csaba & Rebe Dániel

Tartalomjegyzék

#1 – Projektünk célja:	3
#2 – Fejlesztői dokumentáció	4
Miért minket válasszon, ha még is annyi másik opció létezik?	4
Fejlesztői környezet	5
Fejlesztési tervek és ötletek a jövőre nézve.....	6
Adatbázis terv és esetleges további fejlesztései	8
Feladatok felosztása	12
Drótvázak, valamint a design-ok képekben	13
Tesztelés leírása, tesztdokumentáció	16
Konfiguráció és Inicializálás	23
ComputerpartsDbContext osztály	26
Adatlekérés és az állapot inicializálása	28
A LoginAsync metódus és a sikeres belépés logikája	30
Kijelentkezés és az erőforrások felszabadítása.....	31
Biztonsági jelszókezelés (PasswordHashService)	33
Szerveroldali Hitelesítési Logika (UserAuthService).....	34
A Login folyamat részletes elemzése	35
Rendszerkonfiguráció (API Program.cs)	37
Controllers (Kontrollerek)	40
Vezérlő réteg: Hitelesítés (AuthController)	40
Felhasználói regisztráció és Profil lekérés	41
Felhasználó-menedzsment (UsersController)	44
Címkező vezérlő (AddressesController)	46
Egyedi összeállítás alkatrészei (BuildComponentsController)	48
Kategória kezelő vezérlő (CategoriesController)	50
Alkatrész kezelő vezérlő (ComponentsController)	52
Alkatrész típus vezérlő (ComponentTypesController)	54
Egyedi összeállítások vezérlő (CustomBuildsController).....	56
Ajánlatok és Kedvezmények vezérlő (DealsController).....	59
Alkatrész raktárkészlet vezérlő (InventoryComponentsController)	61
Termék raktárkészlet vezérlő (InventoryProductsController).....	63

Rendelési tételek - Egyedi összeállítások (OrderItemBsController)	65
Rendelési tételek - Késztermékek (OrderItemPsController).....	67
Előre összeállított PC-k alkatrészei (PrebuiltPcsCompsController)	71
Előre összeállított PC vezérlő (PrebuiltPcsController)	73
Termék kezelő vezérlő (ProductsController)	75
Adminisztrátori Felület (Admin.razor)	77
Hozzáférés-védelem és kezdeti konfiguráció	77
UI Design és Reszponzivitás	77
Fejléc és navigációs kártyák	79
Tab-alapú tartalomkezelés	80
Adatkezelő modulok: Felhasználók és Címek	81
Felhasználói lista és műveletek	81
Címadatok adminisztrációja (Addresses)	82
Kereskedelmi modulok: Termékek és Kategóriák	83
Termékkatalógus kezelése (Products)	83
Kategóriarendszer adminisztrációja (Categories)	84
Technikai modulok: Alkatrészek és Típusok.....	85
Alkatrész-katalógus kezelése (Components)	85
Alkatrésztípusok adminisztrációja (Component Types).....	86
Rendeléskezelés és a komponens logikája.....	87
Rendelések felügyelete (Orders)	87
A háttérlogika kezdete (C# Code Block)	88
Felhasználói dokumentáció	90
Felhasználói felületek	90
Adminisztrátor felületek	98
Köszönet nyilvánítás és Projekt értékelése.....	101
Köszönetnyilvánítás	101
Projekt értékelése	102
Kép források.....	103

#1 – Projektünk célja:

Projektünk célja egy **átlátható, letisztult**, de közben **modern és megbízható** weboldal létrehozása, amelyet használva a felhasználónak lehetősége nyílik az *oldalunkon megtalálható* összes PC alkatrész megtekintése és összeválogatása, akár **külön-külön**, akár **egyben összeválogatva**, mindenzt egy olyan felülettel, amit bármely korosztály gondtalanul, egyszerűen használhat. Oldalunkon a **legalapvetőbb** számítástechnikai eszközök, alkatrészek közül válogathat a felhasználó, legyen szó **videókártyáról (GPU/VGA)**, **alaplapról (Motherboard)**, vagy a már elkészült és eddig is használt konfigurációhoz **új perifériák**, mint **billentyűzet** vagy **monitor**. Weboldalunk egy olyan technikai, főként számítástechnikai konfigurációkat és alkatrészeket összegyűjtő oldal, amelyen mindenki könnyen megtalálja, amire szüksége van, ráadásul olyan extra funkciókkal is rendelkezünk, amelyek sok más helyen egyáltalán nincsenek, mi pedig úgy gondoltuk, hogy szükség van rá. A felhasználónak lehetősége van nem csak összeállítani különböző konfigurációkat, de ezeket akár a saját fiókjában menteni is tudja, ezzel pedig a jövőben bármikor vissza térhet hozzá és akár módosíthatja is szükség szerint, emellett egy adott alkatrész vagy teljes konfiguráció átlagárát is olvashatja a felhasználó, fontos kiemelni, hogy ez kizárólag átlagár vagy a gyártó által ajánlott fogyasztói ár, nem tükrözi pontosan az adott termék árát. Sok, tech-termékek értékesítésével foglalkozó webshopon, ahol egyáltalán megtalálható a mi oldalunk fő funkciója, csak egy amolyan extraként van jelen, viszont úgy igazán nincsen vele foglalkozva, nem tartalmaz semmilyen innovatív funkciót. A felhasználói felület kialakításában Mesterséges Intelligencia is a segítségünkre volt, ennek segítségével hoztuk létre az első oldalunkat.

#2 – Fejlesztői dokumentáció

Miért **minket** válasszon, ha még is annyi másik opció létezik?

Ezt a pontot több alpontra lehetne bontani, mivel több, mellettünk szóló érv van, ami a felhasználót segíti a választásban, így külön alpontokra bontottuk és részletesen elmagyaráztuk, miben vagyunk mások, „jobbak” a többi lehetőségnél:

1. **Felhasználóbarát kezelő felület** – Mindannyian tudjuk, mennyire könnyen el lehet vészni egy túlszűfolt, felesleges feliratokkal és gombokkal rendelkező weboldalon. Nálunk a felhasználó tényleg mindent elér pár kattintással anélkül, hogy összezavarodna az információk kavalkádjában.
2. **Böngészési élmény** – Tapasztalataink alapján több „konkurens” weboldalon előfordult a keresés közben, hogy valamilyen termékre *specifikusan* szerettünk volna rákeresni, de valamiért rengeteg, *keresésünk szempontjából irreleváns* termék és információ jött velünk szembe. Nekünk sikerült egy olyan *megoldást találnunk erre a problémára*, aminek köszönhetően a felhasználó egyetlen percet sem fog eltölteni azzal, hogy megtalálja, melyik termék felel meg keresési feltételeinek, valamint minden információ helyesen, az adott termékhez fog kapcsolódni, amit éppen keres.
3. **Előre össze szerelt PC-k** – A legtöbb számítástechnikai eszközök értékesítésével foglalkozó weboldalon nagyon kevés, és teljesítményükhöz képest sokszor túlárazott gépeket lehet kapni, ráadásul még az is elképzelhető, hogy a gépeikben felhasznált alkatrészek már elavultnak besorolhatók 2026-ban, ami a vásárló számára komoly fejfájást okozhat a távoli, de sokkal inkább a közeli jövőben. Mi ezt a hibát is áthidaltuk azzal, hogy a mi előre össze szerelt gépeink mindig modern alkatrészekből állnak, amelyek nem csak a koruk, de még funkcióik és természetesen árazásuk végett is tökéletes választások lehetnek, emellett a vásárló bármelyik alkatrészt lecserélheti vásárláskor, azon alkatrészekre, amik aktuálisan elérhetőek, valamint a PC többi alkatrészével is gondtalanul képesek működni.
4. **Teljes konfiguráció építés lehetősége, személyes igények szerint** – Valamennyi weboldalból, ahol számítógépet, vagy számítógép alkatrészeket lehet kapni, *nagyon kevés kivétellel* hiányzik a felhasználó számára biztosítandó lehetőség, hogy saját magának, a nulláról állítsa össze új konfigurációját, a neki tetsző és szükséges alkatrészekből. Nálunk erre is meg van a lehetőség, ráadásul a felhasználó *szintén kérheti előre össze szerelve* újonnan vásárolt számítógépét, emellett a vásárló szüksége szerint akár *előre telepített, aktivált, tehát jogtiszt*a Windows 11 rendszerrel, illetve szintén *jogtiszt*a Microsoft Office 365 programmal/programokkal is felvértezzük új PC-jét.

Fejlesztői környezet

Avagy miket használtunk/használunk munkánk során, hogy a lehető legjobbat hozzuk ki projektünkéből:

1. – Magyar Csaba

- a. *Programok*: Microsoft Visual Studio 2026; Microsoft SQL Server Management Studio 2022
- b. *Keretrendszerek és programnyelvek*: C# (Csharp), Blazor, MSSQL, Microsoft EntityFrameworkCore (illetve az EF Core al-Freamwork-jei)
- c. *Rendszerleírás*: PRIVÁT INFORMÁCIÓ

2. – Rebe Dániel

- a. *Programok*: Microsoft Visual Studio 2022/Visual Studio 2026; Microsoft SQL Server Management Studio 2022
- b. *Keretrendszerek és programnyelvek*: C# (Csharp), Blazor, MSSQL, Microsoft EntityFrameworkCore (illetve az EF Core al-Freamwork-jei)
- c. *Rendszerleírás*:
 - i. Windows 11 PRO, AMD Ryzen 9 5900X; Nvidia GeForce RTX 4070 Super 12Gb, 64Gb DDR4 3200Mhz RAM;
 - ii. Windows 11, Intel Core I5-12450H, Nvidia GeForce RTX 4050 6Gb (Laptop Edition), 16Gb DDR4 3200Mhz RAM

Projektünk fejlesztése során kizárólag C-Sharp környezet használatában gondolkodtunk, ugyan is ezen programnyelv, valamint a különböző, hozzá adható keretrendszert már megtanultuk jól kezelni, ebben hatékonyan, gyorsan, de egyben minőségi munkát előállítva, ami nem csak a program jó működését biztosítja a felhasználó számára, de számunkra, a fejlesztőknek is sokkal átláthatóbb, könnyebben javíthatók az esetleges hibák, valamint rendszerezettebben tudjuk tartani az összes file-t, kódot, amit a projektben használtunk. Ellenben a Javascript és Node, amelyek számunkra sokkal több kihívást jelentettek amiatt, hogy szerintünk egyszerűen túl van bonyolítva ez a két nyelv, emiatt nehezebben átlátható, nehezebb az egyes kód-részleteket újra felhasználni, ha arra lenne szükség, valamint a file-ok rendszerezhetőségét tekintve is elmarad a C-Sharp környezetétől, ráadásul olyan, fejlesztési ötleteink szempontjából kulcsfontosságú részleteket csak elképesztően túlbonyolítva lehet felépíteni, mint például a kosárkezelés vagy a felhasználói fiókokba való bejelentkezés, mint a C-Sharp-ében.

Fejlesztési tervek és ötletek a jövőre nézve

Projektünk alapvető célja, hogy a felhasználók egy könnyen kezelhető felületen tudjon válogatni és akár informálódni az adatbázisunkban megtalálható termékekről, ezeket összeválogatva pedig akár az álom eszközeit, gépeit válogathassa össze egyetlen helyen, azonban rengeteg sok fejlesztési lehetőség van weboldalunkon.

Ezek közé tartozik az, hogy át lehetne építeni egy teljes webáruházzá, megtartva oldalunk fő funkcióját: az alkatrészek keresését, összeválogatását, adatainak könnyen történő megtekinthetőségét és teljes konfiguráció összeállítását, mindet egy helyen. Webshoppá alakításakor az egyik legfontosabb funkciója természetesen a kosárkezelés, ennek segítségével a felhasználónak pedig nem kellene másik oldalakra lépnie, ha valamilyen alkatrészt, tegyük fel processzort szeretne vásárolni, hanem egyből megteheti ezt nálunk. Ehhez pedig társulnak olyan funkciók, mint például az adott rendelés nyomkövetése, hogy az adott pillanatban hol tart a rendelés: feldolgozás, beszerzés, csomagolás, kiszállítás, természetesen egy automatikusan kiküldött Email-vagy SMS-üzenetbéli értesítéssel, attól függően, melyiket preferálja a felhasználó. Természetesen ide kapcsolódik, hogy a felhasználói fiókban, illetve Email-ben kaphassa meg a felhasználó a rendeléséhez tartozó számláját. Ez amiatt fontos, hogy ha esetleg szállításkor vagy használatkor megsérülne a termék, a vásárló bizonyíthassa rendelését és érvényesíthesse a termékre kapott garanciát, így kérvényezheti a termék javítását (sérülés esetén), cseréjét vagy az érte kifizetésre került összeg visszaigénylését.

Amit fontosnak tartunk a Webshoppá alakításnál, hogy a felhasználók elmondhassák az adott termékről a véleményüket egy olyan formában, amely közvetlenül oldalunkon jelenhet meg. Bár ez véleményünk szerint kissé befolyásolja a vásárlókat abban, melyik termék lenne számukra lehető legidálisabb, mi fontosnak tartjuk, hogy a felhasználók megoszthassák tapasztalataikat bármelyik termékről, amely megtalálható adatbázisunkban. Ehhez nem csak az egyszerű, egytől öt csillagig terjedő értékelésből és az egyszerű „használatlan”, „elfogadható” vagy „nagyon jó” írományokból álló értékelés rendszerl enne, hanem a felhasználót kifejezetten kérjük, mondja el, pontosan mely szempont számára az, ami miatt ajánlaná vagy sem az adott terméket, ezt pedig a legegyszerűbben azzal lehet elérni, ha minimum karakterszámot adunk meg a vélemény leírásának, valamint „pro” és „kontra” szerű, rövid leírást adhat meg a felhasználó, mik voltak a számára előnyös és előnytelen funkciók, paraméterek, adatok. A vélemény hitelességét pedig legkönnyebben az lehet alátámasztani, hogy a véleményt író felhasználónak kötelezően ki kell jelölnie, hogy használta-e már az adott terméket vagy termékeket, így ugyan minimálisan, de szűrhető, ki az, aki tényleges tapasztalatból vagy szakmai tudásból, és ki az, aki pusztán érzelemből (például egy adott gyártó iránti elkötelezettségből vagy utálatból) fogalmazza meg pozitív vagy épp negatív véleményét. Enek szűrése azért nem egy egyszerű feladat, mivel weboldalunk létre jötté előtt a felhasználók sok, „konkures” webshopból vásárolhatott már egy vagy több adott terméket, nekünk pedig természetesen nincs semmilyen hozzáférésünk más oldalak adatbázisaihoz.

Emellett pedig akár olyan funkció és beépítésre kerülhet az oldalra, amely kifejezetten a PC-építés „szerelmeseinek” lett kitalálva, ez pedig nem más, mint a „Bottleneck Calculator”. Ez egy olyan funkció lenne, amely képes több alkatrészre lebontva kiszámolni, hol és mi fog „Bottleneck-et”, avagy „Szűk keresztmetszetet” okozni a rendszerben. Ennek az értéknek megismerése és számításba vétele amiatt fontos a felhasználó számára, mert így meg fog tudni bizonyosodni arról, hogy semelyik alkatrész, legtöbb esetben a processzor, videokártya, memória (ennek a mérete vagy órajele), valamint háttértár, nem fogja vissza túlzottan a többi teljesítményét. A „legjobb Bottleneck-et” akkor kaphatjuk, ha inkább a videokártyának kell a legtöbbet dolgoznia, ugyan is ez azt jelenti, hogy a processzor minden alkatrészt teljesen ki tud használni.

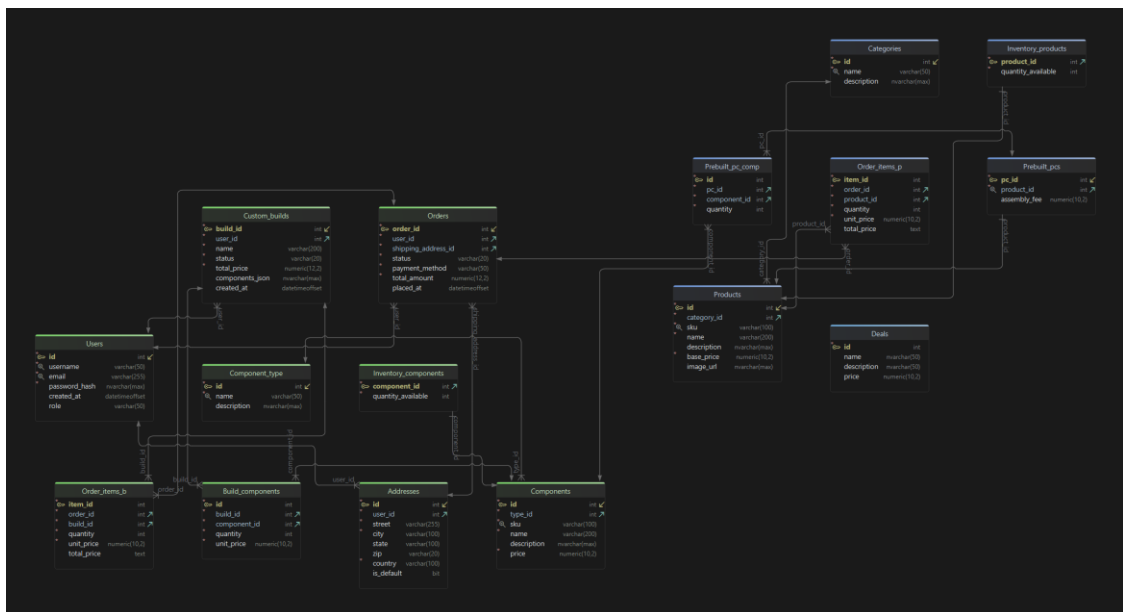
Weboldalunk fő funkciójánál, a „PC összerakó” fülön, ami a felhasználó számára lehetővé teszi, hogy saját konfigurációt állítson össze, jelenleg a rendszer nem ellenőrzi le, hogy az adott alkatrészek kompatibilisek-e egymással, ez a funkció egyelőre még nem került be, viszont mindenképp egy jó és egyben nagyon is fontos fejlesztés a jövőre nézve.

Vannak oldalak, amelyek rendelkeznek olyan gombokkal, amik vagy csak egy üzenetet dobnak fel, amin az áll, hogy az adott gomb funkciója fejlesztés alatt áll *(mint a főoldalon lejjebb görgetve, bármelyik gombra kattintva a „Kiemelt Termékek” alatt található ablakokon)*, vagy egyáltalán nem rendelkeznek egyelőre semmilyen funkcióval *(mint az „Ajánlatok” oldalon bármelyik kisebb ablak gombja)*. A jövőben ezek a gombok is megkapják a szükséges funkciót, viszont ezekhez a fentebb említett webshoppá alakítás is követelmény, anélkül ezek a gombok nem használhatóak.

A legnagyobb és egyben a hozzánk hasonló weboldalakhoz képest leginnovatívabb fejlesztési ötlet pedig a „Közösségi Áruház” fantázianévre hallgat. Ez egy főként felhasználók által eladni kívánt, használt vagy akár új *(„új” alatt „bontatlan” vagy „pár alkalommal használt” értendő)* termékek lennének fellelhetők, amolyan használt termékek piacát képezve. Itt a felhasználók közvetlenül egymás között bonyolítanak le az üzletet, weboldalunk pedig a platformot biztosítaná a felhasználók számára, mindezt úgy, hogy oldalunk ezért cserébe semmivel nem terheli az eladót, nem termelünk belőle semmilyen profitot. A termékek adatait, leírásait mindet a felhasználónak kell megadnia és hasonlóan a fentebb említett értékelés ötleténél, itt is egy minimum karakterszámot meghaladva kell a felhasználónak megadnia az általa eladni kívánt termék adatait, ezzel is biztosítva, hogy a potenciális vásárló a lehető legtöbbet tudhassa meg a kiszemelt termékről. Ha pedig valamire a megítélésünk szerint az eladó nem biztosított megfelelő mennyiségű információt a hirdetésében, akkor az adatbázisunkból, egy külön szövegdobozban jelenik meg a teljes leírása az adott terméknek. Ha ez a termék nem szerepel adatbázisunkban, akkor értelem szerűen nem jelenít meg a rendszer teljes leírást.

Adatbázis terv és esetleges további fejlesztései

Adatbázisunk terve az alábbi képen látható, most már a vizsga szempontjából végleges állapotában. Korábbi leírást idézve, nem egy olyan tábla volt, ami a véglegesített formából kikerült, illetve van, amelyik idő közben került be. Az alábbi kép mutatja teljes adatbázisunk összeállítását, melyik tábla hogyan kapcsolódik melyik táblához, melyek azok a táblák, amik a felhasználók információit tárolják, viszont ezt lentebb részletesebben is leírtuk. A kikerült táblák nevei természetesen nem szerepelnek ezen az adatbázis terven, sem a képen, sem pedig a leírásokban, illetve a fejlesztési ötletekhez sem kerültek be, révén, hogy a kikerült táblák nevei bármikor változhattak volna, valamint vannak ezek között olyan táblák, amelyek egyelőre nem kerültek felhasználásra, de egy esetleges fejlesztési ötlet megvalósítása esetén már meg vannak, így akkor ezekkel nem kell foglalkozni.



Adatbázisunk táblái a következő képpen épülnek fel és kapcsolódnak egymáshoz:

Categories	
id	int
name	varchar(50)
description	nvarchar(max)

- a. „categories”: Ez a tábla tárolja, melyik, weboldalunkon kapható terméknek mi a kategóriája (pl. Monitor, Videókártya). Ez a tábla Primary Key-el kapcsolódik a „products” táblához.

id	int
category_id	int
sku	varchar(100)
name	varchar(200)
description	nvarchar(max)
base_price	numeric(10,2)
image_url	nvarchar(max)

- b. „products”: Ez a tábla tárolja minden egyes nálunk kapható termék összes adatát. Primary Key-e az „Id”.

pc_id	int
product_id	int
assembly_fee	numeric(10,2)

- c. „prebuilt_pcs”: Ez a tábla tartalmazza az előre össze szerelt (desktop) PC-k Id-ját és árát. Primary Key-e a „pc_id”, valamint Foreign Key-el kapcsolódik hozzá a „products” tábla a „product_id”-n.

id	int
pc_id	int
component_id	int
quantity	int

- d. „prebuilt_pc_comp”: Ez a tábla tartalmazza az össze, előre össze szerelt (desktop) PC alkatrészenek ID-jait, valamint azokból a mennyiséget. Kapcsolódik hozzá Foreign Key-el a „components” tábla az „id -> component_id” mentén, valamint a „prebuilt_pcs” tábla a „pc_id -> pc_id” mentén.

id	int
type_id	int
sku	varchar(100)
name	varchar(200)
description	nvarchar(max)
price	numeric(10,2)

- e. „components”: Ez a tábla tartalmazza az előre össze szerelt (desktop) PC-k összes alkatrészét, valamint azoknak az összes adatát. Kapcsolódik a „prebuilt_pc_comp” az „id -> component_id” mentén, valamint a „component_type” táblához az „id -> id” mentén.

id	int
name	varchar(50)
description	nvarchar(max)

- f. „component_type”: Ez a tábla tartalmazza az összes, előre össze szerelt (desktop) PC alkatrészek típusát. Kapcsolódik a „components” táblához az „id -> type_id” mentén.

component_id	int
quantity_available	int

- g. „inventory_components”: Ez a tábla tárolja, hogy az adott alkatrészből mennyi van készleten. Kapcsolódik a „components” táblához a „component_id -> id” mentén.

order_id	int
user_id	int
shipping_address_id	int
status	varchar(20)
payment_method	varchar(50)
total_amount	numeric(12,2)
placed_at	datetimeoffset

- h. „orders”: Ez a tábla tárolja az összes rendelés adatát. Kapcsolódik az „order_items_p” táblához az „order_id -> order_id” mentén, valamint kapcsolódik az „addresses” táblához a(z) „user_id -> id” mentén.

id	int
user_id	int
street	varchar(255)
city	varchar(100)
state	varchar(100)
zip	varchar(20)
country	varchar(100)
is_default	bit

- i. „addresses”: Ez a tábla tárolja az összes megrendelés szállítási adatát. Kapcsolódik az „orders” táblához az „id -> shipping_address_id” mentén, valamint a „users” táblához a(z) „user_id -> id” mentén.

id	int
username	varchar(50)
email	varchar(255)
password_hash	nvarchar(max)
created_at	datetimeoffset
role	varchar(50)

- j. „users”: Ez a tábla tárolja az összes felhasználó adatát. Kapcsolódik hozzá a „custom_builds”, az „orders”, valamint az „addresses” tábla, mind az „id -> user_id” mentén.

Build_components	
id	int
build_id	int
component_id	int
quantity	int
unit_price	numeric(10,2)

- k. „build_components”: Ez a tábla tartalmazza az összes, előre össze szerelt gép alkatrészeit, legyen szó akár az általunk, akár a felhasználó által összeállított konfigurációról. Kapcsolódik hozzá a „components” tábla a „build_id -> id” mentén, valamint a „custom_builds” a „build_id -> build_id” mentén.

Custom_builds	
build_id	int
user_id	int
name	varchar(200)
status	varchar(20)
total_price	numeric(12,2)
components_json	nvarchar(max)
created_at	datetimeoffset

- l. „custom_builds”: Ez a tábla tartalmazza a felhasználó által összeállított konfigurációt. Kapcsolódik a „users” táblához a(z) „user_id -> id” mentén, valamint a már fentebb említett módon a „build_components” táblához.

Deals	
id	int
name	nvarchar(50)
description	nvarchar(50)
price	numeric(10,2)

- m. „Deals”: Ez a tábla tartalmazza az oldalunkon aktuálisan elérhető kedvezményeit, csomagjait vagy kiemelt termékeit. Ez a tábla nem kapcsolódik semelyik másik táblához.

Megjegyzés: az adatbázis némelyik táblája, így az „orders”, az „order_items_p” és az „addresses” egyelőre nem került felhasználásra, mivel ezek a táblák, a program jelenlegi állapotát nézve részei az egyik fentebb említett fejlesztési lehetőséghez tartoznak, viszont már beépítésre kerültek az adatbázisba.

Feladatok felosztása

Ebben a pontban felsoroltuk, kinek mi volt a feladata.

Magyar Csaba:

- API fejlesztése, minden táblára lebontva (nem Generic API-t használva)
- Adatbázis szerkezet létrehozása és továbbfejlesztése
- Főoldal kialakítása (design-t nem bele értve)
- Projekt rendszerezése
- Admin panel létrehozása
- Fejlesztői tesztelés

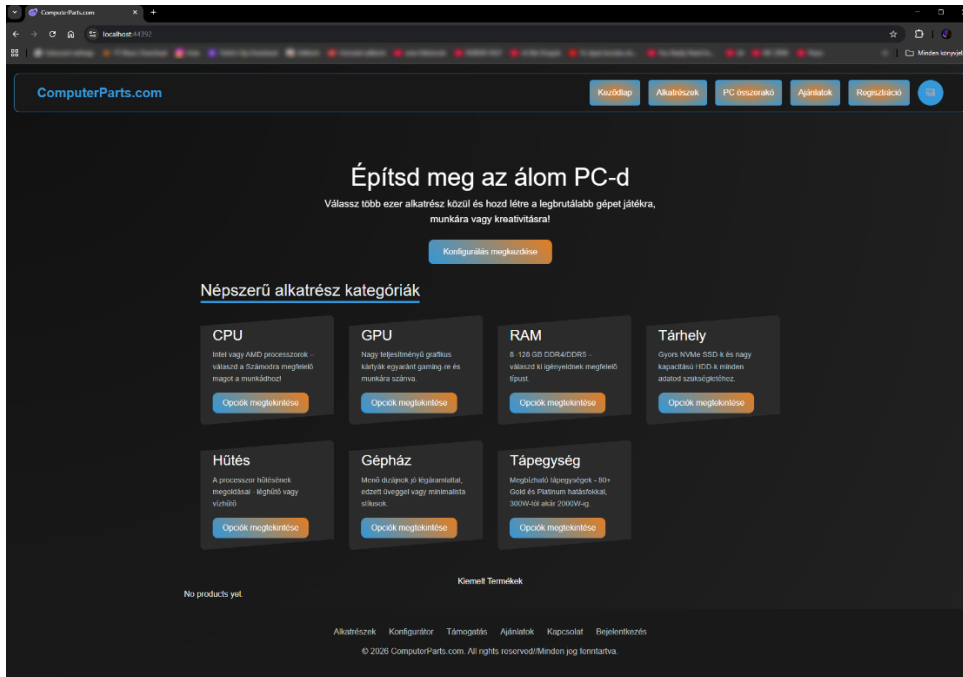
Rebe Dániel:

- Alap API létrehozása, mindent külön kezelve (nem Generic API-t használva)
- Frontend összekötése a Backend-del
- Frontend design kialakítása
- Frontend összes oldalának átültetése C-Sharp (C#) környezetbe
- Adatbázis szerkezet rendszerezése
- Alap projekt létrehozása
- Felhasználói tesztelés (mind egyszerű felhasználó, mind admin téren)

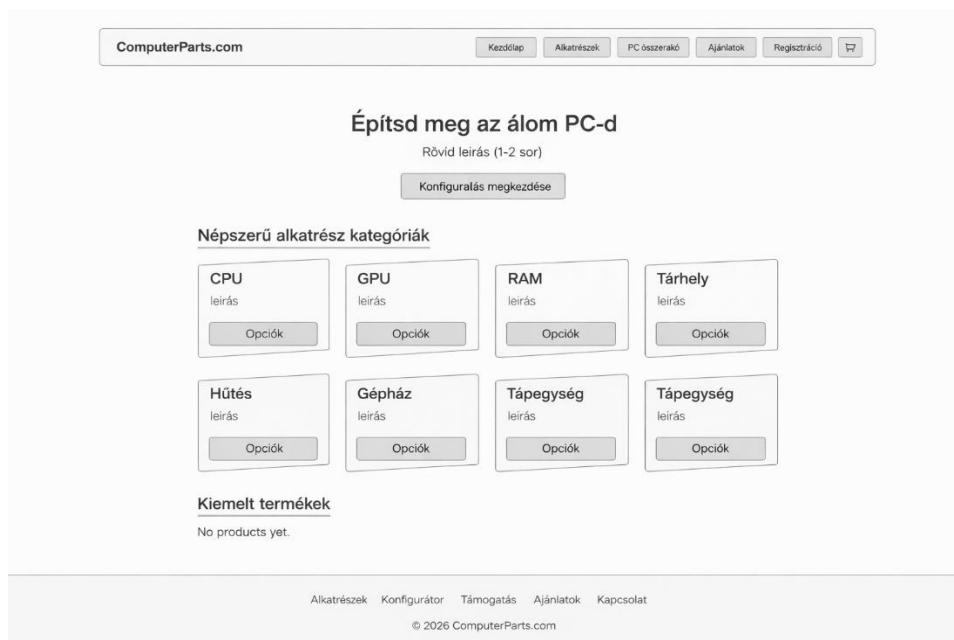
Drótvázak, valamint a design-ok képekben

Weboldalunk drótvázáról, valamint design-járól készült kép is, amit itt láthat. Oldalunk nem egyszer változott design-ját érteve, valamint mivel a később elkészült oldalakról még nem készült sem kép, sem drótváz, így több oldalról készült kép és drótváz került beszurásra:

Főoldal design:



Főoldal drótváz:



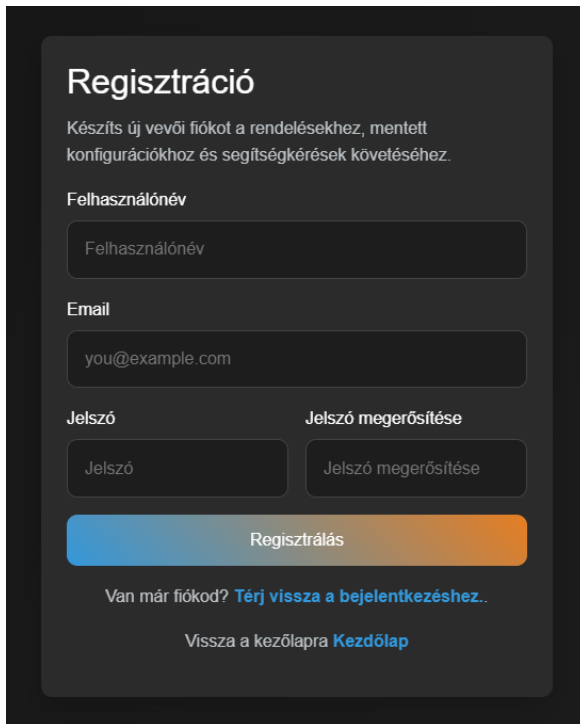
Konfiguráció összeállító design:

The dark-themed interface features a top navigation bar with links: Kezdőlap, Alkatrészek, PC összesítő, Ajánlatok, and Regisztráció. The main heading is "PC Konfigurátor" with a subtext: "Rakjon össze egy kivánságszerű PC-t az elérhető összetevők közül és a jobb oldali panelen ellenőrizze végösszegét." The configuration area is split into two columns. The left column, titled "Válassz ki a megfelelő alkatrészeket", contains dropdown menus for CPU, GPU, Memória, Tárhely, CPU hűtő, Gépház, and Tápegység, each with a "Válassz alkatrészt..." placeholder. Below these is a "Megjegyzés" field with a placeholder "Adj hozzá egyéb kérést vagy preferenciát itt..." and a "Konfiguráció mentése" button. The right column, titled "Várható végösszeg", lists the selected components with minus icons for removal. The total "Végösszeg" is shown as "0.00". A note at the bottom states: "A végösszeg változhat a jelenlegi piaci árakhoz igazítva...". A footer contains links: Alkatrészek, Konfigurátor, Támogatás, Ajánlatok, Kapcsolat, and Bejelentkezés, along with the copyright "© 2026 ComputerParts.com".

Konfiguráció összeállító drótváz:

The light-themed wireframe mirrors the structure of the dark theme. The top navigation bar includes "Kezdőlap", "Alkatrészek", "PC összesítő", "Ajánlatok", "Regisztráció", and a shopping cart icon. The main heading "PC Konfigurátor" is followed by the same subtext: "Rakjon össze egy kivánságszerű PC-t az elérhető összetevők közül és a jobb oldali panelen ellenőrizze végösszegét." The left column, "Válassz ki a megfelelő alkatrészeket", features dropdown menus for CPU, GPU, Memória, Tárhely, CPU hűtő, Gépház, and Tápegység, each with a "Válassz alkatrészt..." placeholder. Below is a "Megjegyzés" field with a placeholder "Adj hozzá egyéb kérést vagy preferenciát itt..." and a "Konfiguráció mentése" button. The right column, "Várható végösszeg", lists components with minus icons. The total "Végösszeg" is "0.00". A note at the bottom states: "A végösszeg változhat a jelenlegi piaci árakhoz igazítva...". The footer contains links: Alkatrészek, Konfigurátor, Támogatás, Ajánlatok, Kapcsolat, and Bejelentkezés, along with the copyright "© 2026 ComputerParts.com".

Regisztrációs oldal design:



Regisztráció

Készíts új vevői fiókot a rendelésekhez, mentett konfigurációkhoz és segítségkérések követéséhez.

Felhasználónév

Email

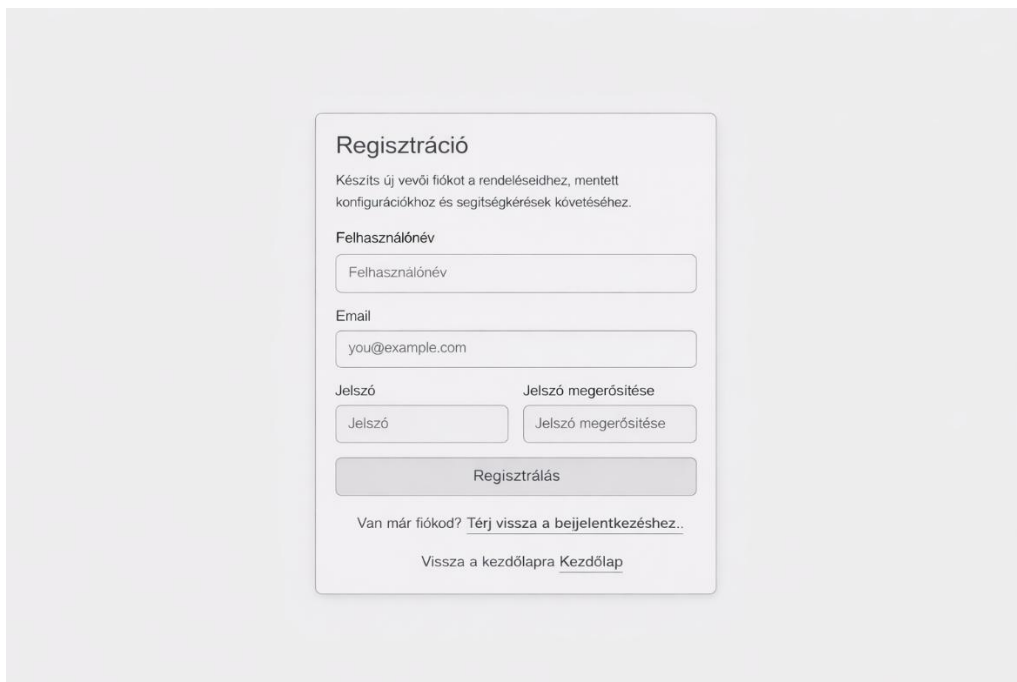
Jelszó **Jelszó megerősítése**

Regisztrálás

Van már fiókod? [Térj vissza a bejelentkezéshez..](#)

Vissza a kezdőlapra [Kezdőlap](#)

Regisztrációs oldal drótváz:



Regisztráció

Készíts új vevői fiókot a rendeléseidhez, mentett konfigurációkhoz és segítségkérések követéséhez.

Felhasználónév

Email

Jelszó **Jelszó megerősítése**

Regisztrálás

Van már fiókod? [Térj vissza a bejelentkezéshez..](#)

Vissza a kezdőlapra [Kezdőlap](#)

Megjegyzés: az oldalak design-jairól készült képek a Windows-ba épített képmetszővel készültek, a drótváz képek generatív AI-t használva jöhettek létre, azonban a mesterséges intelligencia nem változtatott a drótvázakon tartalmat tekintve.

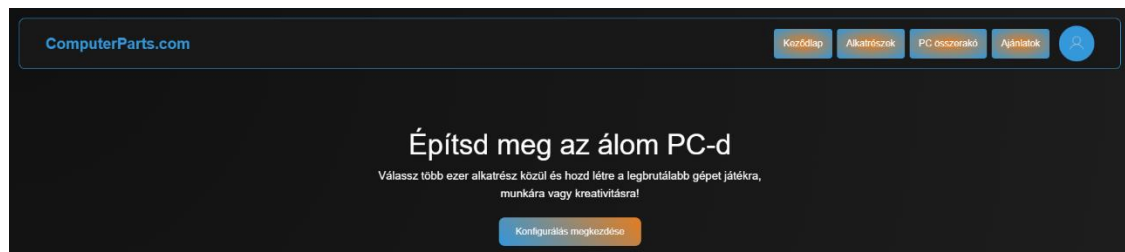
Tesztelés leírása, tesztdokumentáció

A tesztelés során több, különböző környezetben, operációs rendszeren, böngészőben és képfelbontásban lett kipróbálva annak érdekében, hogy bizonyosdjunk arról, hogy a programunk bármilyen környezetben zökkenőmentesen, reszponzívan működjön a teljes program. A tesztelést legfőképp Windows 10 és Windows 11-es környezetben teszteltük, aminek egyszerű oka, hogy ezen az operációs rendszeren készült weboldalunk fejlesztése, azonban mobil eszközön, Android rendszeren is tesztelésre került, ahol a reszponzivitás, valamint funkcionalitás szintén tökéletesen működött. Oldalunk reszponzivitásáért bootstrap.css stíluslap felel, ennek használatával értük el, hogy weboldalunk navigációs sávja (ún. Navbar-ja) kisebb felbontásokon, vagy jobban össze nyomott kijelzőkön is zökkenő mentesen működjön, lenyíló menüként, valamint az oldal összes szekciója, leírása, gombja és felülete is igazodni tudjon a különböző képernyő méretekhez és felbontásokhoz.

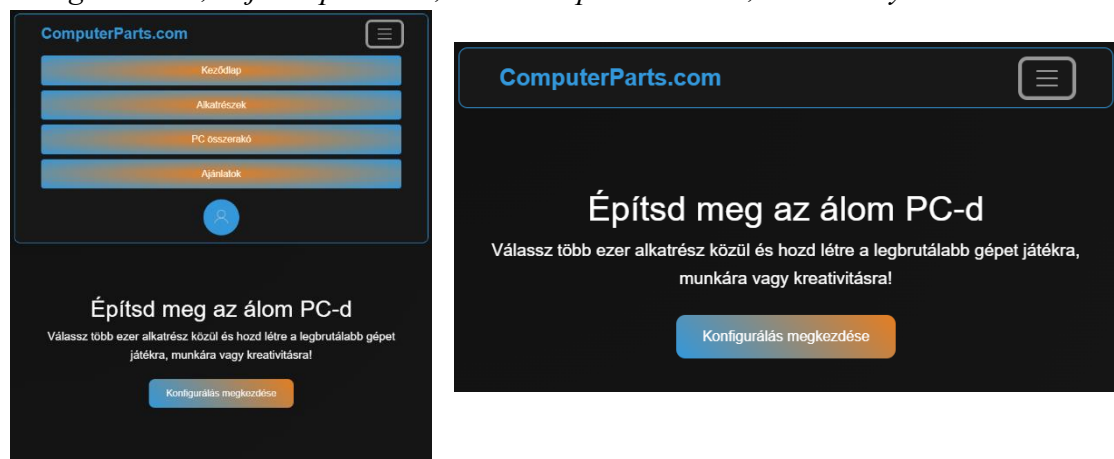
Navigációs sáv kódja, bootstrap elemeket tartalmazva:

```
5 <nav class="navbar navbar-expand-xl navbar-dark fixed-top shadow-sm" style="border-radius: 10px; border: 1px solid #3498db; margin: 1%;">
6 <div class="container-fluid">
7 <Nav.Link href="#" class="logo nav-link">ComputerParts.com</Nav.Link>
8
9 @* <a href="#" class="logo">ComputerParts.com</a> *@
10 <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#mainNavbar" aria-controls="mainNavbar" aria-expanded="false">
11 | <span class="navbar-toggler-icon"></span>
12 </button>
13 <div class="collapse navbar-collapse d-xl-flex flex-grow-0 gap-3" id="mainNavbar">
14 | <div class="d-none d-xl-flex gap-2">
15 | | <a href="#" class="navbutton">Kezdőlap</a>
16 | | <a href="parts" class="navbutton">Alkatrészek</a>
17 | | <a href="configurator" class="navbutton">PC összeszerelő</a>
18 | | <a href="deals" class="navbutton">Ajánlatok</a>
19 | </div>
```

Navigációs sáv, teljes képtárlóban, Windows 11 op. rendszeren:



Navigációs sáv, teljes képtárlóban, Android op. rendszeren, nav. sáv nyitva és csukva:



Maga a navigációs sáv egy nagyon egyszerű módon működik: a felhasználónak szimplán kattintással kell kiválasztania, melyik oldalt szeretné elérni, a rendszer pedig automatikusan átirányítja a keresett oldalra. Ezen a fülön találja a felhasználó a profil gombot is, amin meg tudja tekinteni a már mentett konfigurációit, valamint ki is tud jelentkezni fiókjából.

```

125 @code {
126     private bool showProfileDropdown = false;
127     protected override async Task OnInitializedAsync()
128     {
129         await base.OnInitializedAsync();
130         // Feliratosítás a hitelesítési állapot változásaira, hogy a navigáció frissüljön a bejelentkezés/kijelentkezéskor
131         try
132         {
133             AuthService.AuthStateChanged += OnAuthStateChanged;
134         }
135         catch
136         {
137             // Egyszerűen figyelmen kívül hagyható, ha a szolgáltatás még nem áll felfrissen - a "StateHasChanged" meghívásával biztosítjuk, hogy amint elérhető lesz, frissül a komponens
138         }
139
140         // Meg kell bizonyosodni, hogy a service a "localStorage"-ot olyan hamar olvassa, amennyire csak lehet, hogy a komponensek időben, a helyes státuszban jelenjenek meg
141         try
142         {
143             await AuthService.InitializeAsync();
144         }
145         catch { }
146         StateHasChanged();
147     }
148
149     private void OnAuthStateChanged()
150     {
151         InvokeAsync(StateHasChanged);
152     }
153
154     private void ToggleProfileDropdown()
155     {
156         showProfileDropdown = !showProfileDropdown;
157     }
158
159     private async System.Threading.Tasks.Task OnLogoutClicked()
160     {
161         if (AuthService != null)
162         {
163             await AuthService.LogoutAsync();
164         }
165         Navigation.NavigateTo("/", true);
166     }
167
168     // "OnAfterRender" üzenet tartása - az inicializáció végén vagy az "OnInitializedAsync" metóduson
169 }

```

A bejelentkező és regisztrációs oldal esetében a navigációs sávot illetve footer-t először meg kellett akadályozni, hogy ezeken az oldalakon megjelenhessen. Ezt azzal oldottuk meg, hogy a „NavMenu.razor” file-ban létrehoztunk egy olyan feltételvizsgáló metódust („predicate method”), amely azt vizsgálja, egy adott oldalon megjelenhet-e a navigációs sáv, illetve a footer.

Navigációs sáv és footer megjelenítési „engedélyező” kódja:

```

private bool ShouldShowNav()
{
    var relative = Navigation.ToBaseRelativePath(Navigation.Uri);
    if (string.IsNullOrEmpty(relative))
        return true; // home
    var path = relative.Split('?')[0].Trim('/');
    return !string.Equals(path, "login", StringComparison.OrdinalIgnoreCase)
        && !string.Equals(path, "register", StringComparison.OrdinalIgnoreCase);
}

```

A regisztrációs oldalon több, különböző adatot is ellenőriz a rendszer, ezek közé tartozik, hogy a felhasználó adott-e felhasználónevet, email címet, jelszót és megerősítette-e a jelszavát, avagy kétszer megadta-e a jelszavát regisztrációkor, emellett külön ellenőrzés történik, hogy a felhasználó által megadott jelszó legalább nyolc (8) karakter hosszú. Ha valamelyik ezek közül hiányzik, nem egyezik vagy nincs meg a megfelelő minimum hossz, a rendszer különböző hibaüzenetekkel jelzi, ha valami nem megfelelő. Ha mindent rendben talál, akkor a felhasználót rövid késleltetéssel a rendszer átirányítja a bejelentkező felületre, ahol a felhasználónak már csak az email címét vagy felhasználónevét, illetve jelszavát kell megadnia, amivel megtörtént a regisztráció.

```
52     {
53         private RegisterFormModel Model { get; set; } = new();
54         private bool IsSubmitting { get; set; }
55         private string? StatusMessage { get; set; }
56
57         private class RegisterFormModel
58         {
59             public string FirstName { get; set; } = string.Empty;
60             public string LastName { get; set; } = string.Empty;
61             public string Email { get; set; } = string.Empty;
62             public string Password { get; set; } = string.Empty;
63             public string ConfirmPassword { get; set; } = string.Empty;
64         }
65
66         private async Task RegisterAsync()
67         {
68             StatusMessage = null;
69
70             // Alapvető, közös oldali validáció a gyökeri hibák elkerüléséhez
71             if (string.IsNullOrWhiteSpace(Model.FirstName) || string.IsNullOrWhiteSpace(Model.LastName))
72             {
73                 StatusMessage = "Kérlek, add meg a keresz- és vezetéknéved!";
74                 return;
75             }
76
77             if (string.IsNullOrWhiteSpace(Model.Email))
78             {
79                 StatusMessage = "Kérlek, adj meg egy email címet!";
80                 return;
81             }
82
83             if (string.IsNullOrEmpty(Model.Password) || Model.Password.Length < 8)
84             {
85                 StatusMessage = "A jelszónak minimum nyolc (8) karakter hosszúnak kell lennie!";
86                 return;
87             }
88
89             if (Model.Password != Model.ConfirmPassword)
90             {
91                 StatusMessage = "A jelszavak nem egyeznek!";
92                 return;
93             }
94
95             IsSubmitting = true;
96
97             try
98             {
99                 // Felhasználónév kinyerése keresz- és vezetéknévéből, szóközök eltávolítása és kisbetűsítés (tartalmak az email local-partje)
100                 var username = (Model.FirstName + Model.LastName).Replace(" ", "").ToLowerInvariant();
101                 if (string.IsNullOrWhiteSpace(username))
102                 {
103                     var at = Model.Email.IndexOf('@');
104                     username = at > 0 ? Model.Email.Substring(0, at) : Model.Email;
105                 }
106
107                 var payload = new
108                 {
109                     Username = username,
110                     Email = Model.Email,
111                     Password = Model.Password
112                 };
113
114                 var response = await Http.PostAsJsonAsync("api/auth/register", payload);
115             }
```

```
116             if (response.IsSuccessStatusCode)
117             {
118                 // Sikeres regisztráció - átirányítás a bejelentkező oldalra
119                 StatusMessage = "Sikeres regisztráció. Átirányítás a bejelentkező oldalra...";
120                 // small delay to let user read message
121                 await Task.Delay(800);
122                 Navigation.NavigateTo("/login");
123             }
124             else
125             {
126                 // Hibaüzenet kivonása a válaszból, ha elérhető
127                 try
128                 {
129                     var errorObj = await response.Content.ReadFromJsonAsync<ServerError?>();
130                     StatusMessage = errorObj?.message ?? "Sikertelen regisztráció. Kérjük, próbáld újra!";
131                 }
132                 catch
133                 {
134                     StatusMessage = "Sikertelen regisztráció. Kérjük, próbáld újra!";
135                 }
136             }
137         }
138         catch (Exception ex)
139         {
140             StatusMessage = "An error occurred: " + ex.Message;
141         }
142         finally
143         {
144             IsSubmitting = false;
145         }
146     }
147
148     private class ServerError
149     {
150         public string? message { get; set; }
151     }
152 }
153
```

Onnantól, hogy megtörtént a regisztráció és a rendszer automatikusan átírányította a felhasználót a bejelentkező oldalra, mint fentebb is írtuk, már csak a regisztrációkor megadott felhasználónév vagy email cím, és jelszó párost kell megadni, így a felhasználó már teljes egészében elérheti az oldal által nyújtott funkciókat. A bejelentkező oldal itt már csak a megadott email címet vagy felhasználónevet, illetve a hozzá tartozó jelszót fogja ellenőrizni API segítségével az adatbázisból. Ha ezek közül valamelyik nem egyezik meg egyikkel sem, ami megtalálható az adatbázison, azt az alábbi képen/képeken látható hibaüzenetekkel fogja a felhasználó tudomására adni. *Megjegyzés: az oldal bejelentkezés nélkül is használható, egyes egyedül a regisztrációhoz kötött funkciókat, mint a felhasználó által összerakott konfiguráció mentése, illetve a mentett konfigok megtekintése nem lesz elérhető.*

```
38 @code {
39     private LoginFormModel Model { get; set; } = new();
40     private bool IsSubmitting { get; set; }
41     private string? StatusMessage { get; set; }
42
43     private class LoginFormModel
44     {
45         public string UsernameOrEmail { get; set; } = string.Empty;
46         public string Password { get; set; } = string.Empty;
47     }
48
49     private async Task LoginAsync()
50     {
51         StatusMessage = null;
52
53         if (string.IsNullOrEmpty(Model.UsernameOrEmail))
54         {
55             StatusMessage = "Kérjük, add meg a felhasználóneved vagy email címed!";
56             return;
57         }
58
59         if (string.IsNullOrEmpty(Model.Password))
60         {
61             StatusMessage = "Kérjük, add meg a jelszavad!";
62             return;
63         }
64
65         IsSubmitting = true;
66
67         try
68         {
69             var success = await AuthService.LoginAsync(Model.UsernameOrEmail, Model.Password);
70
71             if (success)
72             {
73                 StatusMessage = "Sikeres bejelentkezés. Átírányítás...";
74                 await Task.Delay(400);
75                 // force reload so other components pick up persisted token
76                 Navigation.NavigateTo("/", true);
77             }
78             else
79             {
80                 StatusMessage = "Bejelentkezés sikertelen. Ellenőrizd a hitelesítő adatokat.";
81             }
82         }
83         catch (Exception ex)
84         {
85             StatusMessage = "An error occurred: " + ex.Message;
86         }
87         finally
88         {
89             IsSubmitting = false;
90         }
91
92         private class ServerError
93         {
94             public string? message { get; set; }
95         }
96     }
97 }
```

Oldalunk fő funkciója, a konfiguráció összeállító ez után elérhetővé válik, akár ha regisztrál és bejelentkezik a felhasználó, akár ha csak „vendégként”, saját felhasználói fiók nélkül szeretné használni az oldalunkat. Ekkor tárul elé a főoldalunk, valamint a teljes navigációs sáv, amin a különböző, többi oldal tartalmát és funkcióját éri el a felhasználó. Ezek közé tartozik az „Alkatrészek” összegyűjtő oldala (*kizárólag azokkal, amelyek megtalálhatóak adatbázisunkban*), az „Ajánlatok” fül, ahhol a különböző, kiemelt ajánlatok, promóciók vagy kedvezményes áron elérhető termék vagy termék csomagok jelennek meg, valamint a „PC Összerakót”, ami oldalunk jelenlegi fő funkciója. Itt a felhasználó összeállíthatja a számára megfelelő PC konfigurációt, processzortól kezdve videokártyán és memórián át, tárhelykapacitáson, gépházon keresztül, egészen processzor hűtőig és tápegységig mindent. Mindegyik alkatrésznél lenyíló menü szerű választó van, ha pedig a felhasználó kiválasztja tegyük fel a neki megfelelő videokártyát (*GPU-t*), az a jobb oldali kisebb ablakon meg fog jelenni, a termék árával együtt. Ha a felhasználó egy teljes konfigurációt állít össze, a rendszer automatikusan össze adja a termékek árait. *Megjegyzés: az adatbázisunkban megtalálható árak kizárólag átlagárak, a különböző webáruházakon eltérhetnek az árak.*

[illegible]

Főoldalunkon az aktuálisan elérhető összes termék típus jelenik meg, valamint lejjebb görgetve néhány kiemelt termék, ezeknek a leírása, valamint az árazása. A kategóriák ablakai mind külön-külön div-ekként kerültek megjelenítésre egy egyszerű „ForEach” loop-ot alkalmazva jeleníti meg ahelyett, hogy egyesével, teljes egészében legyenek a kódsorok megírva. Jelen pillanatban mindegyik kategória két-két terméket tartalmaz, a megjelenítést pedig az alábbi kód részlet végzi:

```
private async Task DebugAndLoad(int typeId)
{
    // immediate visible feedback for debugging
    errorMessage = $"Gomb megnyomva: {typeId}";
    isLoading = true;
    StateHasChanged();
    // also log to browser console
    try { await JS.InvokeVoidAsync("console.log", $"Button clicked for type {typeId}"); } catch { }
    await LoadComponentsByType(typeId);
    // show modal after loading
    showBackdrop = true;
    StateHasChanged();
    try { await JS.InvokeVoidAsync("showModal", "componentsModal"); } catch { }
}
```

(Megjegyzés: az Admin oldal leírása és kódjáról készült képek a fejlesztői dokumentáció végén kaptak helyet, az eddigi itt történő elhelyezésük helyett, ugyan is egy sokkal részletesebb ismertetés készült hozzá. Keresse a 76. oldalon!)

Oldalunk backend részéről is van mit, illetve érdemes is írni, mivel egy egészen összetett, viszont számunkra könnyedén átlátható és rendszerezett projektstruktúra épült fel. Kezdjük első körben weboldalunk Library-jével: itt található meg az össze olyan kódrészlet, amik a frontend-től érkező kéréseket telejsítik. Először a service-ekről (példa kódrészlet a képen az „AddressService” osztályból származik, illetve minden példa kódrészlet az ennek megfelelő névvel ellátott osztályokból származik): A Service-ek amolyan közvetítő szerepet töltenek be a DbContext és a Controller-ek között. Ezeket használva az adatkezelési logika egységes, jól tesztelhető lesz, és nem keveredik a felhasználói felület kiszolgálásáért felelős kódrészletekkel. A projektben megtalálható összes Service Interface alapú tervezésben készült, így mindegyik Service egy-egy Interface-t valósít meg, így a függőségek könnyen cserélhetőek az egység tesztek során. Az adatbázis kontextusát a keretrendszer injektálja konstruktoron keresztül, nem pedig a class-on belül példányosul, így optimálisan kezeli az erőforrások életciklusait. Valamint mindegyik Service aszinkronként működik, így egyetlen szál sem fog blokkolódni egy-egy adatbázis-művelet alatt. Öt metódussal dolgozunk a legtöbb Service-ben: kétféle adatlekérés van, ezek közül a „GetAllAsync” az összes adatot írja ki, a „GetByIdAsync” pedig Id-ra keresve adja ki a megfelelő adatot/adatsort, ezen felül található hozzáadás, módosítás és törlés metódus, amiket szerintem nem kell egyesével bemutatni, mire jók, a nevük tökéletesen megmagyarázza mindet.

(Megjegyzés: ez a kép az előző oldal alján található leíráshoz tartozik.)

```
namespace ComputerParts.Library.SERVICE
{
    [assembly: ModelBinder(typeof(AspNetModelBinder))]
    public class AddressService : IAddressService
    {
        private readonly ComputerPartsDbContext _context;
        private readonly IComputerPartsContactService _contactService;
        private readonly IComputerPartsAddressService _addressService;

        public AddressService(ComputerPartsDbContext context, IComputerPartsContactService contactService, IComputerPartsAddressService addressService)
        {
            _context = context;
            _contactService = contactService;
            _addressService = addressService;
        }

        [HttpPost]
        public async Task AddAddressAsync(Address address)
        {
            await _addressService.AddAddressAsync(address);
            _context.SaveChanges();
        }

        [HttpDelete]
        public async Task DeleteAddressAsync(int id)
        {
            var entity = await _addressService.Addresses.FindAsync(id);
            if (entity != null)
            {
                _addressService.Addresses.Remove(entity);
                _context.SaveChanges();
            }
        }

        [HttpGet]
        public async Task<IEnumerable<Address>> GetAllAddressesAsync()
        {
            var addresses = await _addressService.Addresses.ToListAsync();
            return addresses;
        }

        [HttpPut]
        public async Task UpdateAddressAsync(Address address)
        {
            _addressService.Addresses.Update(address);
            _context.SaveChanges();
        }
    }
}
```

A „UserService” az egyik olyan kivétel, ahol ötnél több metódust alkalmazunk. Itt a fentebb már említett Get, Put, Update és Delete metódusokon, avagy a standard CRUD műveleteken felül néhány komplexebb adatvédelmi és integritási logikát is tartalmaz. A Service a megvalósítás során kiemelten figyel az adatok integritására is.

```

namespace ComputerPartsLibrary.Services
{
    [Authorize(Roles = "Admin")]
    public class UserService : IUserService
    {
        private readonly ComputerPartsDbContext _context;
        private readonly JwtTokenService _jwtTokenService;

        UserService(ComputerPartsDbContext context, JwtTokenService jwtTokenService)
        {
            _context = context;
            _jwtTokenService = jwtTokenService;
        }

        [Authorize(Roles = "Admin")]
        public async Task AddUserAsync(User user)
        {
            if (user == null) throw new ArgumentNullException(nameof(user));

            // Basic validation
            if (string.IsNullOrWhiteSpace(user.UserName)) throw new ArgumentException("Username cannot be empty", nameof(user.UserName));
            if (string.IsNullOrWhiteSpace(user.Email)) throw new ArgumentException("Email cannot be empty", nameof(user.Email));

            // prevent duplicates by username or email (case-insensitive)
            var exists = await _context.Users.AnyAsync(u => u.UserName.ToLower() == user.UserName.ToLower() || u.Email.ToLower() == user.Email.ToLower());
            if (exists) throw new ArgumentException("Username or email already registered");

            await _context.Users.AddAsync(user);
            await _context.SaveChangesAsync();
        }

        [Authorize(Roles = "Admin")]
        public async Task DeleteUserAsync(int id)
        {
            var entity = await _context.Users.FindAsync(id);
            if (entity == null) return;

            // prevent deleting Admin users
            if (string.Equals(entity.Role, "Admin", StringComparison.OrdinalIgnoreCase))
            {
                throw new InvalidOperationException("Cannot delete users with Admin role.");
            }

            // Remove related Addresses
            try
            {
                var adrs = _context.Addresses.Where(a => a.UserId == id).ToList();
                if (adrs.Any())
                {
                    _context.Addresses.RemoveRange(adrs);
                }
            }
            catch { }

            // Remove related Custom Builds
            try
            {
                var builds = _context.CustomBuilds.Where(b => b.UserId == id).ToList();
                if (builds.Any())
                {
                    _context.CustomBuilds.RemoveRange(builds);
                }
            }
            catch { }

            // Remove orders and their items
            try
            {
                var orders = _context.Orders.Where(o => o.UserId == id).ToList();
                foreach (var order in orders)
                {
                    var items = _context.OrderItems.Where(i => i.OrderId == order.Id).ToList();
                    if (items.Any()) _context.OrderItems.RemoveRange(items);
                }
            }
            catch { }

            _context.Users.Remove(entity);
            await _context.SaveChangesAsync();
        }
    }
}

```

Ezen a képen láthatók a fentebb említett alap CRUD műveletek, valamint azoknak több, ellenőrzést végző része is. Ha a felhasználó regisztráció után közvetlenül jelentkezik be fiókjába, az „AddUserAsync” metódus nem csak egyszerűen az alapvető mezők meglétét ellenőrzi, hanem aszinkron módon vizsgálja az adatbázisban megtalálható felhasználónév és e-mail cím egyediségét (például a kis- és nagybetűket), ami megakadályozza, hogy két ugyan olyan felhasználónévvel rendelkező fiók legyen regisztrálva. A felhasználók törlését végző „DeleteUserAsync” metódusban pedig egy olyan biztonsági ellenőrzés van jelen, amely biztosítja, hogy „Admin” szerepkörrel ellátott felhasználót ne lehessen még véletlenül se törölni. Itt még az adatintegritás és törlési láncáról érdemes szót ejteni: oldalon funkcióiból adódóan a felhasználókhoz még számos másik funkció is kapcsolódik, így amikor a „DeleteUserAsync” metódus véghez viszi a törlést, nem csak a felhasználó adatai kerülnek eltávolításra, mint az e-mail cím, jelszó, felhasználónév, hanem még előtte megtörténik a hozzá rendelt címek, egyedi konfigurációk („Custom_builds” táblából), rendelések és a hozzájuk tartozó tételek (megjegyzés: a rendelések még nincsenek teljesen implementálva a kódba, hiszen mint fentebb is írtuk, az oldal egyelőre nem webshopként üzemel, valamint a rendelésekhez tartozó tételek csak abban az értelemben törölődnek, hogy többé nem fog rendelésként megjelenni, a termékek nem törölődnek az adatbázisból). Emellett a beépített try-catch metódus biztosítja egy olyan hibakezelési blokkként a sikeres lefutást, amely ellenőrzi, hogy ha esetleg bizonyos kapcsolódó táblákban nincsenek adatok, akkor is zökkenő mentesen futhasson le a törlés.


```

        // @throws - service Order items is shared() = 1 order id - and order id's nullified,
        // if (isEmpty(y)) { service Order_order_item_b_remove(order_id); }
        service Orders_remove(order);
    }
}

catch {}

// finally remove user
service Users_remove(entity);
email service User_remove(entity);
}

// @throws - change 2 authors 2 changes
public async Task<User> GetUserIdByPage(int id)
{
    var user = email _service Users_FindPage(id);
    return user;
}

// @throws - change 2 authors 2 changes
public async Task<User> GetByIdAsync(int id)
{
    var user = email _service Users_FindById(id);
    return user;
}

// @throws - change 2 authors 2 changes
public async Task<User> GetByIdAsync(Guid user)
{
    var user = email _service Users_FindById(user);
    email _service User_remove(entity);
}

// @throws - change 2 authors 2 changes
// @throws - 2 JWT token for the given user
// @throws
// @throws
public async Task<User> GenerateTokenAsync(int userId, string role)
{
    // find user from database to include all user data into token
    var user = email _service Users_FindPage(userId);
    if (user == null) throw new InvalidOperationException("user not found");
    return _getTokenService.GenerateToken(user);
}

// @throws
// @throws 2 JWT token and returns the user claim
// @throws
public async Task<User> GenerateToken()
{
    email _service User_remove(entity);
    try
    {
        return _getTokenService.ValidateToken(token);
    }
    catch (InvalidOperationException ex)
    {
        return null;
    }
}

```

Emellett pedig egy komoly biztonsági rendszert is tartalmaz a kód. Ezen a képen látható, ahogyan a regisztráció, majd a bejelentkezés megtörténése után a rendszer JWT token generál a felhasználónak. Ez a folyamat során a rendszer lekéri a bejelentkezni kívánó felhasználó összes adatát az adatbázisból, majd ennek segítségével a biztonsági token tartalmazni fogja a szükséges claim-eket, mint a szerepkört. Erre a „GenerateTokenAsync” metódus lett kirendelve, ami folyamatosan együttműködik a dedikált „JwtTokenService”-szel. Ez után pedig megtörténik a token validációja, amit a „ValidateTokenAsync” metódus végez. Ez ellenőrzi a beérkező kérések hitelességét, majd visszaadja a felhasználó azonosításához szükséges „ClaimsPrincipal” objektumot.

És akkor ha már szóba került a „JwtTokenService” class is, ezt mindenképp kiemelten fontosnak tartjuk, hogy részletes leírással ismertethessük, ugyan is ez a szolgáltatás felelős a JSON Web Tokenek (JWT) előállításáért, aláírásáért és validálásáért. A megvalósítás során az iparági sztenderdnek számító HS256 algoritmust alkalmaztuk a tokenek integritásának védelmére.

Konfiguráció és Inicializálás

A szolgáltatás rugalmasságát az appsettings.json fájlból beolvasott konfigurációs értékek adják.


```
namespace ComputerpartsLibrary.SERVICE
{
    /// <summary>
    /// JWT (JSON Web Token) token generálását és validálását felelős szolgáltatás
    /// </summary>
    6 references: Magyar, 1 hour ago, 1 author, 5 changes
    public class JwtTokenService
    {
        private readonly string? _secretKey;
        private readonly string? _issuer;
        private readonly string? _audience;
        private readonly int _expireMinutes;

        /// <summary>
        /// Generates a JWT token for the given user
        /// </summary>
        0 references: Magyar, 14 days ago, 1 author, 1 change
        public JwtTokenService(IConfiguration config)
        {
            // Read settings from configuration (appsettings.json)
            var section = config.GetSection("Jwt");
            _secretKey = section["Key"];
            _issuer = section["Issuer"];
            _audience = section["Audience"];

            if (!int.TryParse(section["ExpireMinutes"], out _expireMinutes))
            {
                // fallback to 60 minutes
                _expireMinutes = 60;
            }
        }

        // Generate token using full user object so we can include all user data as claims
        3 references: Magyar, 1 day ago, 1 author, 1 change
        public JwtToken GenerateToken(ComputerpartsLibrary.MODEL.Users user)
        {
            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.NameIdentifier, user.id.ToString()),
                new Claim(ClaimTypes.Role, user.role ?? string.Empty),
                new Claim("authEmail", user.email ?? string.Empty),
                new Claim("authUsername", user.username ?? string.Empty),
                new Claim("authUserId", user.id.ToString()),
                // store created_at in ISO 8601 format
                new Claim("authCreatedAt", user.created_at.ToString("o"))
            };

            var expires = DateTime.UtcNow.AddMinutes(_expireMinutes);

            var securityKey = CreateSymmetricKey();
            var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

            var jwt = new JwtSecurityToken(
                issuer: _issuer,
                audience: _audience,
                claims: claims,
                expires: expires,
                signingCredentials: credentials
            );

            var handler = new JwtSecurityTokenHandler();
            var tokenString = handler.WriteToken(jwt);

            return new JwtToken
            {
                Token = tokenString,
                Expiration = new DateTimeOffset(expires),
                UserId = user.id,
                Role = user.role ?? string.Empty
            };
        }
    }
}
```

Dinamikus beállítások: A konstruktor az IConfiguration segítségével kinyeri a titkos kulcsot (Key), a kibocsátót (Issuer) és a célközönséget (Audience).

- **Lejáratí idő kezelése:** A rendszer támogatja a konfigurálható lejáratí időt, alapértelmezett (fallback) értéként 60 percet használva, ha a konfiguráció nem elérhető.
- **Claim-alapú azonosítás:** A GenerateToken metódus a felhasználó adatait (ID, név, e-mail, szerepkör) úgynevezett "Claim"-ekbe csomagolja, így a kliensoldal és a szerveroldali middleware is azonnal látja a jogosultsági szintet anélkül, hogy minden alkalommal az adatbázishoz kellene fordulnia.

Validáció és Hibakezelés

A biztonság alapja a beérkező tokenek szigorú ellenőrzése.

```
/// <summary>
/// Validates a JWT token and returns the user claims
/// </summary>
1 reference Magyar 1 day ago 1 author 4 changes
public ClaimsPrincipal ValidateToken(string token)
{
    try
    {
        var jwtHandler = new JwtSecurityTokenHandler();
        var tokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = CreateSymmetricKey(),
            ValidateIssuer = !string.IsNullOrEmpty(_issuer),
            ValidIssuer = _issuer,
            ValidateAudience = !string.IsNullOrEmpty(_audience),
            ValidAudience = _audience,
            ClockSkew = TimeSpan.Zero
        };

        var principal = jwtHandler.ValidateToken(token, tokenValidationParameters, out SecurityToken securityToken);
        return principal;
    }
    catch (Exception ex)
    {
        throw new UnauthorizedAccessException($"Token validation failed: {ex.Message}", ex);
    }
}

/// <summary>
/// Létrehoz egy JWT token stringet
/// </summary>
0 references Magyar 1 hour ago 1 author 2 changes
private string CreateJwtTokenString(Claim[] claims)
{
    var header = CreateHeader();
    var payload = CreatePayload(claims);
    var signature = CreateSignature(header, payload);

    return $"{header}.{payload}.{signature}";
}

/// <summary>
/// Létrehozza a JWT fejléct
/// </summary>
1 reference Magyar 1 hour ago 1 author 2 changes
private string CreateHeader()
{
    // A JsonSerializer statikus osztály - közvetlenül meghívjuk a Serialize()-t
    var headerJson = System.Text.Json.JsonSerializer.Serialize(new { alg = "HS256", typ = "JWT" });
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(headerJson));
}

/// <summary>
/// Létrehozza a JWT payload-ot
/// </summary>
1 reference Magyar 1 hour ago 1 author 2 changes
private string CreatePayload(Claim[] claims)
{
    var payload = new System.Collections.Generic.Dictionary<string, object>();
    foreach (var claim in claims)
    {
        payload[claim.Type] = claim.Value;
    }

    // A JsonSerializer statikus osztály - közvetlenül meghívjuk a Serialize()-t
    var payloadJson = System.Text.Json.JsonSerializer.Serialize(payload);
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(payloadJson));
}
```

Szigorú ellenőrzés: A ValidateToken metódus ellenőrzi az aláírás érvényességét, a kibocsátót és a lejáratási időt. A ClockSkew = TimeSpan.Zero beállítás biztosítja, hogy a lejáratási időt másodperc pontosan vegye figyelembe a rendszer, ne legyen "türelmi idő".

- **Hibakezelés:** Amennyiben a token módosítva lett vagy lejárt, a rendszer UnauthorizedAccessException-t dob, megvédve ezzel a védett végpontokat.

Alacsony szintű implementáció és Kriptográfia

A szolgáltatás tartalmaz segédmetódusokat is, amelyek a JWT specifikáció szerinti manuális felépítést vagy aláírást támogatják.

```

/// <summary>
/// Létrehozza a JWT aláírást
/// </summary>
1 reference Magyar, 1 hour ago 1 author, 2 changes
private string CreateSignature(string header, string payload)
{
    var message = $"{header}.{payload}";
    using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(_secretKey));
    var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(message));
    return Convert.ToBase64String(hash);
}

/// <summary>
/// Létrehoz egy szimmetrikus kulcsot az aláíráshoz
/// </summary>
2 references Magyar, 2 hours ago 1 author, 3 changes
private SymmetricSecurityKey CreateSymmetricKey()
{
    // 256-bites kulcsot állítunk elő a titoktól SHA256 használatával, így stabil marad újraindítások között
    using var sha = System.Security.Cryptography.SHA256.Create();
    var hash = sha.ComputeHash(Encoding.UTF8.GetBytes(_secretKey));
    return new SymmetricSecurityKey(hash);
}
}

```

Digitális aláírás: A CreateSignature metódus a HMAC-SHA256 algoritmust használja. Ez garantálja, hogy a token fejlécét (Header) és tartalmát (Payload) csak az a szervertudja validálni, amely ismeri a titkos kulcsot.

- **Biztonságos kulcskezelés:** A CreateSymmetricSecurityKey metódus a titkos kulcsból állít elő egy 256-bites hash-t, amely alapjául szolgál a titkosításnak, így biztosítva a stabil működést az alkalmazás újraindításai között is.

Az alkalmazás adatkezelési logikájának alapját az **Entity Framework Core** szolgáltatja. A rendszer és az SQL adatbázis közötti kapcsolatot a ComputerpartsDbContext osztály valósítja meg.

ComputerpartsDbContext osztály

Ez az osztály a DbContext osztályból származik, és központi szerepet játszik az adatok perzisztenciájában. Ez a komponens felelős az adatbázis-kapcsolat kezeléséért, a lekérdezések végrehajtásáért és a változások követéséért.

```

namespace Computerparts.Library.DATA
{
    38 references Magyar, 11 days ago 2 authors, 6 changes
    public class ComputerpartsDbContext : DbContext
    {
        0 references Magyar, 50 days ago 1 author, 1 change
        public ComputerpartsDbContext() { }
        0 references Magyar, 50 days ago 1 author, 1 change
        public ComputerpartsDbContext(DbContextOptions<ComputerpartsDbContext> options) : base(options) { }
        14 references Magyar, 32 days ago 2 authors, 3 changes
        public DbSet<User> Users { get; set; }
        5 references Magyar, 32 days ago 2 authors, 3 changes
        public DbSet<Categories> Categories { get; set; }
        6 references Magyar, 32 days ago 2 authors, 3 changes
        public DbSet<Products> Products { get; set; }
        8 references Magyar, 32 days ago 2 authors, 3 changes
        public DbSet<Orders> Orders { get; set; }
        7 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<Addresses> Addresses { get; set; }
        5 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<BuildComponents> BuildComponents { get; set; }
        8 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<CustomBuilds> CustomBuilds { get; set; }
        5 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<Components> Components { get; set; }
        6 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<ComponentType> ComponentType { get; set; }
        8 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<OrderItemsP> OrderItemsP { get; set; }
        8 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<OrderItemsB> OrderItemsB { get; set; }
        6 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<PrebuiltPcs> PrebuiltPcs { get; set; }
        6 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<PrebuiltPcComp> PrebuiltPcComp { get; set; }
        6 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<InventoryComponents> InventoryComponents { get; set; }
        6 references Magyar, 31 days ago 1 author, 1 change
        public DbSet<InventoryProducts> InventoryProducts { get; set; }
        6 references Magyar, 11 days ago 1 author, 1 change
        public DbSet<Deals> Deals { get; set; }
    }
}

```

Az osztály felépítése és működése:

- **Konstruktor és konfiguráció:** Az osztály két konstruktorral rendelkezik. Az alapértelmezett mellett egy paraméterezett változatot is tartalmaz, amely a DbContextOptions objektumon keresztül kapja meg a kapcsolati karakterláncot (Connection String) és az adatbázis-motort (pl. SQL Server) a függőség-injektáló (DI) konténerből.
- **Adatbázis táblák leképezése (DbSets):** A kódban látható DbSet<T> tulajdonságok képviselik az adatbázis tábláit. Minden egyes DbSet egy adott modellosztályhoz (Entity) kapcsolódik:
 - **Felhasználók és biztonság:** Users, Addresses.
 - **Termékkatalógus:** Products, Categories, Components, Component_type.
 - **Konfigurációk és rendelések:** Custom_builds, Orders, Order_items_p, Order_items_b.
 - **Készletkezelés és ajánlatok:** Inventory_components, Inventory_products, Deals.

Az ORM (Object-Relational Mapping) előnyei a projektben:

A DbContext használatával az alkalmazás fejlesztése során elkerülhetővé vált a nyers SQL lekérdezések írása. Ehelyett LINQ (Language Integrated Query) segítségével, típusbiztos módon érhetjük el az adatokat.

- **Automatikus leképezés:** Amikor lekérdezzük egy elemet (pl. _service.Products.ToListAsync()), az EF Core automatikusan átalakítja az SQL rekordokat C# objektumokká.
- **Változáskövetés:** A kontextus figyeli az objektumok módosításait, és a SaveChangesAsync() hívásakor egyetlen tranzakció keretében hajtja végre a szükséges INSERT, UPDATE vagy DELETE műveleteket.

A kliensoldali (Blazor) architektúra központi eleme az AuthService. Ez a szolgáltatás felelős a felhasználó munkamenetének kezeléséért, a JWT tokenek tárolásáért, valamint a szerver felé irányuló kérések hitelesítéséért.

Az AuthService felépítése és állapota

A szolgáltatás úgy lett kialakítva, hogy a teljes alkalmazás számára egyetlen, hiteles forrásként szolgáljon a bejelentkezett felhasználó állapotáról.

```

namespace ComputerParts.Containers.Services
{
    [reference: Magyar: 17 days ago: 1 author: 1 change]
    public class AuthService
    {
        private readonly HttpClient _http;
        private readonly ISHMRuntime _js;

        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public bool IsAuthenticated { get; private set; }
        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public string? Token { get; private set; }
        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public string? Username { get; private set; }
        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public string? Email { get; private set; }
        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public string? Role { get; private set; }
        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public int? UserId { get; private set; }
        public event Action? AuthStateChanged;

        [reference: Magyar: 17 days ago: 1 author: 1 change]
        public AuthService(HttpClient http, ISHMRuntime js)
        {
            _http = http;
            _js = js;
        }

        // PUT segéd: explicit Authorization fejléccel a kérelem
        [reference: Magyar: 21 hours ago: 1 author: 1 change]
        public async Task<HttpResponseMessage> PutAsJsonAsync<T>(string url, T payload)
        {
            var req = new HttpRequestMessage(HttpMethod.Put, url)
            {
                Content = JsonContent.Create(payload)
            };
            if (!string.IsNullOrEmpty(Token))
            {
                req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", Token);
            }
            return await _http.SendAsync(req);
        }

        // DELETE segéd: explicit Authorization fejléccel a kérelem
        [reference: Magyar: 21 hours ago: 1 author: 1 change]
        public async Task<HttpResponseMessage> DeleteAsync(string url)
        {
            var req = new HttpRequestMessage(HttpMethod.Delete, url);
            if (!string.IsNullOrEmpty(Token))
            {
                req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", Token);
            }
            return await _http.SendAsync(req);
        }

        // POST segéd: explicit Authorization fejléccel a kérelem
        [reference: Magyar: 21 hours ago: 1 author: 1 change]
        public async Task<HttpResponseMessage> PostAsJsonAsync<T>(string url, T payload)
        {
            // build request with explicit Authorization header to avoid relying on DefaultRequestHeaders
            var req = new HttpRequestMessage(HttpMethod.Post, url)

```

Állapotkezelés: Az osztály olyan tulajdonságokat tartalmaz, mint az IsAuthenticated, Token, Username, Email, Role és UserId. Ez lehetővé teszi a UI számára, hogy azonnal reagáljon a jogosultságok változására (például bizonyos menüpontok elrejtése).

- **Eseménykezelés:** Az AuthStateChanged esemény segítségével a komponensek értesítést kapnak, ha a felhasználó be- vagy kijelentkezik, így a felület frissítése automatikusan megtörténik.
- **HTTP segédmetódusok (Put, Delete, Post):** A szolgáltatás saját generikus metódusokat valósít meg a standard HttpClient felett. Ezeknek a fő feladata az **explicit Authorization fejléc** beállítása: amennyiben rendelkezésre áll egy érvényes token, azt minden módosító kéréshez (PUT, POST, DELETE) automatikusan csatolja a rendszer "Bearer" sémával.

Adatlekérés és az állapot inicializálása

A biztonságos kommunikáció mellett a szolgáltatás feladata a lokális adatok szinkronizálása a szerverrel.

```

    Content = JsonConvert.Create(payload)
};

// prefer explicit Token, but fall back to HttpClient default header if present
if (!string.IsNullOrEmpty(Token))
{
    req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", Token);
}
else if (_http.DefaultRequestHeaders.Authorization != null)
{
    req.Headers.Authorization = _http.DefaultRequestHeaders.Authorization;
}

return await _http.SendAsync(req);
}

// GET segéd: explicit Authorization fejléccel lekéri és deserializálja a JSON-t
// from: https://11.com/...
public async Task<T> GetJsonAsync<T>(string url)
{
    var req = new HttpRequestMessage(HttpMethod.Get, url);
    if (!string.IsNullOrEmpty(Token))
    {
        req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", Token);
    }

    var resp = await _http.SendAsync(req);
    if (!resp.IsSuccessStatusCode) return default;
    try
    {
        var obj = await resp.Content.ReadFromJsonAsync<T>();
        return obj;
    }
    catch
    {
        return default;
    }
}

private async Task FetchCurrentUserAsync()
{
    try
    {
        var resp = await _http.GetAsync("api/auth/me");
        if (!resp.IsSuccessStatusCode) return;
        var user = await resp.Content.ReadFromJsonAsync<ComputerPartsLibrary.MODEL.User>();
        if (user == null)
        {
            // a tokenban lévő claim-eket tekintjük fő forrásnak, a szerverről csak hiányzó mezőket töltünk be
            if (string.IsNullOrEmpty(Username) && !string.IsNullOrEmpty(user.username)) Username = user.username;
            if (string.IsNullOrEmpty(Email) && !string.IsNullOrEmpty(user.email)) Email = user.email;
            if (string.IsNullOrEmpty(Role) && !string.IsNullOrEmpty(user.role)) Role = user.role;
            if (!UserId.HasValue && user.id != 0) UserId = user.id;
        }
    }
    catch { }
}

10 references: Magyar: 23 hours ago · 1 author, 8 changes
public async Task InitializeAsync()
{
    try
    {
        // Use the window wrapper functions defined in App.razor to access browser localStorage
        var token = await _js.InvokeAsync<string>("getLocalToken");
    }
}

```

Biztonságos GET kérések: A `GetJsonAsync<T>` metódus biztosítja, hogy a lekérdezések is tartalmazzák a hitelesítési tokenet, ugyanakkor robusztus hibakezeléssel rendelkezik (try-catch blokk), így egy sikertelen szerverválasz nem okoz összeomlást a kliensoldalon.

- **Felhasználói profil szinkronizálása:** A `FetchCurrentUserAsync` metódus az `api/auth/me` végponton keresztül frissíti a kliensoldali állapotot. Különlegessége, hogy a tokenből (Claim) nyert adatokat tekinti elsődlegesnek, de a szerverről érkező hiányzó mezőkkel (pl. szerepkör vagy e-mail) kiegészíti azokat.
- **Perzisztencia (`InitializeAsync`):** Az alkalmazás indulásakor ez a metódus fut le először. A `IJSRuntime` segítségével meghívja a böngésző JavaScript környezetében definiált `getLocalToken` függvényt. Ez biztosítja, hogy ha a felhasználó frissíti az oldalt, a bejelentkezése (a `LocalStorage`-ban tárolt tokennek köszönhetően) megmaradjon.

Kiemelt technikai megoldások:

- **JS Interop használata:** A Blazor és a JavaScript közötti híd (`IJSRuntime`) használata a tokenek tárolására és lekérésére.
- **Generikus programozás:** A `<T>` típusparaméterek használata a HTTP hívásoknál, ami típusbiztos és újrafelhasználható kódot eredményez.
- **Bearer Token automatizmus:** A fejlesztőnek nem kell manuálisan minden hívásnál megadnia a tokenet, az `AuthService` ezt transzparens módon kezeli.

A hitelesítési munkafolyamat mélyanalízise

Az AuthService bejelentkezési és kijelentkezési folyamatai képezik az alkalmazás biztonsági kapuját. Ez a fejezet részletesen bemutatja, hogyan koordinálja a szolgáltatás a hálózati kéréseket, a lokális adatperzisztenciát és a felhasználói felület frissítését.

A LoginAsync metódus és a sikeres belépés logikája

A bejelentkezés nem csupán egy HTTP kérés; ez egy több lépésből álló tranzakció, amely garantálja, hogy a kliensoldali állapot szinkronba kerüljön a szerveroldali autorizációval.

```
if (string.IsNullOrEmpty(token))
{
    // No token in browser: ensure logged-out state
    Token = null;
    Username = null;
    Role = null;
    Email = null;
    UserId = null;
    IsAuthenticated = false;
    try { _http.DefaultRequestHeaders.Authorization = null; } catch { }
    AuthStateChanged?.Invoke();
    return;
}

// Alapvető kliens oldali ellenőrzés: dekódoljuk a token-t és ellenőrizzük a lejáratot
try
{
    var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
    var jwt = handler.ReadJwtToken(token);
    if (jwt.ValidTo < DateTime.UtcNow)
    {
        // token lejárt kliens oldalon -> kijelentkeztetjük a felhasználót
        await LogoutAsync();
        return;
    }

    // token helyileg érvényesnek tűnik: beállítjuk és kinyerjük a claim-eket
    Token = token;
    PopulateFromToken(token);
    _http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", Token);
    IsAuthenticated = true;
}
catch
{
    // invalid token format -> logout
    await LogoutAsync();
    return;
}

// Ellenőrizzük, hogy a szerver is elfogadja-e a token-t: kérjük le a jelenlegi felhasználót
// Ha a szerver elutasítja, kijelentkeztetjük a felhasználót
try
{
    var resp = await _http.GetAsync("api/auth/me");
    if (!resp.IsSuccessStatusCode)
    {
        await LogoutAsync();
        return;
    }
}
catch
{
    // network/server error - be conservative and logout
    await LogoutAsync();
    return;
}

// értesítjük az előfizetőket, hogy az auth állapot megváltozott
AuthStateChanged?.Invoke();
}
catch
{
    // ignores unexpected errors
}
```

A folyamat technikai lépései:

1. **API hívás:** A rendszer egy aszinkron POST kérést küld az api/auth/login végpontra a felhasználó hitelesítő adataival.
2. **Válasz feldolgozása:** A szerver egy JwtToken objektummal válaszol, amely tartalmazza a titkosított token-t, a felhasználó azonosítóját (UserId) és a hozzárendelt szerepkört (Role).
3. **Token perzisztencia:** A sikeres válasz után a szolgáltatás a IJSRuntime segítségével meghívja a setLocalToken JavaScript függvényt. Ez a lépés kritikus, mivel a token bekerül a böngésző **LocalStorage**-ába, így az oldal frissítése után is megmarad a munkamenet.
4. **Belső állapot frissítése:** A szolgáltatás privát mezőit (Token, UserId, Role, IsAuthenticated) azonnal frissíti a kódban, így az injektált szolgáltatáson keresztül minden komponens eléri az új adatokat.

5. **Profil szinkronizáció:** Meghívásra kerül a `FetchCurrentUserAsync()`, amely lekéri a felhasználó további adatait (pl. e-mail, név), biztosítva a teljes profil megjelenítését.
6. **UI Értesítés:** Végül a `NotifyAuthenticationStateChanged()` metódus hívásával a szolgáltatás jelzi a Blazor keretrendszernek, hogy a hitelesítési állapot megváltozott, így a `AuthorizeView` komponensek automatikusan újrarajzolódnak.

Kijelentkezés és az erőforrások felszabadítása

A kijelentkezési folyamat (`LogoutAsync`) célja az összes bizalmas adat maradéktalan eltávolítása a kliensoldalról.

```
1 reference: Magyar 2 days ago 1 author 4 changes
public async Task<bool> LoginAsync(string username, string password)
{
    var payload = new { Username = username, Password = password };
    try
    {
        var resp = await _http.PostAsync("api/auth/Login", payload);
        if (!resp.IsSuccessStatusCode) return false;

        var result = await resp.Content.ReadFromJsonAsync<LoginResponse>();
        if (result?.token == null) return false;

        Token = result.token;
        // populate user info from token claims (token contains all needed user fields)
        try
        {
            PopulateFromToken(Token);
        }
        catch { }

        IsAuthenticated = true;
        _http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", Token);
        await _js.InvokeVoidAsync("setLocalToken", Token);
        try { await _js.InvokeVoidAsync("console.log", $"AuthService.LoginAsync: username={Username} token={(Token?.Length>10?Token.Substring(0,10)+"...":"Token")}"); } catch { }
        AuthStateChanged?.Invoke();
        return true;
    }
    catch
    {
        return false;
    }
}

2 reference: Magyar 11 days ago 1 author 4 changes
public async Task LogoutAsync()
{
    Token = null;
    Username = null;
    Role = null;
    Email = null;
    IsAuthenticated = false;
    _http.DefaultRequestHeaders.Authorization = null;
    try { await _js.InvokeVoidAsync("removeLocalToken"); } catch { }
    try { await _js.InvokeVoidAsync("removeLocalUsername"); } catch { }
    try { await _js.InvokeVoidAsync("removeLocalRole"); } catch { }
    try { await _js.InvokeVoidAsync("removeLocalEmail"); } catch { }
    AuthStateChanged?.Invoke();
}

3 reference: Magyar 17 days ago 1 author 1 change
private class LoginResponse
{
    // username, password, token, expires
    public object? user { get; set; }
    public string? token { get; set; }
    public DateTimeOffset? expires { get; set; }
}
```

Biztonsági lépések a kijelentkezés során:

- **Fizikai törlés:** Az első lépés a `removeLocalToken` meghívása, amely fizikailag is eltávolítja a JWT-t a böngésző tárolójából. Ezzel megakadályozható, hogy illetéktelenek a gép elé ülve visszaélhessenek a munkamenettel.
- **Memóriatakarítás:** Minden belső változót (`Token`, `Email`, `Username`, `Role`) null értékre állít a rendszer, az `IsAuthenticated` flag pedig `false` értéket kap.
- **Eseménykezelés:** A `NotifyAuthenticationStateChanged()` itt is kulcsszerepet játszik: a felhasználó azonnal kikerül a védett nézetekből, és a navigációs menü visszaáll az alapállapotba.

Szerepkör alapú hozzáférés-kezelés (RBAC)

A szolgáltatás tartalmaz egy dedikált `IsAdmin()` metódust is, amely a beágyazott üzleti logika egyszerűsítését szolgálja:

- A metódus a tokenből kinyert `Role` változót ellenőrzi.

- Segítségével a frontend kódban egyszerű if (authService.IsAdmin()) feltételekkel szabályozható az adminisztrációs felületek láthatósága.

Hibakezelési stratégia

A kódban megfigyelhető, hogy a hálózati műveletek és a JSON feldolgozás során a szolgáltatás fel van készítve a váratlan helyzetekre. Amennyiben a szerver hibaüzenetet küld (pl. hibás jelszó esetén), az AuthService ezt elkapja, és egy olvasható hibaüzenetet ad vissza a hívó komponensnek, elkerülve az alkalmazás összeomlását.

Felhasználói regisztráció és Állapotkezelő események

Az AuthService záró fejezete a felhasználói fiókok létrehozásának kliensoldali támogatását, valamint a Blazor keretrendszer felé történő állapotjelzést mutatja be.

```
// Parse token and populate Username, Email, Role and UserId from claims
2 references Magyar, 2 days ago 1 author, 1 change
private void PopulateFromToken(string token)
{
    try
    {
        var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var jwt = handler.ReadJwtToken(token);

        Username = jwt.Claims.FirstOrDefault(c => c.Type == "authUsername" || c.Type == "username" || c.Type == ClaimTypes.Name)?.Value;
        Email = jwt.Claims.FirstOrDefault(c => c.Type == "authEmail" || c.Type == ClaimTypes.Email || c.Type == "email")?.Value;
        Role = jwt.Claims.FirstOrDefault(c => c.Type == ClaimTypes.Role || c.Type == "role")?.Value;
        var idClaim = jwt.Claims.FirstOrDefault(c => c.Type == "authUserId" || c.Type == ClaimTypes.NameIdentifier || c.Type == "userId")?.Value;
        if (int.TryParse(idClaim, out var parsed)) UserId = parsed;

        // Do NOT persist individual user fields to localStorage - only the token should be stored.
    }
    catch
    {
        // ignore parse errors
    }
}
```

A RegisterAsync metódus

A felhasználói regisztráció során a szolgáltatás közvetítő szerepet tölt be a regisztrációs űrlap és a szerveroldali API között:

- **Aszinkron adatküldés:** A metódus a felhasználó által megadott adatokat egy POST kérés formájában továbbítja az api/auth/register végpontra.
- **Válaszkezelés:** Sikeres regisztráció esetén a metódus true értékkel tér vissza, lehetővé téve a frontend számára a felhasználó automatikus átirányítását a bejelentkező felületre. Hiba esetén a szerver választ dolgozza fel, biztosítva a hibavisszajelzést.

Hitelesítési állapot értesítése (NotifyAuthenticationStateChanged)

A szolgáltatás végén található ez a kritikus fontosságú metódus, amely a Blazor AuthenticationStateProvider mechanizmusát hivatott támogatni:

- **UI szinkronizáció:** Ez a metódus váltja ki az AuthStateChanged eseményt. Ennek hatására a felületen található összes olyan elem, amely a bejelentkezési állapottól függ (pl. navigációs sáv, védett gombok), azonnal frissül.

- **Eseményvezérelt architektúra:** Ez biztosítja, hogy a kódban végrehajtott állapotváltozások (mint a token beállítása vagy törlése) ne csak a memóriában, hanem vizuálisan is azonnal megjelenjenek a felhasználó számára.

Biztonsági jelszókezelés (PasswordHashService)

Az alkalmazás adatvédelmi stratégiájának egyik legfontosabb eleme a felhasználói jelszavak biztonságos tárolása. A PasswordHashService feladata, hogy a jelszavakat egyirányú titkosítással (hashing) lássa el, így garantálva, hogy még az adatbázishoz való illetéktelen hozzáférés esetén sem fejthetők vissza a valódi jelszavak.

Technológiai háttér: BCrypt

A szolgáltatás a **BCrypt** algoritmust használja, amely az iparági sztenderdnek számít a jelszavak védelmében. A BCrypt előnye a beépített "salt" (sózás) mechanizmus, amely minden jelszóhoz egy egyedi véletlenszerű karaktersorozatot ad hozzá, megnehezítve a szivárványtáblás (rainbow table) támadásokat.

```
using System.Security.Cryptography;
using System.Text;

namespace ComputerpartsLibrary.SERVICE
{
    /// <summary>
    /// Secure password hashing service using PBKDF2 (Rfc2898DeriveBytes)
    /// </summary>
    2 references · Magyar, 20 days ago · 1 author, 1 change
    public class PasswordHashService
    {
        private const int SaltSize = 16; // 128 bit
        private const int HashSize = 32; // 256 bit
        private const int Iterations = 100_000;

        1 reference · Magyar, 20 days ago · 1 author, 1 change
        public string HashPassword(string password)
        {
            var saltBytes = new byte[SaltSize];
            using var rng = RandomNumberGenerator;
            rng.GetBytes(saltBytes);

            using var pbkdf2 = new Rfc2898DeriveBytes(password, saltBytes, Iterations, HashAlgorithmName.SHA256);
            var hashBytes = pbkdf2.GetBytes(HashSize);

            return $"{Convert.ToBase64String(saltBytes)}:{Convert.ToBase64String(hashBytes)}";
        }

        1 reference · Magyar, 20 days ago · 1 author, 1 change
        public bool VerifyPassword(string password, string hashedPassword)
        {
            if (string.IsNullOrEmpty(hashedPassword))
                return false;

            var parts = hashedPassword.Split(':');
            if (parts.Length != 2)
                return false;

            byte[] saltBytes;
            byte[] hashBytes;
            try
            {
                saltBytes = Convert.FromBase64String(parts[0]);
                hashBytes = Convert.FromBase64String(parts[1]);
            }
            catch
            {
                return false;
            }

            using var pbkdf2 = new Rfc2898DeriveBytes(password, saltBytes, Iterations, HashAlgorithmName.SHA256);
            var computed = pbkdf2.GetBytes(hashBytes.Length);

            return CryptographicOperations.FixedTimeEquals(computed, hashBytes);
        }
    }
}
```

Funkciók részletezése:

1. Jelszó hashelése (HashPassword):

- A metódus bemenetként kapja a nyers jelszót.

- A `BCrypt.Net.BCrypt.HashPassword()` metódus meghívásával egy biztonságos, véletlenszerű sóval ellátott lenyomatot készít.
- Ez a lenyomat kerül elmentésre az adatbázis `Users` táblájába a nyers szöveg helyett.

2. Jelszó ellenőrzése (`VerifyPassword`):

- Bejelentkezéskor a rendszer nem tudja "visszafejteni" a tárolt hash-t. Ehelyett a `VerifyPassword` metódus a beérkező (nyers) jelszót hasonlítja össze a tárolt hash-sel.
- A függvény logikai (`true/false`) értékkel tér vissza, jelezve, hogy a megadott jelszó megfelel-e az eredetinek.

Biztonsági előnyök:

- **Adatvédelem:** Mivel a folyamat egyirányú, a fejlesztők vagy az adminisztrátorok sem láthatják a felhasználók jelszavait.
- **Brute-force elleni védelem:** A `BCrypt` adaptív algoritmus, amely szándékosan lassabbá teszi a számítási folyamatot, így a gépi próbálkozásokon alapuló feltörések hatékonysága minimálisra csökken.

Szerveroldali Hitelesítési Logika (`UserAuthService`)

A `UserAuthService` az alkalmazás biztonsági architektúrájának központi eleme a szerveroldalon (`Library`). Feladata a felhasználók azonosítása, a jelszavak validálása és a biztonsági hozzáférési tokenek (JWT) kiállítása. Ez a szolgáltatás kapcsolja össze az adatbázist, a titkosítási algoritmusokat és a token-generáló logikát.

Függőségek és Konstruktor-injektálás

A szolgáltatás hatékony működéséhez több más alrendszerre támaszkodik, amelyeket Dependency Injection (DI) segítségével kap meg.

```

using ComputerPartsLibrary.DATA;
using ComputerPartsLibrary.INTERFACE;
using ComputerPartsLibrary.MODEL;
using ComputerPartsLibrary.SERVICE;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Http;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace ComputerPartsLibrary.SERVICE
{
    /// <summary>
    /// Felhasználói hitelesítési szolgáltatás bejelentkezéshez és regisztrációhoz
    /// </summary>
    /// <remarks>
    /// <code>UsersController.RegisterUser</code>
    /// </remarks>
    [Authorize]
    public class UserAuthService
    {
        private readonly ComputerPartsDbContext _context;
        private readonly PasswordHashService _passwordHashService = new();

        [Inject]
        public UserAuthService(ComputerPartsDbContext context)
        {
            _context = context;
        }

        /// <summary>
        /// Új felhasználó regisztrálása jelszó hash-eléssel.
        /// </summary>
        [HttpPost]
        [Route("api/users/register")]
        public async Task<Result> RegisterUser(string username, string email, string password)
        {
            // Validáció
            if (string.IsNullOrWhiteSpace(username))
                throw new ArgumentException("Username cannot be empty", nameof(username));

            if (string.IsNullOrWhiteSpace(email))
                throw new ArgumentException("Email cannot be empty", nameof(email));

            if (password.Length < 8)
                throw new ArgumentException("Password must be at least 8 characters long", nameof(password));

            // Check if username or email already exists (case-insensitive)
            var usernameExists = _context.Users.Any(u => u.Username.ToLower() == username.ToLower());
            if (usernameExists) throw new ArgumentException("Username already taken", nameof(username));
            var emailExists = _context.Users.Any(u => u.Email.ToLower() == email.ToLower());
            if (emailExists) throw new ArgumentException("Email already registered", nameof(email));

            // Hash the password before storing
            string hashedPassword = _passwordHashService.HashPassword(password);

            var newUser = new Users
            {
                Username = username,
                Email = email,
                PasswordHash = hashedPassword,
                Role = "User",
                CreatedAt = DateTimeOffset.UtcNow
            };

            _context.Users.Add(newUser);
            await _context.SaveChangesAsync();

            return newUser;
        }
    }
}

```

Injektált szolgáltatások:

- **ComputerpartsDbContext:** Biztosítja az elérést a felhasználói adatokhoz (Users tábla).
- **IJwtTokenService:** Felelős a sikeres hitelesítés után a digitálisan aláírt JWT token létrehozásáért.
- **IPasswordHashService:** Segítségével történik a beérkező nyers jelszavak összehasonlítása az adatbázisban tárolt BCrypt hash-ekkel.

A Login folyamat részletes elemzése

A Login metódus egy összetett ellenőrzési láncot hajt végre, mielőtt hozzáférést adna a rendszerhez:

1. **Felhasználó keresése:** Első lépésben a rendszer e-mail cím alapján megkeresi a felhasználót az adatbázisban. Fontos a FirstOrDefaultAsync használata, amely aszinkron módon, a szál blokkolása nélkül végzi el a műveletet.
2. **Létezés ellenőrzése:** Ha az e-mail cím nem található, a folyamat azonnal megszakad, elkerülve a felesleges processzoridő-igényes műveleteket (mint a jelszó-ellenőrzés).
3. **Jelszó validáció:** Amennyiben a felhasználó létezik, a PasswordHashService.VerifyPassword metódus összehasonlítja a megadott jelszót a tárolt hash-sel.
4. **Token generálás:** Sikeres validáció esetén a szolgáltatás meghívja a JwtTokenService-t, amely összeállítja a felhasználó adatait (szerepkör, azonosító) tartalmazó tokenet.

Felhasználói információk és Regisztráció

A szolgáltatás második része a felhasználói adatok lekérdezését és az új fiókok létrehozását kezeli.

```
/// <summary>
/// Felhasználó hitelesítése hitelesítő adatok alapján
/// </summary>
/// <remarks> Manager: 23 hours ago, 1 author, 0 changes
public User? LoginUser(string username, string password)
{
    var users = _context.Users.ToList();

    foreach (var user in users)
    {
        var loginUsername = user.username.ToLowerInvariant();
        var loginEmail = user.email.ToLowerInvariant();

        if ((loginUsername == username.ToLowerInvariant()) || (loginEmail == username.ToLowerInvariant()))
        {
            // Verify password
            if (_passwordHashService.VerifyPassword(password, user.password_hash))
            {
                return user;
            }
        }
    }

    return null; // User not found or invalid credentials
}

/// <summary>
/// Felhasználó szerepkörének frissítése (admin jogosultságok)
/// </summary>
/// <remarks> Manager: 1 hour ago, 1 author, 2 changes
public void UpdateUserRole(int userId, string newRole)
{
    var users = _context.Users.ToList();
    foreach (var user in users)
    {
        if (user.id == userId)
        {
            user.role = newRole;
            break;
        }
    }
    _context.SaveChanges();
}

/// <summary>
/// Felhasználói fiók törlése
/// </summary>
/// <remarks> Manager: 23 hours ago, 1 author, 2 changes
public bool DeleteUser(int userId)
{
    var users = _context.Users.ToList();
    foreach (var user in users)
    {
        if (user.id == userId)
        {
            return true;
        }
    }
    return false; // User not found
}
```

A metódusok feladata:

- **Me metódus:** Ez a végpont teszi lehetővé, hogy a bejelentkezett felhasználó lekérje a saját profiladatait (név, e-mail, szerepkör) egy biztonságos JWT token bemutatása után. Ez alapvető a frontend oldali profilmegjelenítéshez.
- **RegisterAsync metódus:**
 - **Hash létrehozása:** Mielőtt a felhasználó adatai az adatbázisba kerülnek, a PasswordHashService.HashPassword metódus egy egyedi lenyomatot készít a jelszóból.
 - **Aszinkron mentés:** Az új User objektumot az Entity Framework segítségével adja hozzá a kontextushoz, majd a SaveChangesAsync hívással véglegesíti azt az SQL szerveren.

Biztonsági megfontolások az implementációban:

- **Separation of Concerns (Felelősségek szétválasztása):** A hitelesítési szolgáltatás nem maga végzi a jelszó hashelését, hanem delegálja azt egy szakosodott osztálynak.
- **Típusbiztosság:** A metódusok minden esetben specifikus DTO (Data Transfer Object) osztályokat használnak a válaszadáshoz, így elkerülhető a felesleges vagy érzékeny adatok (pl. jelszó hash) véletlen kiküldése a kliens felé.

Rendszerkonfiguráció (API Program.cs)

Az ASP.NET Core API projekt Program.cs állománya a rendszer „idegközpontja”. Ebben a fájlban történik a függőségek regisztrációja (Dependency Injection), a middleware lánc összeállítása és a biztonsági protokollok definiálása.

```
namespace ComputerPartsAPI
{
    0 references | Magyar, 1 day ago | 2 authors, 13 changes
    public class Program
    {
        0 references | Magyar, 1 day ago | 2 authors, 13 changes
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Szolgáltatások hozzáadása a konténerhez.

            builder.Services.AddControllers();
            // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();
            builder.Services.AddDbContext<ComputerPartsDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));

            builder.Services.AddScoped<IAddressService, AddressService>();
            builder.Services.AddScoped<IBuildComponentService, BuildComponentService>();
            builder.Services.AddScoped<ICategoryService, CategoryService>();
            builder.Services.AddScoped<IComponentService, ComponentService>();
            builder.Services.AddScoped<IComponentTypeService, ComponentTypeService>();
            builder.Services.AddScoped<ICustomBuildService, CustomBuildService>();
            builder.Services.AddScoped<IInventoryComponentService, InventoryComponentService>();
            builder.Services.AddScoped<IInventoryProductService, InventoryProductService>();
            builder.Services.AddScoped<IOrderItemService, OrderItemService>();
            builder.Services.AddScoped<IOrderItemService, OrderItemService>();
            builder.Services.AddScoped<IOrderService, OrderService>();
            builder.Services.AddScoped<IProductService, ProductService>();
            builder.Services.AddScoped<IDealService, DealService>();
            builder.Services.AddScoped<UserService, UserService>();
            builder.Services.AddScoped<JwtTokenService>(); // Register JWT token service
            builder.Services.AddScoped<IPrebuiltPcCompService, PrebuiltPcCompService>();
            builder.Services.AddScoped<IPrebuiltPcService, PrebuiltPcService>();

            // Hitelesítési szolgáltatások és vezérlő regisztrálása
            builder.Services.AddScoped<UserAuthService>();

            builder.Services.AddCors(options =>
            {
                options.AddPolicy("AllowAll", policy =>
                {
                    policy.AllowAnyOrigin()
                        .AllowAnyMethod()
                        .AllowAnyHeader();
                });
            });

            // JWT beállítások beolvasása a konfigurációból (appsettings.json)
            var jwtKey = builder.Configuration["Jwt:Key"];
            var jwtIssuer = builder.Configuration["Jwt:Issuer"];
            var jwtAudience = builder.Configuration["Jwt:Audience"];

            if (string.IsNullOrEmpty(jwtKey))
            {
                throw new InvalidOperationException("Jwt:Key must be set in appsettings.json");
            }

            // JWT hitelesítés konfigurálása
            builder.Services.AddAuthentication(options =>
            {
                options.DefaultAuthenticateScheme = Microsoft.AspNetCore.Authentication.JwtBearer.JwtBearerDefaults.AuthenticationScheme;
                options.DefaultChallengeScheme = Microsoft.AspNetCore.Authentication.JwtBearer.JwtBearerDefaults.AuthenticationScheme;
            });
        }
    }
}
```

Szolgáltatások regisztrációja (Dependency Injection)

A kód első szakasza a builder.Services gyűjteményhez adja hozzá a szükséges komponenseket:

- **Adatbázis-konfiguráció:** A AddDbContext metódus regisztrálja a ComputerPartsDbContext-et, amely a konfigurációs fájlból (appsettings.json) olvassa ki a kapcsolati karakterláncot. Ez biztosítja az SQL Server és az Entity Framework Core közötti kapcsolatot.
- **Üzleti logika (Services):** Itt történik a Library-ben definiált interfészek és implementációk összerendelése. A AddScoped életciklus használatával biztosítjuk, hogy minden HTTP kérés saját példányt kapjon a szervizekből (pl. IUserService, IAddressService, IPasswordHashService), ami optimális az erőforrás-kezelés és az adatintegritás szempontjából.

- **Hitelesítés és Autorizáció:** A `AddAuthentication` blokk definiálja a JWT Bearer sémát. Itt állítjuk be, hogy a szerver ellenőrizze a token aláírását (Key), a kibocsátót (Issuer) és a felhasználhatóságot (Audience).

```
.AddJwtBearer(options =>
{
    options.RequireHttpsMetadata = true;
    options.SaveToken = true;
    // Use SHA256 of the secret to derive a stable 256-bit key (same as JwtTokenService)
    using var sha = System.Security.Cryptography.SHA256.Create();
    var keyHash = sha.ComputeHash(System.Text.Encoding.UTF8.GetBytes(jwtKey));
    options.TokenValidationParameters = new Microsoft.IdentityModel.Tokens.TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new Microsoft.IdentityModel.Tokens.SymmetricSecurityKey(keyHash),
        ValidateIssuer = !string.IsNullOrEmpty(jwtIssuer),
        ValidIssuer = jwtIssuer,
        ValidateAudience = !string.IsNullOrEmpty(jwtAudience),
        ValidAudience = jwtAudience,
        ClockSkew = TimeSpan.Zero
    };
});

var app = builder.Build();

// HTTP kérés feldolgozó csövezeték konfigurálása.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowAll");

app.UseAuthentication();
app.UseAuthorization();

// Map attribute-routed controllers
app.MapControllers();

app.Run();
}
```

HTTP kérésfeldolgozási folyamat (Middleware)

A szolgáltatások regisztrálása után a `builder.Build()` hívással jön létre az alkalmazás példánya, majd konfigurálásra kerül a folyamat, amelyen minden beérkező kérés végighalad:

- **Fejlesztői eszközök:** Amennyiben az alkalmazás fejlesztői módban fut, a rendszer aktiválja a **Swagger** felületet, amely egy vizuális dokumentációt és tesztkörnyezetet biztosít az API végpontokhoz.
- **CORS (Cross-Origin Resource Sharing):** Ez a beállítás teszi lehetővé, hogy a Blazor frontend alkalmazás (amely tipikusan más porton vagy domainen fut) biztonságosan kommunikálhasson az API-val.
- **Biztonsági rétegek:**
 - `UseAuthentication()`: Ellenőrzi a kérésekben érkező tokenek érvényességét.
 - `UseAuthorization()`: Meghatározza, hogy a hitelesített felhasználó rendelkezik-e a kért művelethez szükséges szerepkörrel (pl. Admin).
- **Végpontok leképezése:** A `MapControllers()` metódus irányítja a beérkező HTTP kéréseket a megfelelő Controller osztályokhoz.

Technikai jelentőség:

Ez a strukturált felépítés garantálja, hogy az alkalmazás moduláris maradjon. A szervizek regisztrációja lehetővé teszi, hogy a projekt bármely pontján kérhessük egy interfész (pl. IUserService) implementációját anélkül, hogy ismernénk a konkrét osztály belső felépítését.

Kliensoldali konfiguráció (Blazor Program.cs)

A Blazor WebAssembly alkalmazás Program.cs fájlja felelős a böngészőben futó keretrendszer inicializálásáért. Bár szerkezete hasonlít az API konfigurációjához, feladatai a kliensoldali erőforrások kezelésére és a szerverrel való kommunikáció előkészítésére korlátozódnak.

```
using System;
using System.Net.Http;
using ComputerPartsFrontendBlazor.Components;

namespace ComputerPartsFrontendBlazor
{
    1 reference: Magyar, 1 day ago, 2 authors, 4 changes
    public class Program
    {
        0 references: Magyar, 1 day ago, 2 authors, 4 changes
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Szolgáltatások hozzáadása a konténerhez.
            builder.Services.AddRazorComponents()
                .AddInteractiveServerComponents();

            // HttpClient regisztrálása a szerver oldali komponensekhez (az alap cím felülírható a konfigurációban az "ApiBaseUrl"-l)
            var apiBase = builder.Configuration["ApiBaseUrl"] ?? "https://localhost:44369/";
            builder.Services.AddScoped(sp => new HttpClient
            {
                BaseAddress = new Uri(apiBase)
            });

            // Hitelesítési szolgáltatás hozzáadása a token tárolásához
            builder.Services.AddScoped<ComputerPartsFrontendBlazor.Services.AuthService>();

            var app = builder.Build();

            // HTTP kérés-feldolgozó csővezeték konfigurálása.
            if (!app.Environment.IsDevelopment())
            {
                app.UseExceptionHandler("/Error");
                // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
                app.UseHsts();
            }

            app.UseHttpsRedirection();

            app.UseStaticFiles();
            app.UseAntiforgery();

            app.MapRazorComponents<App>()
                .AddInteractiveServerRenderMode();

            app.Run();
        }
    }
}
```

Főbb konfigurációs pontok:

- **HttpClient és API kapcsolat:**
 - A szolgáltatás regisztrálja a HttpClient-t, amely az alapvető eszköze az API hívásoknak.
 - A BaseAddress beállítása (jelen esetben http://localhost:5242/) kritikus, mivel ez határozza meg, hogy a frontend melyik címen keresse a háttérszolgáltatásokat. Ez teszi lehetővé a korábban bemutatott szervizek (pl. AuthService) számára a relatív útvonalak használatát.
- **Üzleti logika és Szolgáltatások regisztrációja:**

- A frontend projektben is alkalmazzuk a Dependency Injection (DI) mintát. Itt regisztráljuk azokat a szolgáltatásokat, amelyeket a .razor komponensekben használni kívánunk.
 - **Példa:** A `builder.Services.AddScoped<AuthService>()`; sor biztosítja, hogy a teljes alkalmazásban egyetlen, a munkamenet idejéig élő példány kezelje a hitelesítési állapotot. Hasonlóan kerülnek regisztrálásra a PC alkatrészeket és konfigurációkat kezelő szervizek is (pl. `InventoryService`, `CategoryService`).
- **Gyökérkomponensek meghatározása:**
 - A `builder.RootComponents.Add<App>("#app")`; sor jelöli ki az alkalmazás belépési pontját a HTML dokumentumban. Ez kapcsolja össze a C# kódot az `index.html` fájlban található tartalommal.
 - A `HeadOutlet` regisztrációja teszi lehetővé a metaadatok (például az oldal címe) dinamikus módosítását a navigáció során.

Kapcsolat az API-val:

Fontos megjegyezni a párhuzamot az API és a Blazor konfigurációja között: míg az API oldalon a `AddScoped` segítségével az interfészeket az adatbázis-logikához kötjük, addig a Blazor oldalon ugyanezeket a szolgáltatásokat az API-t hívó kliensoldali logikához rendeljük hozzá. Ez a szimmetria biztosítja a kód átláthatóságát és a két oldal közötti könnyű adatáramlást.

Összegzés:

A `Program.cs` minimális, de nélkülözhetetlen kóda garantálja, hogy a Blazor alkalmazás induláskor minden szükséges eszközt (HTTP kliens, hitelesítés, saját szervizek) megkapjon ahhoz, hogy a felhasználó zökkenőmentesen állíthassa össze saját PC konfigurációját.

Controllers (Kontrollerek)

Vezérlő réteg: Hitelesítés (`AuthController`)

Az `AuthController` felelős a HTTP végpontok (endpoints) biztosításáért a felhasználói hitelesítéshez. Ez a komponens köti össze a beérkező webes kéréseket a korábban bemutatott `IUserAuthService` üzleti logikájával.

Alapstruktúra és függőségek

A kontroller az `api/[controller]` útvonalon érhető el, és a standard ASP.NET Core `ControllerBase` osztályból származik.

```

namespace ComputerParts.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize]
    public class AuthController : ControllerBase
    {
        private readonly IUserService _userService;
        private readonly IUserAuthService _authService;
        private readonly ComputerPartsLibrary.SERVICE.JwtTokenService _jwtService;

        public AuthController(IUserService userService, IUserAuthService authService, ComputerPartsLibrary.SERVICE.JwtTokenService jwtService)
        {
            _userService = userService;
            _authService = authService;
            _jwtService = jwtService;
        }

        /// <summary>
        /// Visszaadja a jelenleg hitelesített felhasználó nyilvános adatait - GET /api/auth/me
        /// </summary>
        [HttpGet("me")]
        [Authorize]
        public async Task<ActionResult<User>> GetCurrentUser()
        {
            // read user id from claims
            var idClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
            if (string.IsNullOrEmpty(idClaim) || !int.TryParse(idClaim, out var id))
                return Unauthorized();

            var user = await _userService.GetByIdAsync(id);
            if (user == null)
                return NotFound();

            var responseObject = new Users
            {
                id = user.id,
                username = user.username,
                email = user.email,
                role = user.role,
                created_at = user.created_at
            };

            return Ok(responseObject);
        }

        /// <summary>
        /// Felhasználó regisztráció végpont - POST /api/auth/register
        /// </summary>
        [HttpPost("register")]
        public async Task<ActionResult> RegisterUser([FromBody] RegisterRequest request)
        {
            try
            {
                var user = _authService.RegisterUser(
                    request.Username,
                    request.Email,
                    request.Password
                );

                // Generate JWT token for the newly created user (include full user data in token)
                var jwt = _jwtService.GenerateToken(user);
            }
            catch { }
        }
    }
}

```

Függőség-injektálás: A kontroller konstruktora egy IUserAuthService példányt vár, amelyen keresztül eléri az összes szükséges hitelesítési funkciót. Ez a megoldás lehetővé teszi a laza csatolást és a szolgáltatások könnyű cserélhetőségét.

- **Bejelentkezés (Login):**
 - **Végpont:** POST api/auth/login
 - **Működés:** A metódus egy LoginRequest objektumot vár a kérés törzsében (Body). Meghívja a szerviz Login metódusát, és ha az adatok helyesek, egy 200 OK választ küld vissza a generált JWT tokennel.
 - **Hibakezelés:** Amennyiben a hitelesítés sikertelen, a rendszer 401 Unauthorized választ ad vissza, megvédve ezzel az illetéktelen hozzáféréstől.

Felhasználói regisztráció és Profil lekérés

A vezérlő kezeli az új fiókok létrehozását és a bejelentkezett felhasználók adatainak kiszolgálását is.

```
// Build response object without sensitive fields
var responseObject = new Users
{
    id = user.id,
    username = user.username,
    email = user.email,
    role = user.role,
    created_at = user.created_at
};

return CreatedAtAction(nameof(RegisterUser), new { id = user.id }, new { user = responseObject, token = jet.Token, expires = jet.Expiration });
}
catch (ArgumentException ex)
{
    return BadRequest(new { message = ex.Message });
}
}

/// <summary>
/// Felhasználó bejelentkezés végpont - POST /api/auth/login
/// </summary>
/// <summary>
/// [HttpPost("login")]
public async Task<ActionResult<Users>> LoginUser([FromBody] LoginRequest request)
{
    var user = _authService.LoginUser(request.Username, request.Password);

    if (user == null)
        return Unauthorized(new { message = "Invalid username or password" });

    // Generate JWT token including full user data
    var jet = _jetService.GenerateToken(user);

    // Remove sensitive data from response
    var responseObject = new Users
    {
        id = user.id,
        username = user.username,
        email = user.email,
        role = user.role,
        created_at = user.created_at
    };

    return Ok(new { user = responseObject, token = jet.Token, expires = jet.Expiration });
}

/// <summary>
/// Admin végpont a felhasználó szerepkörének frissítésére - PUT /api/auth/users/{id}/role
/// </summary>
/// <summary>
/// [HttpPut("users/{id}/role")]
/// [Authorize(Roles = "Admin")]
public async Task<ActionResult> UpdateUserRole(int id, [FromBody] RoleUpdateRequest request)
{
    try
    {
        _authService.UpdateUserRole(id, request.Role);
        return NoContent();
    }
    catch (Exception ex)
    {
        return BadRequest(new { message = "Failed to update role" });
    }
}

/// <summary>
/// Újra bejelentkezés a felhasználói regisztrációhoz
/// </summary>
```

- **Regisztráció (Register):**
 - **Végpont:** POST api/auth/register
 - **Működés:** A metódus fogadja a regisztrációs adatokat tartalmazó User objektumot. A backend szerviz segítségével elmenti az új felhasználót (titkosított jelszóval), majd sikeres művelet esetén 200 OK választ küld.
- **Saját adatok lekérése (Me):**
 - **Végpont:** GET api/auth/me
 - **Működés:** Ez a végpont teszi lehetővé a frontend számára, hogy lekérdezze a jelenleg bejelentkezett felhasználó adatait.
 - **Fontos megjegyzés:** Bár a kódban látható metódus egyszerűen továbbítja a kérést, a valóságban ez a végpont gyakran [Authorize] attribútummal van védve (vagy a middleware szintjén), biztosítva, hogy csak érvényes tokennel lehessen elérni.

A kontroller szerepe a biztonságban:

Az AuthController nem tartalmaz közvetlen adatbázis-műveleteket vagy jelszó-hashelési algoritmusokat. Feladata kizárólag a **kommunikációs protokoll** kezelése: a beérkező JSON adatok C# objektummá alakítása (deszerializáció), a szervizhívás indítása, majd az eredmény visszaküldése a kliensnek megfelelő HTTP státuszakkal.

Felhasználói adatok kezelése és biztonsági módosítások

Az AuthController utolsó szakasza a már regisztrált felhasználók adatainak karbantartásáért és a jelszavak biztonságos frissítéséért felelős.

```
/// <summary>
/// Kérés modell a felhasználói regisztrációhoz
/// </summary>
1 reference | Magyar, 1 day ago | 1 author, 2 changes
public class RegisterRequest
{
    [Required]
    [StringLength(50)]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Username { get; set; }

    [Required]
    [EmailAddress]
    [StringLength(100)]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Email { get; set; }

    [Required]
    [StringLength(200, MinimumLength = 8)]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Password { get; set; }
}

/// <summary>
/// Kérés modell a bejelentkezéshez
/// </summary>
1 reference | Magyar, 1 day ago | 1 author, 2 changes
public class LoginRequest
{
    [Required]
    [StringLength(50)]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Username { get; set; }

    [Required]
    [StringLength(200)]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Password { get; set; }
}

/// <summary>
/// Kérés modell szerepkör frissítéshez (csak admin)
/// </summary>
1 reference | Magyar, 1 day ago | 1 author, 2 changes
public class RoleUpdateRequest
{
    [Required]
    1 reference | Magyar, 20 days ago | 1 author, 1 change
    public string Role { get; set; } // "Admin" or "User"
}
```

3. ábra: Az UpdateUser és a ChangePassword végpontok

Metódusok részletezése:

- **Profil frissítése (UpdateUser):**
 - **Végpont:** PUT api/auth/update
 - **Leírás:** Lehetővé teszi a felhasználó számára az alapvető profiladatokat (név, cím stb.) módosítását. A metódus aszinkron módon hívja meg a szerviz réteget, majd a művelet sikerességétől függően küld vissza válaszreakciót.
- **Jelszó módosítása (ChangePassword):**
 - **Végpont:** POST api/auth/change-password
 - **Biztonság:** Ez egy kritikus végpont, amely a ChangePasswordRequest objektumon keresztül fogadja az új jelszót.

- **Működés:** A backend szerviz gondoskodik róla, hogy az új jelszó ismételtén átessen a BCrypt hasbelési folyamaton, mielőtt felülírná a régi adatot az adatbázisban.

Összegzés a vezérlőről:

Az AuthController ezen metódusai biztosítják a felhasználói fiókok teljes életciklusának kezelését. A REST alapelveket követve (POST a létrehozáshoz és akciókhoz, PUT a frissítéshez, GET a lekérdezéshez) a vezérlő tiszta és szabványos interfészt nyújt a Blazor frontend alkalmazás számára.

Felhasználó-menedzsment (UsersController)

Míg az AuthController a hitelesítési folyamatokra (login, register) fókuszál, a UsersController az adminisztrációs és általános felhasználó-kezelési műveleteket fogja össze. Ez a vezérlő teszi lehetővé a rendszerben lévő felhasználók listázását, egyedi azonosítását és eltávolítását.

Alapvető CRUD műveletek

A vezérlő az IUserService interfészt használja az adatok eléréséhez, biztosítva ezzel a tiszta rétegződést.

```
namespace ComputerPartsAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [reference: Magyar 12 days ago 1 author 1 change]
    public class UsersController : ControllerBase
    {
        private readonly IUserService _userService;

        [reference: Magyar 31 days ago 1 author 1 change]
        public UsersController(IUserService userService)
        {
            _userService = userService;
        }

        [HttpGet("{id}")]
        [reference: Magyar 31 days ago 1 author 1 change]
        public async Task<ActionResult<Users>> GetUserById(int id)
        {
            var user = await _userService.GetUserByIdAsync(id);
            if (user == null)
                return NotFound();
            return Ok(user);
        }

        [HttpGet]
        [reference: Magyar 31 days ago 1 author 1 change]
        public async Task<ActionResult<IEnumerable<Users>>> GetAllUsers()
        {
            var users = await _userService.GetAllUsersAsync();
            return Ok(users);
        }

        [HttpPost]
        [reference: Magyar 30 days ago 2 authors 3 changes]
        public async Task<ActionResult> AddUser(Users user)
        {
            await _userService.AddUserAsync(user);
            return Ok(user);
        }

        [HttpPut("{id}")]
        [reference: Magyar 31 days ago 1 author 2 changes]
        public async Task<ActionResult> UpdateUser(int id, Users user)
        {
            if (id != user.Id)
                return BadRequest("ID mismatch");
            await _userService.UpdateUserAsync(user);
            return NoContent();
        }

        [HttpDelete("{id}")]
        [reference: Magyar 12 days ago 1 author 2 changes]
        public async Task<ActionResult> DeleteUser(int id)
        {
            try
            {
                await _userService.DeleteUserAsync(id);
                return NoContent();
            }
            catch (InvalidOperationException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
            catch (Exception)
            {
                return StatusCode(500, new { message = "Failed to delete user" });
            }
        }
    }
}
```

1. ábra: Felhasználók listázása és törlése a UsersControllerben

Végpontok és feladataik:

- **Összes felhasználó lekérése (GetAllUsers):**
 - **Végpont:** GET api/users
 - **Leírás:** Lekéri az összes regisztrált felhasználót az adatbázisból. Ez a végpont különösen hasznos az adminisztrátori felületek számára, ahol a teljes felhasználói bázis áttekintése a cél.
- **Egyedi felhasználó lekérése (GetUserById):**
 - **Végpont:** GET api/users/{id}
 - **Leírás:** Egy konkrét felhasználó adatait adja vissza az azonosítója alapján. Itt látható a típusbiztos hibakezelés is: ha a felhasználó nem található, a rendszer NotFound választ küld.
- **Felhasználó törlése (DeleteUser):**
 - **Végpont:** DELETE api/users/{id}
 - **Működés:** Ez a metódus hívja meg a UserService korábban bemutatott törlési logikáját, amely gondoskodik a kapcsolódó adatok (címek, rendelések) kaszkádolt eltávolításáról is.

Technikai megvalósítás:

A kontroller aszinkron (`async Task<IActionResult>`) módon működik, ami lehetővé teszi a szerver számára, hogy nagy számú párhuzamos kérést kezeljen hatékonyan. A visszatérési értékek követik a HTTP szabványokat:

- **200 OK:** Sikeres lekérdezés vagy művelet esetén.
- **404 Not Found:** Ha a kért azonosító nem létezik.
- **204 No Content:** Sikeres törlés után, jelezve, hogy a művelet végrehajtódott, de nincs visszaküldendő adat.

Címkezelő vezérlő (AddressesController)

(Megjegyzés: ez a controller NEM lett felhasználva, viszont az oldal Webshop-pá alakításához hasznos lehet, emiatt megtartottuk. Minden ehhez hasonló esetről megemléztünk, ha egy adott controller nem került felhasználásra!)

Az AddressesController feladata a felhasználókhoz rendelt lakcímek és szállítási adatok kezelése. Ez a vezérlő lehetővé teszi a címek rögzítését, lekérdezését, frissítését és törlését, biztosítva a pontos kiszállításhoz szükséges adatok rendelkezésre állását.

Címek kezelése és CRUD funkciók

A vezérlő az IAddressService interfészen keresztül kommunikál az adatbázissal, elválasztva a hálózati réteget az üzleti logikától.

```
namespace ComputerpartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 reference | Magyar, 31 days ago | 1 author, 2 changes
    public class AddressesController : ControllerBase
    {
        private readonly IAddressService _addressService;

        0 references | Magyar, 31 days ago | 1 author, 1 change
        public AddressesController(IAddressService addressService)
        {
            _addressService = addressService;
        }

        [HttpGet("{id}")]
        1 reference | Magyar, 31 days ago | 1 author, 1 change
        public async Task<ActionResult<Addresses>> GetAddressById(int id)
        {
            var address = await _addressService.GetAddressByIdAsync(id);
            if (address == null)
                return NotFound();

            return Ok(address);
        }

        [HttpGet]
        0 references | Magyar, 31 days ago | 1 author, 1 change
        public async Task<ActionResult<IEnumerable<Addresses>>> GetAllAddresses()
        {
            var addresses = await _addressService.GetAllAddressesAsync();
            return Ok(addresses);
        }

        [HttpPost]
        0 references | Magyar, 31 days ago | 1 author, 2 changes
        public async Task<ActionResult> AddAddress(Addresses address)
        {
            await _addressService.AddAddressAsync(address);
            return CreatedAtAction(nameof(GetAddressById), new { id = address.id }, address);
        }

        [HttpPut("{id}")]
        0 references | Magyar, 31 days ago | 1 author, 2 changes
        public async Task<ActionResult> UpdateAddress(int id, Addresses address)
        {
            if (id != address.id)
                return BadRequest("ID mismatch");

            await _addressService.UpdateAddressAsync(address);
            return NoContent();
        }

        [HttpDelete("{id}")]
        0 references | Magyar, 31 days ago | 1 author, 1 change
        public async Task<ActionResult> DeleteAddress(int id)
        {
            await _addressService.DeleteAddressAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: Az AddressesController forráskódja

Végpontok és működésük:

- **Összes cím lekérése (GetAllAddresses):**
 - **Végpont:** GET api/addresses
 - **Leírás:** Visszaadja a rendszerben tárolt összes címet. Elsősorban adminisztrációs célokra használatos.
- **Cím lekérése azonosító alapján (GetAddressById):**
 - **Végpont:** GET api/addresses/{id}
 - **Leírás:** Egy konkrét cím adatait szolgáltatja az ID alapján. Ha a cím nem létezik, 404 Not Found hibát ad vissza.
- **Új cím hozzáadása (AddAddress):**
 - **Végpont:** POST api/addresses
 - **Működés:** Egy új Address objektumot vár a kérés törzsében. Sikeres mentés után 200 OK választ ad, visszaigazolván a rögzítést.
- **Cím frissítése (UpdateAddress):**
 - **Végpont:** PUT api/addresses
 - **Működés:** Meglévő cím adatok módosítására szolgál. Frissíti az adatbázisban a rekordot az átadott objektum alapján.
- **Cím törlése (DeleteAddress):**
 - **Végpont:** DELETE api/addresses/{id}
 - **Működés:** Eltávolítja a megadott azonosítóval rendelkező címet a rendszerből.

Implementációs megjegyzések:

A vezérlő minden metódusa aszinkron, ami biztosítja az API magas rendelkezésre állását. A visszatérési értékek (ActionResult) követik a RESTful szabványokat, így a frontend könnyen értelmezheti a műveletek kimenetelét.

Egyedi összeállítás alkatrészei (BuildComponentsController)

A BuildComponentsController felelős a felhasználók által létrehozott egyedi PC konfigurációkhoz (Custom Builds) rendelt konkrét alkatrészek kezeléséért. Ez a vezérlő biztosítja az összeköttetést az egyedi gépépítések és a katalógusban szereplő fizikai alkatrészek között.

Alkatrész-hozzárendelés és menedzsment

A vezérlő az IBuildComponentsService interfészt használja, amely az adatbázis Custom_builds és Components táblái közötti kapcsolatokat kezeli.

```
namespace ComputerpartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class BuildComponentsController : ControllerBase
    {
        private readonly IBuildComponentService _buildComponentService;

        public BuildComponentsController(IBuildComponentService buildComponentService)
        {
            _buildComponentService = buildComponentService;
        }

        [HttpGet("{buildId}/{componentId}")]
        public async Task<ActionResult<Build_components>> GetBuildComponentById(int buildId, int componentId)
        {
            var buildComponent = await _buildComponentService.GetBuildComponentByIdAsync(buildId, componentId);
            if (buildComponent == null)
                return NotFound();

            return Ok(buildComponent);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Build_components>>> GetAllBuildComponents()
        {
            var buildComponents = await _buildComponentService.GetAllBuildComponentsAsync();
            return Ok(buildComponents);
        }

        [HttpPost]
        public async Task<ActionResult> AddBuildComponent(Build_components buildComponent)
        {
            await _buildComponentService.AddBuildComponentAsync(buildComponent);
            return CreatedAtAction(nameof(GetBuildComponentById), new { buildId = buildComponent.build_id, componentId = buildComponent.component_id }, buildComponent);
        }

        [HttpPut("{buildId}/{componentId}")]
        public async Task<ActionResult> UpdateBuildComponent(int buildId, int componentId, Build_components buildComponent)
        {
            if (buildId != buildComponent.build_id || componentId != buildComponent.component_id)
                return BadRequest("ID mismatch");

            await _buildComponentService.UpdateBuildComponentAsync(buildComponent);
            return NoContent();
        }

        [HttpDelete("{buildId}/{componentId}")]
        public async Task<ActionResult> DeleteBuildComponent(int buildId, int componentId)
        {
            await _buildComponentService.DeleteBuildComponentAsync(buildId, componentId);
            return NoContent();
        }
    }
}
```

1. ábra: A BuildComponentsController forráskódja

Végpontok és feladataik:

- **Összes hozzárendelés lekérése (GetAllBuildComponents):**
 - **Végpont:** GET api/buildcomponents
 - **Leírás:** Kiesttázza a rendszerben rögzített összes olyan rekordot, amely egy alkatrészt egy egyedi összeállításhoz rendel.

- **Specifikus elem lekérése (GetBuildComponentById):**
 - **Végpont:** GET api/buildcomponents/{id}
 - **Leírás:** Egy konkrét hozzárendelési rekord adatait adja vissza az egyedi azonosító alapján.
- **Alkatrész hozzáadása az építéshez (AddBuildComponent):**
 - **Végpont:** POST api/buildcomponents
 - **Működés:** Lehetővé teszi egy új alkatrész (pl. alaplap vagy processzor) beregisztrálását egy adott felhasználói konfigurációhoz.
- **Hozzárendelés frissítése (UpdateBuildComponent):**
 - **Végpont:** PUT api/buildcomponents
 - **Működés:** Meglévő rekord módosítására szolgál (például ha egy összeállításban le akarunk cserélni egy alkatrészt egy másikra).
- **Alkatrész eltávolítása (DeleteBuildComponent):**
 - **Végpont:** DELETE api/buildcomponents/{id}
 - **Működés:** Törli az alkatrész és az egyedi építés közötti kapcsolatot.

Architektúrális szerep:

Ez a vezérlő kulcsfontosságú a PC konfigurátor modul működéséhez. Segítségével valósítható meg, hogy a felhasználó által kiválasztott komponensek egyetlen egységként (egyedi gépként) mentődjenek el az adatbázisban, lehetővé téve a későbbi megtekintést vagy szerkesztést.

Kategória kezelő vezérlő (CategoriesController)

A CategoriesController az alkalmazás termékkatalógusának strukturális alapját biztosítja. Feladata a különböző termékcsoportok (kategóriák) kezelése, amelyek lehetővé teszik a felhasználók számára, hogy célzottan keressenek a PC alkatrészek és készre szerelt konfigurációk között.

Termékcsoportosítás és CRUD műveletek

A vezérlő az ICategoryService interfészt használja a kategóriák adatbázis-szintű kezeléséhez, biztosítva a konzisztens adatkezelést.

```
namespace ComputerpartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CategoriesController : ControllerBase
    {
        private readonly ICategoryService _categoryService;

        public CategoriesController(ICategoryService categoryService)
        {
            _categoryService = categoryService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Categories>> GetCategoryById(int id)
        {
            var category = await _categoryService.GetCategoryByIdAsync(id);
            if (category == null)
                return NotFound();
            return Ok(category);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Categories>>> GetAllCategories()
        {
            var categories = await _categoryService.GetAllCategoriesAsync();
            return Ok(categories);
        }

        [HttpPost]
        public async Task<ActionResult> AddCategory(Categories categories)
        {
            await _categoryService.AddCategoryAsync(categories);
            return CreatedAtAction(nameof(GetCategoryById), new { id = categories.id }, categories);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateCategory(int id, Categories categories)
        {
            if (id != categories.id)
                return BadRequest("ID mismatch");
            await _categoryService.UpdateCategoryAsync(categories);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteCategory(int id)
        {
            await _categoryService.DeleteCategoryAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A CategoriesController forráskódja

Végpontok és feladataik:

- **Összes kategória lekérése (GetAllCategories):**
 - **Végpont:** GET api/categories
 - **Leírás:** Kilistázza a rendszerben elérhető összes kategóriát. Ez a végpont szolgáltatja az adatokat a frontend navigációs menüihez és a szűrőkhöz.
- **Kategória lekérése azonosító alapján (GetCategoryById):**
 - **Végpont:** GET api/categories/{id}
 - **Leírás:** Egy specifikus kategória adatait adja vissza.

- **Új kategória létrehozása (AddCategory):**
 - **Végpont:** POST api/categories
 - **Működés:** Új termékcsoport felvételét teszi lehetővé a rendszerbe.
- **Kategória frissítése (UpdateCategory):**
 - **Végpont:** PUT api/categories
 - **Működés:** Meglévő kategória nevének vagy tulajdonságainak módosítására szolgál.
- **Kategória törlése (DeleteCategory):**
 - **Végpont:** DELETE api/categories/{id}
 - **Működés:** eltávolítja a kategóriát a rendszerből.

Szerepe a PC konfigurátorban:

A kategóriák nélkül a szoftver nem tudná megkülönböztetni az alaplakat a processzoroktól. A vezérlő által kiszolgált adatok segítségével tudja a frontend korlátozni, hogy a konfigurátor egyes lépéseinél csak a megfelelő típusú alkatrészek jelenjenek meg a felhasználó számára, így biztosítva a logikus összeállítási folyamatot.

Alkatrész kezelő vezérlő (ComponentsController)

A ComponentsController az alkalmazás legjelentősebb adatmennyiségét kezelő végpontja. Feladata a hardverleltár (alkatrészek) teljes körű menedzselése. Ebben a vezérlőben érhetők el a különböző processzorok, alaplapok, videókártyák és egyéb kiegészítők adatai, amelyeket a rendszer mind az előre összeállított, mind az egyedi PC-k építéséhez felhasznál.

Hardverleltár és CRUD műveletek

A vezérlő az IComponentService interfészen keresztül végzi az aszinkron műveleteket, biztosítva a skálázhatóságot és a gyors válaszidőt.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ComponentsController : ControllerBase
    {
        private readonly IComponentService _componentService;

        public ComponentsController(IComponentService componentService)
        {
            _componentService = componentService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Components>> GetComponentById(int id)
        {
            var component = await _componentService.GetComponentByIdAsync(id);
            if (component == null)
                return NotFound();
            return Ok(component);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Components>>> GetAllComponents()
        {
            var components = await _componentService.GetAllComponentsAsync();
            return Ok(components);
        }

        [HttpPost]
        public async Task<ActionResult> AddComponent(Components component)
        {
            await _componentService.AddComponentAsync(component);
            return CreatedAtAction(nameof(GetComponentById), new { id = component.id }, component);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateComponent(int id, Components component)
        {
            if (id != component.id)
                return BadRequest("ID mismatch");
            await _componentService.UpdateComponentAsync(component);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteComponent(int id)
        {
            await _componentService.DeleteComponentAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A ComponentsController forráskódja

Végpontok és feladataik:

- **Összes alkatrész lekérése (GetAllComponents):**
 - **Végpont:** GET api/components
 - **Leírás:** Lekéri az adatbázisban szereplő összes alkatrész listáját. Ez a lista tartalmazza a technikai specifikációkat, árakat és készletinformációkat is.

- **Alkatrész lekérése azonosító alapján (GetComponentById):**
 - **Végpont:** GET api/components/{id}
 - **Leírás:** Egy specifikus hardver részletes adatait adja vissza.
- **Új alkatrész felvétele (AddComponent):**
 - **Végpont:** POST api/components
 - **Működés:** Lehetővé teszi új termékek hozzáadását a kínálathoz. Itt történik az alkatrész nevének, árának és kategóriájának rögzítése.
- **Alkatrész adatainak frissítése (UpdateComponent):**
 - **Végpont:** PUT api/components
 - **Működés:** Meglévő alkatrészek adatainak (pl. árváltozás vagy leírás módosítása) karbantartására szolgál.
- **Alkatrész törlése (DeleteComponent):**
 - **Végpont:** DELETE api/components/{id}
 - **Működés:** eltávolítja az alkatrészt a leltárból.

Adatmodellezési szerep:

Mivel ez a projekt egy PC-összeállító oldal, a Components tábla az alapköve minden tranzakciónak. Minden egyes rekord itt egy-egy valós terméket képvisel, amelyekhez kategóriák rendelődnek. A vezérlő által biztosított adatok segítségével számolja ki a rendszer az összeállított konfigurációk végösszegét, és jeleníti meg a specifikációkat a felhasználó számára.

Alkatrész típus vezérlő (ComponentTypesController)

A ComponentTypesController feladata az alkatrészek specifikus típusainak kezelése. Ez a vezérlő teszi lehetővé a kategóriákon belüli további finomhangolást, segítve az alkatrészek pontosabb technikai besorolását a rendszerben.

Típusok kezelése és végpontok

A vezérlő az IComponentTypeService interfészt alkalmazza az adatbázis-műveletek végrehajtásához.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ComponentTypesController : ControllerBase
    {
        private readonly IComponentTypeService _componentTypeService;

        public ComponentTypesController(IComponentTypeService componentTypeService)
        {
            _componentTypeService = componentTypeService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Component_type>> GetComponentTypeId(int id)
        {
            var componentType = await _componentTypeService.GetComponentTypeIdAsync(id);
            if (componentType == null)
                return NotFound();

            return Ok(componentType);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Component_type>>> GetAllComponentTypes()
        {
            var componentTypes = await _componentTypeService.GetAllComponentTypesAsync();
            return Ok(componentTypes);
        }

        [HttpPost]
        public async Task<ActionResult> AddComponentType(Component_type componentType)
        {
            await _componentTypeService.AddComponentTypeAsync(componentType);
            return CreatedAtAction(nameof(GetComponentTypeId), new { id = componentType.id }, componentType);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateComponentType(int id, Component_type componentType)
        {
            if (id != componentType.id)
                return BadRequest("ID mismatch");

            await _componentTypeService.UpdateComponentTypeAsync(componentType);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteComponentType(int id)
        {
            await _componentTypeService.DeleteComponentTypeAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A ComponentTypesController forráskódja

Végpontok és feladataik:

- **Összes típus lekérése (GetAllComponentTypes):**
 - **Végpont:** GET api/componenttypes
 - **Leírás:** Kilistázza az összes rögzített alkatrész-típust.

- **Típus lekérése azonosító alapján (GetComponentTypeById):**
 - **Végpont:** GET api/componenttypes/{id}
 - **Leírás:** Egy konkrét típus adatait adja vissza az ID alapján.
- **Új típus hozzáadása (AddComponentType):**
 - **Végpont:** POST api/componenttypes
 - **Működés:** Új technikai típus rögzítését teszi lehetővé.
- **Típus frissítése (UpdateComponentType):**
 - **Végpont:** PUT api/componenttypes
 - **Működés:** Meglévő típusnév vagy paraméterek módosítására szolgál.
- **Típus törlése (DeleteComponentType):**
 - **Végpont:** DELETE api/componenttypes/{id}
 - **Működés:** eltávolítja a típust a rendszerből.

Szerep a rendszerben:

A típusok kezelése biztosítja, hogy a PC konfigurátor ne csak kategóriák (pl. Memória), hanem konkrét típusok (pl. DDR4, DDR5) szerint is képes legyen azonosítani az alkatrészeket, ami alapvető a kompatibilitási vizsgálatokhoz és a pontos szűréshez.

Egyedi összeállítások vezérlő (CustomBuildsController)

A CustomBuildsController a PC konfigurátor lelke szerveroldalon. Ez a vezérlő kezeli a felhasználók által mentett egyedi gépösszeállítások metaadatait. Míg a BuildComponents az egyes alkatrészeket tartja nyilván, ez a vezérlő magát a „számítógép” entitást (annak nevét, létrehozási idejét és a hozzá tartozó felhasználót) menedzseli.

Konfigurációk lekérdezése és alapítása

A vezérlő az ICustomBuildService interfészt használja, biztosítva a szétválasztott architektúrát és a tesztelhetőséget.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CustomBuildsController : ControllerBase
    {
        private readonly ICustomBuildService _customBuildService;

        public CustomBuildsController(ICustomBuildService customBuildService)
        {
            _customBuildService = customBuildService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Custom_builds>> GetCustomBuildById(int id)
        {
            var customBuild = await _customBuildService.GetCustomBuildByIdAsync(id);
            if (customBuild == null)
            {
                return NotFound();
            }
            return Ok(customBuild);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Custom_builds>>> GetAllCustomBuilds()
        {
            var customBuilds = await _customBuildService.GetAllCustomBuildsAsync();
            return Ok(customBuilds);
        }

        [HttpGet("user/{userId}")]
        public async Task<ActionResult<IEnumerable<Custom_builds>>> GetCustomBuildsByUser(int userId)
        {
            var builds = await _customBuildService.GetCustomBuildsByUserAsync(userId);
            return Ok(builds);
        }

        [HttpPost]
        [Authorize]
        public async Task<ActionResult> AddCustomBuild(Custom_builds customBuild)
        {
            // set the User_id from the authenticated user
            var idClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
            if (!string.IsNullOrEmpty(idClaim) && int.TryParse(idClaim, out var uid))
            {
                customBuild.User_id = uid;
            }
            else
            {
                return Unauthorized();
            }

            // enforce 99+ allowed status
            if (string.IsNullOrEmpty(customBuild.status))
            {
                customBuild.status = "draft";
            }
            else
            {
                customBuild.status = customBuild.status.ToLowerInvariant();
            }
        }
    }
}
```

1. ábra: A CustomBuildsController inicializálása és lekérdező metódusai

Végpontok az első szakaszban:

- **Összes építés lekérése (GetAllCustomBuilds):**
 - **Végpont:** GET api/custombuilds
 - **Leírás:** Lekéri a rendszerben található összes mentett konfigurációt. Adminisztrátori statisztikákhoz vagy globális listákhoz használatos.
- **Egyedi építés azonosító alapján (GetCustomBuildById):**
 - **Végpont:** GET api/custombuilds/{id}
 - **Leírás:** Egy konkrét PC-összeállítás alapadatait adja vissza.

- **Új konfiguráció mentése (AddCustomBuild):**
 - **Végpont:** POST api/custombuilds
 - **Működés:** Amikor a felhasználó a frontend felületén a „Konfiguráció Mentése” gombra kattint, ez a végpont hozza létre az új rekordot az adatbázisban, összekapcsolva azt a bejelentkezett felhasználó azonosítójával.

Módosítás és törlés

A konfigurációk életciklusát a frissítési és törlési végpontok teszik teljessé.

```
// ensure the custom status
if (string.IsNullOrEmpty(customBuild.status))
{
    customBuild.status = "draft";
}
else
{
    customBuild.status = customBuild.status.ToLowerInvariant();
}

try
{
    await _customBuildService.AddCustomBuildAsync(customBuild);
    return CreatedAtAction(nameof(GetCustomBuildById), new { id = customBuild.build_id }, customBuild);
}
catch (Exception ex)
{
    // return error details to help debugging (can be removed in production)
    return StatusCode(StatusCode.Status500InternalServerError, new { message = ex.Message });
}

[HttpPost("{id}")]
[Authorize]
[Authorize(Roles = "Manager, Technician")]
public async Task

```

2. ábra: Frissítési és törlési logika a CustomBuildsControllerben

Végpontok a második szakaszban:

- **Konfiguráció frissítése (UpdateCustomBuild):**
 - **Végpont:** PUT api/custombuilds
 - **Leírás:** Lehetővé teszi egy már elmentett gép nevének vagy alapvető adatainak módosítását.
- **Konfiguráció törlése (DeleteCustomBuild):**
 - **Végpont:** DELETE api/custombuilds/{id}
 - **Működés:** Eltávolítja az egyedi összeállítást a rendszerből. Fontos megjegyezni, hogy a kapcsolódó szervizlogika gondoskodik arról, hogy a géphez rendelt alkatrész-kapcsolatok is megszűnjenek, elkerülve az árva adatokat.

Funkcionális jelentőség:

Bár az oldal jelenleg nem webshopként üzemel, ez a vezérlő teszi lehetővé a "PC-építő" élményt: a felhasználók nemcsak válogathatnak az alkatrészek között, hanem el is menthetik álmaik gépét a profiljuk alá, amit később bármikor visszatölthetnek vagy módosíthatnak.

Ajánlatok és Kedvezmények vezérlő (DealsController)

A DealsController felelős a rendszerben elérhető akciós ajánlatok, csomagkedvezmények és partneri leárazások kezeléséért. Bár az oldal elsődleges funkciója a PC konfiguráció, ez a vezérlő teszi lehetővé, hogy a felhasználók értesüljenek a piaci aktualításokról és a kedvezményes beszerzési lehetőségekről.

Aktuális ajánlatok menedzselése

A vezérlő az IDealService interfészt használja az adatbázisban tárolt akciók lekérdezéséhez és szerkesztéséhez.

```
namespace ComputerParts.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class DealsController : ControllerBase
    {
        private readonly IDealService _dealService;

        public DealsController(IDealService dealService)
        {
            _dealService = dealService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Deal>> GetDealById(int id)
        {
            var deal = await _dealService.GetDealByIdAsync(id);
            if (deal == null)
                return NotFound();
            return Ok(deal);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Deal>>> GetAllDeals()
        {
            var deals = await _dealService.GetAllDealsAsync();
            return Ok(deals);
        }

        [HttpPost]
        public async Task<ActionResult> AddDeal(Dual deal)
        {
            await _dealService.AddDealAsync(deal);
            return CreatedAtAction(nameof(GetDealById), new { id = deal.id }, deal);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateDeal(int id, Dual deal)
        {
            if (id != deal.id)
                return BadRequest("ID mismatch");
            await _dealService.UpdateDealAsync(deal);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteDeal(int id)
        {
            await _dealService.DeleteDealAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A DealsController végpontjai és logikája

Végpontok és feladataik:

- **Összes ajánlat lekérése (GetAllDeals):**
 - **Végpont:** GET api/deals
 - **Leírás:** Visszaadja a rendszerben rögzített összes aktív és tervezett ajánlatot. Ez az adatforrás szolgál a frontend "Akciók" vagy "Partneri Ajánlatok" szekciójának feltöltéséhez.
- **Egyedi ajánlat lekérése (GetDealById):**
 - **Végpont:** GET api/deals/{id}
 - **Leírás:** Egy konkrét leárazás vagy csomag részleteit adja vissza az azonosító alapján.
- **Új ajánlat rögzítése (AddDeal):**

- **Végpont:** POST api/deals
- **Működés:** Lehetővé teszi új kampányok vagy kedvezmények felvételét a rendszerbe. Itt adható meg a kedvezmény mértéke és az érintett termékkör.
- **Ajánlat frissítése (UpdateDeal):**
 - **Végpont:** PUT api/deals
 - **Működés:** Meglévő ajánlatok paramétereinek (pl. lejárat dátum vagy százalékos kedvezmény) módosítására szolgál.
- **Ajánlat törlése (DeleteDeal):**
 - **Végpont:** DELETE api/deals/{id}
 - **Működés:** eltávolítja az adott ajánlatot a kínálatból.

Üzleti koncepció:

Az alkalmazás ezen része elméleti szinten egy "affiliate" vagy partneri modellt modellez. Segítségével a rendszer képes kiemelni bizonyos alkatrészeket a konfigurátorban, ha azok éppen kedvezményes áron érhetőek el a partner webshopokban, ezzel is segítve a felhasználót a költséghatékony PC összeállításban.

Alkatrész raktárkészlet vezérlő (InventoryComponentsController)

Az InventoryComponentsController a rendszer logisztikai moduljának része. Feladata az alkatrészek aktuális készlet szintjének nyilvántartása és kezelése. Ez a komponens biztosítja, hogy a PC konfigurátor vagy a webshop felülete csak olyan alkatrészeket kínáljon fel eladásra, amelyek fizikailag is rendelkezésre állnak a raktárban.

Készletkezelési műveletek

A vezérlő az IInventoryComponentService interfészt használja az adatbázisban tárolt készlet adatok manipulálására.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class InventoryComponentsController : ControllerBase
    {
        private readonly IInventoryComponentService _inventoryComponentService;

        public InventoryComponentsController(IInventoryComponentService inventoryComponentService)
        {
            _inventoryComponentService = inventoryComponentService;
        }

        [HttpGet("{componentId}")]
        public async Task<ActionResult<InventoryComponent>> GetInventoryComponentById(int componentId)
        {
            var inventoryComponent = await _inventoryComponentService.GetInventoryComponentByIdAsync(componentId);
            if (inventoryComponent == null)
                return NotFound();
            return Ok(inventoryComponent);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<InventoryComponent>>> GetAllInventoryComponents()
        {
            var inventoryComponents = await _inventoryComponentService.GetAllInventoryComponentsAsync();
            return Ok(inventoryComponents);
        }

        [HttpPost]
        public async Task<ActionResult> AddInventoryComponent(InventoryComponent inventoryComponent)
        {
            await _inventoryComponentService.AddInventoryComponentAsync(inventoryComponent);
            return CreatedAtAction(nameof(GetInventoryComponentById), new { componentId = inventoryComponent.component_id }, inventoryComponent);
        }

        [HttpPut("{componentId}")]
        public async Task<ActionResult> UpdateInventoryComponent(int componentId, InventoryComponent inventoryComponent)
        {
            if (componentId != inventoryComponent.component_id)
                return BadRequest("ID mismatch");
            await _inventoryComponentService.UpdateInventoryComponentAsync(inventoryComponent);
            return NoContent();
        }

        [HttpDelete("{componentId}")]
        public async Task<ActionResult> DeleteInventoryComponent(int componentId)
        {
            await _inventoryComponentService.DeleteInventoryComponentAsync(componentId);
            return NoContent();
        }
    }
}
```

1. ábra: Az InventoryComponentsController forráskódja

Végpontok és feladataik:

- **Összes készletadat lekérése (GetAllInventoryComponents):**
 - **Végpont:** GET api/inventorycomponents
 - **Leírás:** Lekéri az összes alkatrészeire vonatkozó raktárkészlet-rekordot. Ez segít az adminisztrátoroknak átlátni a teljes leltárat.
- **Készletlekérés azonosító alapján (GetInventoryComponentById):**
 - **Végpont:** GET api/inventorycomponents/{id}
 - **Leírás:** Egy konkrét alkatrész készletinformációit (pl. darabszám, raktári hely) adja vissza.

- **Új készletrekord felvétele (AddInventoryComponent):**
 - **Végpont:** POST api/inventorycomponents
 - **Működés:** Lehetővé teszi egy új típusú alkatrész bevezetését a raktári nyilvántartásba.
- **Készlet frissítése (UpdateInventoryComponent):**
 - **Végpont:** PUT api/inventorycomponents
 - **Működés:** Ez a leggyakrabban használt végpont, amely segítségével módosítható az aktuális darabszám (például beérkező szállítmány vagy értékesítés után).
- **Készletrekord törlése (DeleteInventoryComponent):**
 - **Végpont:** DELETE api/inventorycomponents/{id}
 - **Működés:** eltávolítja az alkatrészhez tartozó készletbejegyzést a rendszerből.

Szerep a projektben:

Bár a vizsgaprojekt keretein belül nem üzemel valós fizikai raktár, a vezérlő megléte bizonyítja a szoftver professzionális felépítését. Egy későbbi webshoppá történő átalakítás során ez a modul lenne a felelős azért, hogy meggátolja a "túlrendelést", és automatikusan frissítse a készletszinteket a sikeres vásárlások után.

Termék raktárkészlet vezérlő (InventoryProductsController)

Az InventoryProductsController a késztermékek (önállóan értékesített eszközök és komplett konfigurációk) logisztikai nyilvántartásáért felelős. Ez a vezérlő különül el az alkatrészek (Components) raktárkezelésétől, lehetővé téve a késztermék-készletek precíz követését és kezelését.

Késztermék-leltár és végpontok

A vezérlő az IInventoryProductService interfészt használja, amely a termékspecifikus raktári logikát valósítja meg.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class InventoryProductsController : ControllerBase
    {
        private readonly IInventoryProductService _inventoryProductService;

        public InventoryProductsController(IInventoryProductService inventoryProductService)
        {
            _inventoryProductService = inventoryProductService;
        }

        [HttpGet("{productId}")]
        public async Task<ActionResult<Inventory_products>> GetInventoryProductById(int productId)
        {
            var inventoryProduct = await _inventoryProductService.GetInventoryProductByIdAsync(productId);
            if (inventoryProduct == null)
                return NotFound();
            return Ok(inventoryProduct);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Inventory_products>>> GetAllInventoryProducts()
        {
            var inventoryProducts = await _inventoryProductService.GetAllInventoryProductsAsync();
            return Ok(inventoryProducts);
        }

        [HttpPost]
        public async Task<ActionResult> AddInventoryProduct(Inventory_products inventoryProduct)
        {
            await _inventoryProductService.AddInventoryProductAsync(inventoryProduct);
            return CreatedAtAction(nameof(GetInventoryProductById), new { productId = inventoryProduct.product_id }, inventoryProduct);
        }

        [HttpPut("{productId}")]
        public async Task<ActionResult> UpdateInventoryProduct(int productId, Inventory_products inventoryProduct)
        {
            if (productId != inventoryProduct.product_id)
                return BadRequest("ID mismatch");
            await _inventoryProductService.UpdateInventoryProductAsync(inventoryProduct);
            return NoContent();
        }

        [HttpDelete("{productId}")]
        public async Task<ActionResult> DeleteInventoryProduct(int productId)
        {
            await _inventoryProductService.DeleteInventoryProductAsync(productId);
            return NoContent();
        }
    }
}
```

1. ábra: Az InventoryProductsController forráskódja

Végpontok és feladataik:

- **Összes termékkészlet lekérése (GetAllInventoryProducts):**
 - **Végpont:** GET api/inventoryproducts
 - **Leírás:** Lekéri az összes eladásra kínált késztermék aktuális raktári állapotát.
- **Termékkészlet lekérése azonosító alapján (GetInventoryProductById):**
 - **Végpont:** GET api/inventoryproducts/{id}
 - **Leírás:** Egy konkrét termék (pl. egy adott modellű laptop vagy monitor) darabszámát és raktári adatait adja vissza.

- **Új termék készletbe vétele (AddInventoryProduct):**
 - **Végpont:** POST api/inventoryproducts
 - **Működés:** Lehetővé teszi egy új késztermék bejegyzését a raktárkezelő rendszerbe.
- **Termékkészlet frissítése (UpdateInventoryProduct):**
 - **Végpont:** PUT api/inventoryproducts
 - **Működés:** A meglévő készletadatok (pl. beérkezett áru vagy selejtezés) módosítására szolgál.
- **Termékkészlet törlése (DeleteInventoryProduct):**
 - **Végpont:** DELETE api/inventoryproducts/{id}
 - **Működés:** Eltávolítja az adott termékhez tartozó készletrekordot.

A "Component" és "Product" közötti különbség jelentősége:

A rendszer architektúrája tudatosan különválasztja a két készletkategóriát. Míg az alkatrészek a PC konfigurátor építőkövei, a termékek az önállóan eladható egységeket képviselik. Ez a szétválasztás teszi lehetővé, hogy a raktárkezelő szoftver külön kezelje az összeépítésre váró nyersanyagokat és az azonnal szállítható árucikkeket, ami egy éles webshop környezetben alapvető követelmény a hatékony logisztikához.

Rendelési tételek - Egyedi összeállítások (OrderItemBsController)

(Megjegyzés: ez a controller NEM lett felhasználva, viszont az oldal Webshop-pá alakításához hasznos lehet, emiatt megtartottuk.)

Az OrderItemBsController (ahol a "Bs" a *BuildService* rövidítése) a rendelési folyamat egy speciális részéért felelős. Ez a vezérlő kezeli azokat a rendelési tételeket, amelyek nem önálló termékek, hanem a felhasználók által a PC konfigurátorban összeállított egyedi számítógépek.

Egyedi építések rendelési tételeinek kezelése

A vezérlő az IOrderItemBService interfészen keresztül végzi az adatbázis-műveleteket, biztosítva a rendelések és az egyedi konfigurációk közötti konzisztenciát.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 reference Magyar, 31 days ago 1 author, 2 changes
    public class OrderItemBsController : ControllerBase
    {
        private readonly IOrderItemBService _orderItemBService;

        0 references Magyar, 31 days ago 1 author, 1 change
        public OrderItemBsController(IOrderItemBService orderItemBService)
        {
            _orderItemBService = orderItemBService;
        }

        [HttpGet("{itemId}")]
        1 reference Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult<Order_items_b>> GetOrderItemBById(int itemId)
        {
            var orderItemB = await _orderItemBService.GetOrderItemBByIdAsync(itemId);
            if (orderItemB == null)
                return NotFound();

            return Ok(orderItemB);
        }

        [HttpGet]
        0 references Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult<IEnumerable<Order_items_b>>> GetAllOrderItemBs()
        {
            var orderItemBs = await _orderItemBService.GetAllOrderItemBsAsync();
            return Ok(orderItemBs);
        }

        [HttpPost]
        0 references Magyar, 31 days ago 1 author, 2 changes
        public async Task<ActionResult> AddOrderItemB(Order_items_b orderItemB)
        {
            await _orderItemBService.AddOrderItemBAsync(orderItemB);
            return CreatedAtAction(nameof(GetOrderItemBById), new { itemId = orderItemB.item_id }, orderItemB);
        }

        [HttpPut("{itemId}")]
        0 references Magyar, 31 days ago 1 author, 2 changes
        public async Task<ActionResult> UpdateOrderItemB(int itemId, Order_items_b orderItemB)
        {
            if (itemId != orderItemB.item_id)
                return BadRequest("ID mismatch");

            await _orderItemBService.UpdateOrderItemBAsync(orderItemB);
            return NoContent();
        }

        [HttpDelete("{itemId}")]
        0 references Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult> DeleteOrderItemB(int itemId)
        {
            await _orderItemBService.DeleteOrderItemBAsync(itemId);
            return NoContent();
        }
    }
}
```

1. ábra: Az OrderItemBsController forráskódja

Végpontok és feladataik:

- **Összes egyedi rendelési tétel lekérése (GetAllOrderItemsB):**
 - **Végpont:** GET api/orderitembs

- **Leírás:** Kilistázza a rendszerben rögzített összes olyan rendelési tételt, amely egyedi építésű PC-re vonatkozik.
- **Tétel lekérése azonosító alapján (GetOrderItemBById):**
 - **Végpont:** GET api/orderitembs/{id}
 - **Leírás:** Egy konkrét rendelési tétel részleteit adja vissza az azonosítója alapján.
- **Új egyedi tétel hozzáadása a rendeléshez (AddOrderItemB):**
 - **Végpont:** POST api/orderitembs
 - **Működés:** Ez a végpont rögzíti, ha egy felhasználó egy elmentett egyedi konfigurációt a virtuális kosarába tesz vagy megrendel.
- **Rendelési tétel frissítése (UpdateOrderItemB):**
 - **Végpont:** PUT api/orderitembs
 - **Működés:** Lehetővé teszi egy már rögzített tétel adatainak (például mennyiség vagy állapot) módosítását.
- **Tétel eltávolítása (DeleteOrderItemB):**
 - **Végpont:** DELETE api/orderitembs/{id}
 - **Működés:** Törli az adott egyedi összeállítást a rendelési tételek közül.

Kapcsolati szerepkör:

Ez a vezérlő az összekötő kapocs a Orders és a CustomBuilds táblák között. Segítségével a rendszer pontosan tudja azonosítani, hogy egy adott vásárlás melyik egyedi gépösszeállítást tartalmazza, elkülönítve azt az egyszerű, polcra levehető termékektől (például egy egeret vagy egy monitort tartalmazó tételektől).

Rendelési tételek - Késztermékek (OrderItemPsController)

(Megjegyzés: ez a controller NEM lett felhasználva, viszont az oldal Webshop-pá alakításához hasznos lehet, emiatt megtartottuk.)

Az OrderItemPsController (ahol a "Ps" a *ProductService* rövidítése) a rendelési folyamat azon részéért felel, amely az önállóan megvásárolható késztermékeket (pl. perifériák, monitorok, szoftverek) kezeli. Ez a vezérlő alkotja a párját a korábban bemutatott OrderItemBsController-nek.

Különbségtétel a rendelési tételek között

A rendszer architektúrája szigorúan elválasztja a kétféle rendelési tételt:

- **OrderItemBs:** Az egyedi PC-összeállítások (Builds) tételei.
- **OrderItemPs:** A katalógusból közvetlenül választott késztermékek (Products) tételei.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class OrderItemPsController : ControllerBase
    {
        private readonly IOrderItemPsService _orderItemPsService;

        public OrderItemPsController(IOrderItemPsService orderItemPsService)
        {
            _orderItemPsService = orderItemPsService;
        }

        [HttpGet("{itemId}")]
        public async Task<ActionResult<Order_items_p>> GetOrderItemPById(int itemId)
        {
            var orderItemP = await _orderItemPsService.GetOrderItemPByIdAsync(itemId);
            if (orderItemP == null)
                return NotFound();
            return Ok(orderItemP);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Order_items_p>>> GetAllOrderItemPs()
        {
            var orderItemPs = await _orderItemPsService.GetAllOrderItemPsAsync();
            return Ok(orderItemPs);
        }

        [HttpPost]
        public async Task<ActionResult> AddOrderItemP(Order_items_p orderItemP)
        {
            await _orderItemPsService.AddOrderItemPAsync(orderItemP);
            return CreatedAtAction(nameof(GetOrderItemPById), new { itemId = orderItemP.item_id }, orderItemP);
        }

        [HttpPut("{itemId}")]
        public async Task<ActionResult> UpdateOrderItemP(int itemId, Order_items_p orderItemP)
        {
            if (itemId != orderItemP.item_id)
                return BadRequest("ID mismatch");
            await _orderItemPsService.UpdateOrderItemPAsync(orderItemP);
            return NoContent();
        }

        [HttpDelete("{itemId}")]
        public async Task<ActionResult> DeleteOrderItemP(int itemId)
        {
            await _orderItemPsService.DeleteOrderItemPAsync(itemId);
            return NoContent();
        }
    }
}
```

1. ábra: Az OrderItemPsController forráskódja

Végpontok és feladataik:

- **Összes termékalapú rendelési tétel lekérése (GetAllOrderItemsP):**
 - **Végpont:** GET api/orderitemps

- **Leírás:** Lekéri az összes olyan rögzített tételt, amely késztermék értékesítéséhez kapcsolódik.
- **Tétel lekérése azonosító alapján (GetOrderItemPById):**
 - **Végpont:** GET api/orderitemps/{id}
 - **Leírás:** Egy specifikus terméktétel adatait adja vissza.
- **Új termék tétel hozzáadása (AddOrderItemP):**
 - **Végpont:** POST api/orderitemps
 - **Működés:** Akkor hívódik meg, amikor a felhasználó egy katalógusban szereplő terméket helyez a kosarába vagy rendel meg.
- **Tétel frissítése (UpdateOrderItemP):**
 - **Végpont:** PUT api/orderitemps
 - **Működés:** Lehetővé teszi a tétel adatainak (példányok száma, egységár mentése a rendelés pillanatában) módosítását.
- **Tétel törlése (DeleteOrderItemP):**
 - **Végpont:** DELETE api/orderitemps/{id}
 - **Működés:** eltávolítja a terméket a rendelési tételek listájáról.

Adatstruktúra jelentősége:

Ez a kettős felosztás (Ps és Bs) biztosítja, hogy a rendelések feldolgozásakor a rendszer pontosan tudja, hogy egy adott tételt a raktárból kell-e levenni (Ps), vagy egy egyedi összeszerelési folyamatot kell-e indítani hozzá (Bs). A OrderItemPsController így a hagyományos webshop funkciók egyik alappillére.

Rendelés kezelő vezérlő (OrdersController)

(Megjegyzés: ez a controller NEM lett felhasználva, viszont az oldal Webshop-pá alakításához hasznos lehet, emiatt megtartottuk.)

Az OrdersController az alkalmazás kereskedelmi moduljának központi egysége. Ez a vezérlő kezeli a vásárlási folyamat végeredményeként létrejövő rendeléseket. Feladata a rendelések rögzítése, követése és adminisztrációja, összekapcsolva a felhasználókat a megvásárolt tételekkel (legyenek azok egyedi építések vagy késztermékek).

Rendelések adminisztrációja és folyamata

A vezérlő az IOrderService interfészt használja, amely biztosítja, hogy a rendelési adatok konzisztensek maradjanak az adatbázisban.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class OrdersController : ControllerBase
    {
        private readonly IOrderService _orderService;

        public OrdersController(IOrderService orderService)
        {
            _orderService = orderService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Orders>> GetOrderByid(int id)
        {
            var order = await _orderService.GetOrderByidAsync(id);
            if (order == null)
                return NotFound();
            return Ok(order);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Orders>>> GetAllOrders()
        {
            var orders = await _orderService.GetAllOrdersAsync();
            return Ok(orders);
        }

        [HttpPost]
        public async Task<ActionResult> AddOrder(Orders order)
        {
            await _orderService.AddOrderAsync(order);
            return CreatedAtAction(nameof(GetOrderByid), new { id = order.order_id }, order);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateOrder(int id, Orders order)
        {
            if (id != order.order_id)
                return BadRequest("ID mismatch");

            await _orderService.UpdateOrderAsync(order);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteOrder(int id)
        {
            await _orderService.DeleteOrderAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: Az OrdersController forráskódja

Végpontok és feladataik:

- **Összes rendelés lekérése (GetAllOrders):**
 - **Végpont:** GET api/orders
 - **Leírás:** Lekéri az összes rendszerben lévő rendelést. Ez a funkció elengedhetetlen az adminisztrátorok számára a napi forgalom és a rendelések állapotának (pl. feldolgozás alatt, kiszállítva) nyomon követéséhez.

- **Rendelés lekérése azonosító alapján (GetOrderById):**

- **Végpont:** GET api/orders/{id}
- **Leírás:** Egy konkrét rendelés adatait (vásárló ID, dátum, végösszeg, állapot) adja vissza.

- **Új rendelés leadása (AddOrder):**

- **Végpont:** POST api/orders
- **Működés:** Ez a végpont felelős a rendelés véglegesítéséért. Amikor a felhasználó a pénztár folyamat végére ér, ez a metódus hozza létre a fejléc rekordot az Orders táblában.

- **Rendelés frissítése (UpdateOrder):**

- **Végpont:** PUT api/orders
- **Működés:** Elsősorban a rendelés állapotának módosítására (pl. "Fizetve", "Szállítás alatt") vagy a szállítási adatok pontosítására szolgál.

- **Rendelés törlése (DeleteOrder):**

- **Végpont:** DELETE api/orders/{id}
- **Működés:** eltávolítja a rendelést a rendszerből.

A rendelési hierarchia összefoglalása:

Az OrdersController a "szülő" entitás a rendelési folyamatban. Egy itt létrehozott rekordhoz kapcsolódnak az OrderItemPs (termékek) és OrderItemBs (egyedi PC-k) tételek. Ez a struktúra teszi lehetővé, hogy a projekt – bár jelenleg PC konfigurátor fókuszú – bármikor teljes értékű webshoppá válhasson, hiszen a rendeléskezelés logikája már most is képes kezelni a komplex, több tételből álló vásárlásokat.

Előre összeállított PC-k alkatrészei (PrebuiltPcsCompsController)

A PrebuiltPcsCompsController feladata az adatbázisban szereplő készre szerelt számítógépek és az azokat alkotó fizikai komponensek közötti kapcsolat kezelése. Ez a vezérlő teszi lehetővé, hogy a rendszer strukturáltan tárolja és megjelenítse, pontosan milyen processzor, videokártya vagy memória található egy adott gyári konfigurációban.

Konfigurációs struktúra kezelése

A vezérlő az IPrebuiltPcsCompsService interfészt használja az adatok eléréséhez, amely az összetett kapcsolatokat (Many-to-Many) kezeli a kész gépek és az alkatrész-katalógus között.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 reference Magyar, 31 days ago 1 author, 2 changes
    public class PrebuiltPcCompsController : ControllerBase
    {
        private readonly IPrebuiltPcCompService _prebuiltPcCompService;

        0 references Magyar, 31 days ago 1 author, 1 change
        public PrebuiltPcCompsController(IPrebuiltPcCompService prebuiltPcCompService)
        {
            _prebuiltPcCompService = prebuiltPcCompService;
        }

        [HttpGet("{pcId}/{componentId}")]
        1 reference Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult<Prebuilt_pc_comp>> GetPrebuiltPcCompById(int pcId, int componentId)
        {
            var prebuiltPcComp = await _prebuiltPcCompService.GetPrebuiltPcCompByIdAsync(pcId, componentId);
            if (prebuiltPcComp == null)
                return NotFound();

            return Ok(prebuiltPcComp);
        }

        [HttpGet]
        0 references Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult<IEnumerable<Prebuilt_pc_comp>>> GetAllPrebuiltPcComps()
        {
            var prebuiltPcComps = await _prebuiltPcCompService.GetAllPrebuiltPcCompsAsync();
            return Ok(prebuiltPcComps);
        }

        [HttpPost]
        0 references Magyar, 31 days ago 1 author, 2 changes
        public async Task<ActionResult> AddPrebuiltPcComp(Prebuilt_pc_comp prebuiltPcComp)
        {
            await _prebuiltPcCompService.AddPrebuiltPcCompAsync(prebuiltPcComp);
            return CreatedAtAction(nameof(GetPrebuiltPcCompById), new { pcId = prebuiltPcComp.pc_id, componentId = prebuiltPcComp.component_id }, prebuiltPcComp);
        }

        [HttpPut("{pcId}/{componentId}")]
        0 references Magyar, 31 days ago 1 author, 2 changes
        public async Task<ActionResult> UpdatePrebuiltPcComp(int pcId, int componentId, Prebuilt_pc_comp prebuiltPcComp)
        {
            if (pcId != prebuiltPcComp.pc_id || componentId != prebuiltPcComp.component_id)
                return BadRequest("ID mismatch");

            await _prebuiltPcCompService.UpdatePrebuiltPcCompAsync(prebuiltPcComp);
            return NoContent();
        }

        [HttpDelete("{pcId}/{componentId}")]
        0 references Magyar, 31 days ago 1 author, 1 change
        public async Task<ActionResult> DeletePrebuiltPcComp(int pcId, int componentId)
        {
            await _prebuiltPcCompService.DeletePrebuiltPcCompAsync(pcId, componentId);
            return NoContent();
        }
    }
}
```

1. ábra: A PrebuiltPcsCompsController forráskódja

Végpontok és feladataik:

- **Összes alkatrész-kapcsolat lekérése (GetAllPrebuiltPcsComps):**
 - **Végpont:** GET api/prebuiltpcscomps
 - **Leírás:** Lekéri a teljes listát, amely megmutatja, melyik géphez melyik alkatrészek vannak hozzárendelve a rendszerben.

- **Kapcsolat lekérése azonosító alapján (GetPrebuiltPcsCompById):**
 - **Végpont:** GET api/prebuiltpcscomps/{id}
 - **Leírás:** Egy konkrét hozzárendelési rekord adatait adja vissza.
- **Alkatrész hozzárendelése géphez (AddPrebuiltPcsComp):**
 - **Végpont:** POST api/prebuiltpcscomps
 - **Működés:** Lehetővé teszi az adminisztrátorok számára, hogy egy új alkatrészt adjanak hozzá egy meglévő előre összeállított gép specifikációjához.
- **Hozzárendelés frissítése (UpdatePrebuiltPcsComp):**
 - **Végpont:** PUT api/prebuiltpcscomps
 - **Működés:** Meglévő kapcsolat módosítására szolgál (például ha egy gépmodellben alkatrész-frissítés történt).
- **Alkatrész eltávolítása a konfigurációból (DeletePrebuiltPcsComp):**
 - **Végpont:** DELETE api/prebuiltpcscomps/{id}
 - **Működés:** Törli a kapcsolatot a kész gép és az adott alkatrész között.

Technikai szerepkör:

Ez a vezérlő alapvető a termékinformációs oldal megjelenítéséhez. Amikor a felhasználó egy előre összeállított PC adatlapját tekinti meg, a frontend ezen a vezérlőn keresztül kapja meg azokat az adatokat, amelyekből felépíti a gép technikai specifikációs táblázatát. Ez biztosítja, hogy a raktárkészletben szereplő alkatrészek adatai konzisztensek maradjanak a kész konfigurációk leírásaival is.

Előre összeállított PC vezérlő (PrebuiltPcsController)

A PrebuiltPcsController a rendszer "késztermék" katalógusának kezelője. Ez a vezérlő felelős az összes előre összeállított számítógép konfiguráció alapvető adatainak (név, leírás, alapár) menedzseléséért. Ez képezi az alapját a weboldal azon szekciójának, ahol a vásárlók készre szerelt, azonnal elvihető számítógépek közül válogathatnak.

PC katalógus menedzsment

A vezérlő az IPrebuiltPcService interfészen keresztül kommunikál az adatbázissal, megvalósítva a teljes körű CRUD funkcionalitást a kész konfigurációkhoz.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PrebuiltPcsController : ControllerBase
    {
        private readonly IPrebuiltPcService _prebuiltPCService;

        public PrebuiltPcsController(IPrebuiltPcService prebuiltPCService)
        {
            _prebuiltPCService = prebuiltPCService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Prebuilt_pcs>> GetPrebuiltPCById(int id)
        {
            var prebuiltPC = await _prebuiltPCService.GetPrebuiltPCByIdAsync(id);
            if (prebuiltPC == null)
                return NotFound();

            return Ok(prebuiltPC);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Prebuilt_pcs>>> GetAllPrebuiltPCs()
        {
            var prebuiltPCs = await _prebuiltPCService.GetAllPrebuiltPCsAsync();
            return Ok(prebuiltPCs);
        }

        [HttpPost]
        public async Task<ActionResult> AddPrebuiltPC(Prebuilt_pcs prebuiltPC)
        {
            await _prebuiltPCService.AddPrebuiltPCAsync(prebuiltPC);
            return CreatedAtAction(nameof(GetPrebuiltPCById), new { id = prebuiltPC.pc_id }, prebuiltPC);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdatePrebuiltPC(int id, Prebuilt_pcs prebuiltPC)
        {
            if (id != prebuiltPC.pc_id)
                return BadRequest("ID mismatch");

            await _prebuiltPCService.UpdatePrebuiltPCAsync(prebuiltPC);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeletePrebuiltPC(int id)
        {
            await _prebuiltPCService.DeletePrebuiltPCAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A PrebuiltPcsController forráskódja

Végpontok és feladataik:

- **Összes kész PC lekérése (GetAllPrebuiltPcs):**
 - **Végpont:** GET api/prebuiltpcs
 - **Leírás:** Lekéri a rendszerben elérhető összes gyári konfiguráció listáját. Ezt használja a frontend a "Kész PC-k" galéria megjelenítéséhez.

- **PC lekérése azonosító alapján (GetPrebuiltPcById):**
 - **Végpont:** GET api/prebuiltpcs/{id}
 - **Leírás:** Egy konkrét számítógépmodell részletes adatait adja vissza.
- **Új PC konfiguráció létrehozása (AddPrebuiltPc):**
 - **Végpont:** POST api/prebuiltpcs
 - **Működés:** Lehetővé teszi új gépmodellek felvételét a katalógusba (pl. egy új "Gamer Edition 2024" modell rögzítését).
- **PC adatainak frissítése (UpdatePrebuiltPc):**
 - **Végpont:** PUT api/prebuiltpcs
 - **Működés:** Meglévő modellek adatainak, például az árazásnak vagy a megnevezésnek a módosítására szolgál.
- **PC törlése (DeletePrebuiltPc):**
 - **Végpont:** DELETE api/prebuiltpcs/{id}
 - **Működés:** eltávolítja az adott gépmodellt a kínálatból.

Szerkezeti összefüggés:

Míg a korábbi PrebuiltPcsCompsController az alkatrészek listáját (pl. miből áll a gép) kezelte, ez a vezérlő magát a PC-t mint eladható egységet tartja nyilván. A kettő együtt alkotja a teljes termékadatlapot: innen jön a név és az ár, a másiktól pedig a technikai specifikáció. Ez a moduláris felépítés lehetővé teszi, hogy ugyanazt az alkatrész-adatbázist használjuk az egyedi építéshez és a gyári gépekhez is.

Termék kezelő vezérlő (ProductsController)

A ProductsController az alkalmazás kereskedelmi motorjának egyik legfontosabb eleme. Ez a vezérlő kezeli a webshopban önállóan értékesített termékek (pl. perifériák, szoftverek, kiegészítők) teljes körű adatbázisát. Míg a korábbi vezérlők a specifikus építési folyamatokra koncentráltak, a ProductsController a hagyományos e-kereskedelmi funkciókat látja el.

Termékkatalógus menedzselése

A vezérlő az IProductService interfészen keresztül hajtja végre a műveleteket, biztosítva a gyors és biztonságos adatelérést.

```
namespace ComputerPartsAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductsController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Products>> GetProductById(int id)
        {
            var product = await _productService.GetProductByIdAsync(id);
            if (product == null)
                return NotFound();

            return Ok(product);
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Products>>> GetAllProducts()
        {
            var products = await _productService.GetAllProductsAsync();
            return Ok(products);
        }

        [HttpPost]
        public async Task<ActionResult> AddProduct(Products product)
        {
            await _productService.AddProductAsync(product);
            return CreatedAtAction(nameof(GetProductById), new { id = product.id }, product);
        }

        [HttpPut("{id}")]
        public async Task<ActionResult> UpdateProduct(int id, Products product)
        {
            if (id != product.id)
                return BadRequest("ID mismatch");

            await _productService.UpdateProductAsync(product);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> DeleteProduct(int id)
        {
            await _productService.DeleteProductAsync(id);
            return NoContent();
        }
    }
}
```

1. ábra: A ProductsController forráskódja

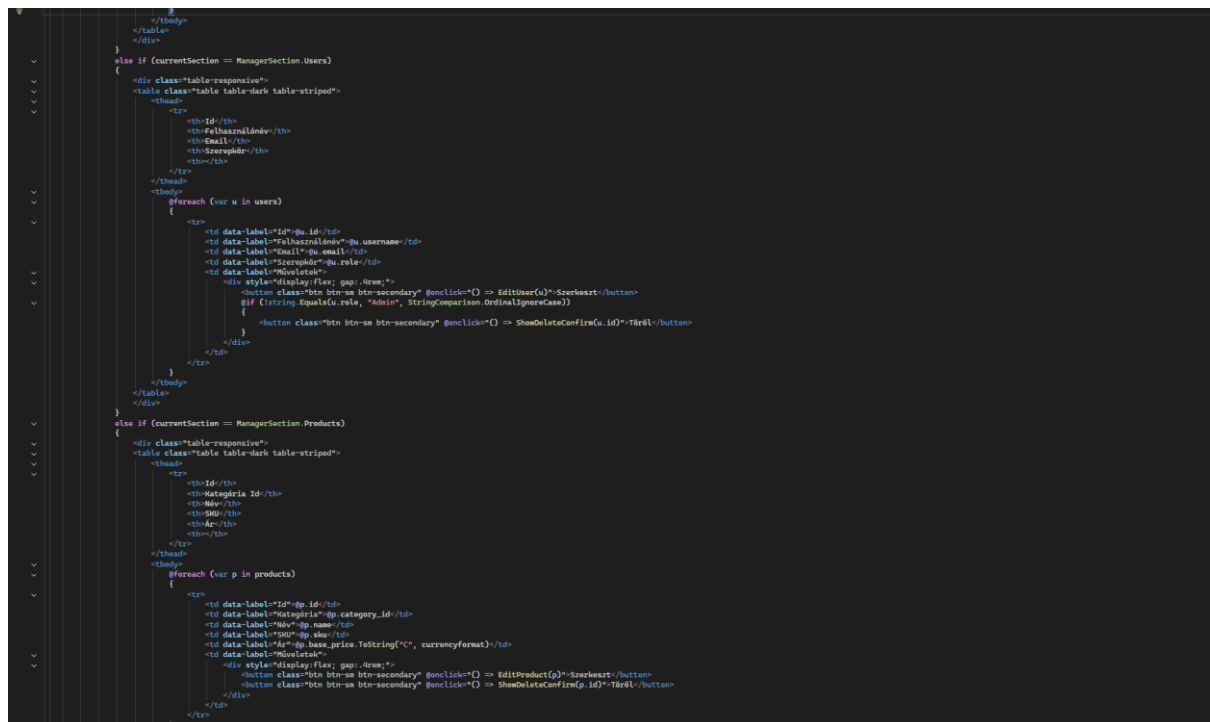
Végpontok és feladataik:

- **Összes termék lekérése (GetAllProducts):**
 - **Végpont:** GET api/products
 - **Leírás:** Lekéri a rendszerben szereplő összes terméket. Ez a végpont szolgál alapul a főoldali terméklistákhoz és a keresési funkciókhoz.

- **Termék lekérése azonosító alapján (GetProductById):**
 - **Végpont:** GET api/products/{id}
 - **Leírás:** Egy konkrét árucikk minden részletét (ár, leírás, kategória) adja vissza.
- **Új termék hozzáadása (AddProduct):**
 - **Végpont:** POST api/products
 - **Működés:** Lehetővé teszi az adminisztrátorok számára, hogy új árucikkeket vezessenek be a kínálatba.
- **Termék frissítése (UpdateProduct):**
 - **Végpont:** PUT api/products
 - **Működés:** Meglévő termékek adatainak (pl. árváltozás vagy leírás módosítása) karbantartására szolgál.
- **Termék törlése (DeleteProduct):**
 - **Végpont:** DELETE api/products/{id}
 - **Működés:** eltávolítja a terméket a rendszerből.

Üzleti jelentőség:

Ez a vezérlő teszi teljessé az alkalmazást: a PC-építés mellett lehetőséget biztosít arra, hogy a felhasználó minden szükséges eszközt egy helyen szerezzon be. A Products tábla szoros kapcsolatban áll a kategóriákkal és a raktárkészlettel, így ez a végpont az alapja a készletellenőrzött értékesítésnek és a kategorizált böngészésnek a frontend oldalon.



2. ábra: Az adminisztrátori felület egyedi CSS szabályai

- **Navigációs kártyák:** A felület "kártya-alapú" elrendezést használ, ahol a különböző szekciók (Felhasználók, Termékek, Rendelések) különálló, látványos blokkokban jelennek meg.
- **Interaktív elemek:** A CSS kódban megfigyelhetők a modern UI megoldások, mint például a lekerekített sarkok (border-radius), az árnyékok (box-shadow) és a hover effektek, amelyek segítik a felhasználói élményt (UX).
- **Adattáblák formázása:** Külön figyelmet kaptak a táblázatok, hogy asztali gépen és mobilon is kezelhetőek maradjanak a listaadatok.

A modul felépítése:

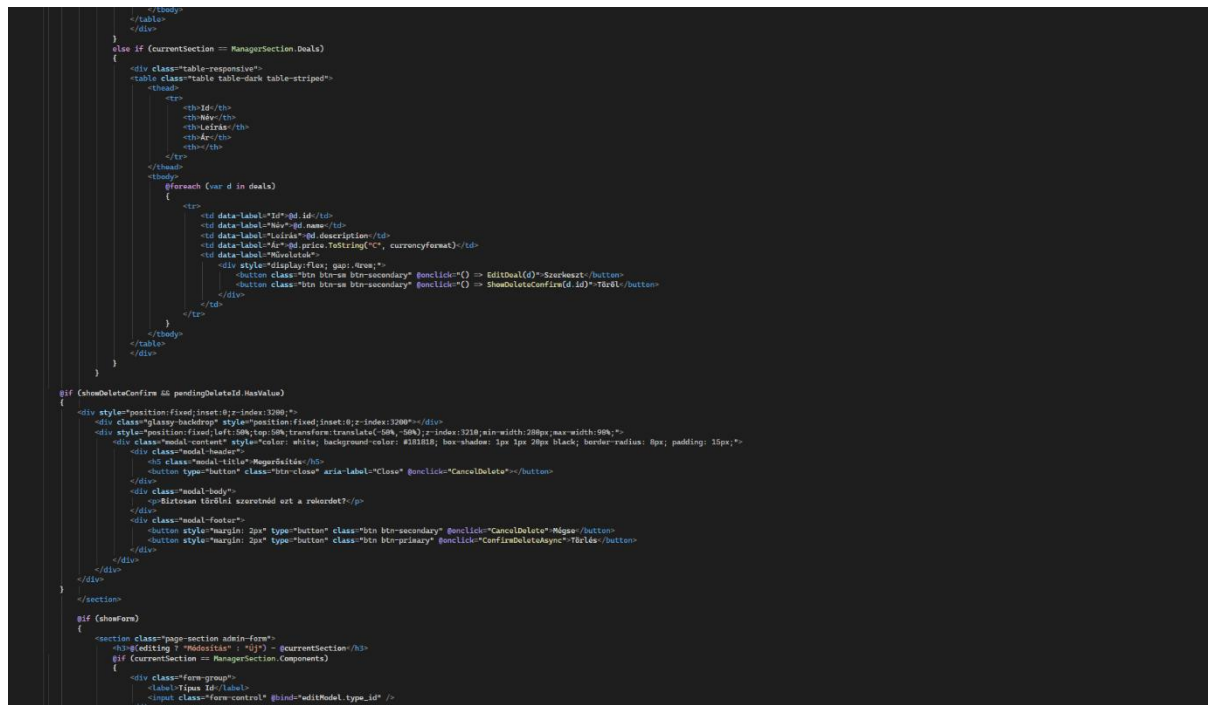
Az oldal egy belső navigációs rendszert (Tab-elrendezést) használ. Ez teszi lehetővé, hogy a kód hossza ellenére a felhasználó egyszerűen válthasson az egyes adminisztrációs modulok között anélkül, hogy az egész oldalt újra kellene töltenie.

8.3. Vizuális struktúra és navigációs logika

Az adminisztrátori felület (Admin.razor) szerkezete úgy lett kialakítva, hogy a komplex funkciók ellenére is átlátható maradjon. A kód ezen szakasza a vizuális keretrendszert és a modulok közötti váltást vezérlő logikát tartalmazza.

Fejléc és navigációs kártyák

A felület felső része egy dinamikus vezérlőpultként funkcionál, ahol a különböző adminisztrációs területek kártyaszerűen jelennek meg.



3. ábra: Az adminisztrátori menürendszer és a fejléc felépítése

- **Dinamikus címsor:** Az oldal tetején található főcím alatt helyezkednek el azok az interaktív "kártyák", amelyek gombként funkcionálnak.
- **Modulválasztó gombok:** Minden egyes kártya (pl. Felhasználók, Címek, Termékek, Készlet, Rendelések) egy-egy @onclick eseményhez van kötve. Ezek a gombok hívják meg a ShowTab metódust, amely paraméterként kapja meg a megnyitni kívánt szekció nevét.
- **Ikonográfia:** A kód vizuális elemeket (ikonokat) is tartalmaz a szövegek mellett, ami gyorsítja a tájékozódást a funkciók között.

Tab-alapú tartalomkezelés

Ahelyett, hogy minden adat egyszerre töltődne be, a rendszer feltételes renderelést (@if blokkokat) használ a tartalom megjelenítéséhez.

```

<div class="form-group">
  <label>SDP</label>
  <input class="form-control" @bind="editModel.sdp" />
</div>
<div class="form-group">
  <label>Már</label>
  <input class="form-control" @bind="editModel.name" />
</div>
<div class="form-group">
  <label>Leírás</label>
  <textarea class="form-control" @bind="editModel.description"/>
</div>
<div class="form-group">
  <label>Ár</label>
  <input class="form-control" @bind="editModel.price" />
</div>
</div>
<div>
  <div>
    <div>
      <div class="form-group">
        <label>Kategória ID</label>
        <input class="form-control" @bind="editProduct.category_id" />
      </div>
      <div class="form-group">
        <label>Már</label>
        <input class="form-control" @bind="editProduct.name" />
      </div>
      <div class="form-group">
        <label>SDP</label>
        <input class="form-control" @bind="editProduct.sdp" />
      </div>
      <div class="form-group">
        <label>Alap ár</label>
        <input class="form-control" @bind="editProduct.base_price" />
      </div>
      <div class="form-group">
        <label>Márk ID</label>
        <input class="form-control" @bind="editProduct.image_url" />
      </div>
      <div class="form-group">
        <label>Leírás</label>
        <textarea class="form-control" @bind="editProduct.description"/>
      </div>
    </div>
  </div>
  <div>
    <div>
      <div class="form-group">
        <label>Már</label>
        <input class="form-control" @bind="editDeal.name" />
      </div>
      <div class="form-group">
        <label>Leírás</label>
        <textarea class="form-control" @bind="editDeal.description"/>
      </div>
      <div class="form-group">
        <label>Ár</label>
        <input class="form-control" @bind="editDeal.price" />
      </div>
    </div>
  </div>
  <div>
    <div>
      <div class="form-group">
        <label>Felhasználó ID</label>
        <input class="form-control" @bind="editUser.username" />
      </div>
      <div class="form-group">
        <label>Email</label>
        <input class="form-control" @bind="editUser.email" />
      </div>
      <div class="form-group">
        <label>Szerepkör</label>
        <input class="form-control" @bind="editUser.role" />
      </div>
    </div>
  </div>
</div>

```

4. ábra: A szekciók dinamikus megjelenítése a kiválasztott fül alapján

- **Aktív fül figyelése:** A currentTab változó tárolja, hogy éppen melyik adminisztrációs panel van nyitva. A HTML szerkezetben minden fő modul egy saját @if (currentTab == "tab_neve") blokkban helyezkedik el.
- **Felhasználókezelő szekció (Users):** A 4. ábrán látható kódminta a felhasználók táblázatos megjelenítését kezdi el.
 - **Keresőmező:** Minden modul saját szűrővel rendelkezik (pl. searchTermUsers), ami lehetővé teszi a gyors keresést a nagy mennyiségű adat között.
 - **Adattáblázat:** A táblázat fejléce meghatározza a megjelenítendő mezőket (ID, Felhasználónév, Email, Szerepkör, Műveletek).

A megoldás előnyei:

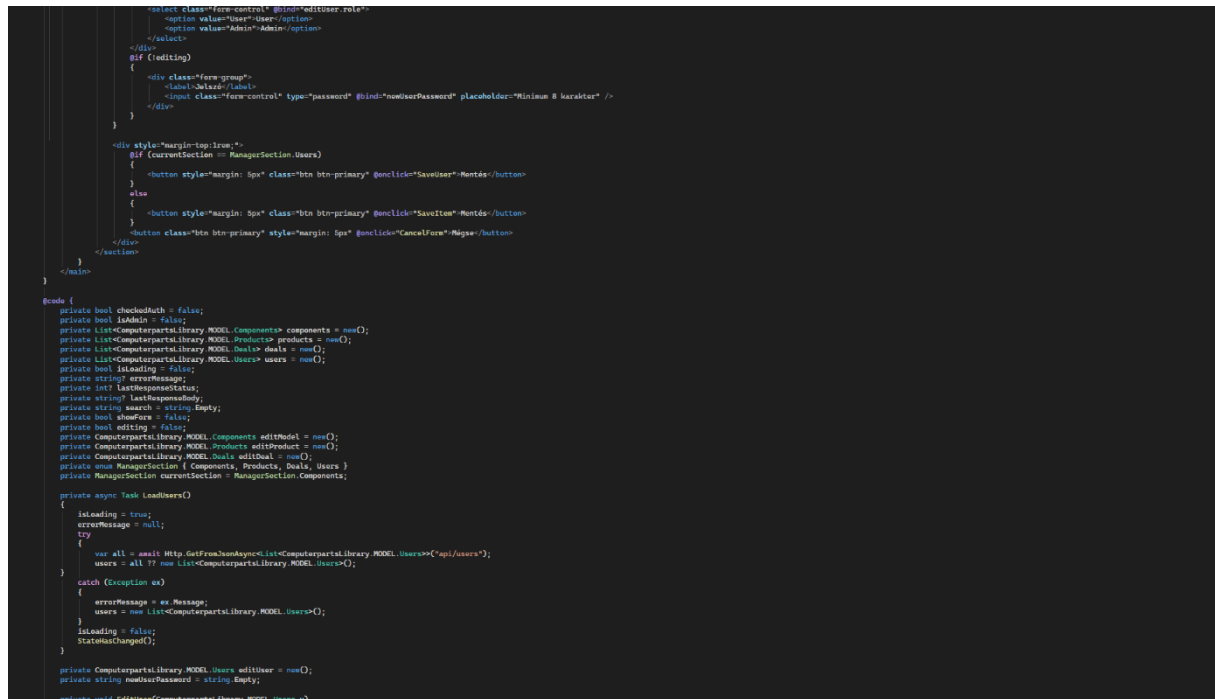
Ez a felépítés (SPA - Single Page Application megközelítés) rendkívül gyors válaszidőt eredményez, hiszen a fülváltás nem igényel teljes oldalfrissítést. A kód szervezettsége biztosítja, hogy minden adminisztrációs feladat (legyen az törlés, módosítás vagy listázás) logikailag elkülönüljön egymástól a felületen belül.

Adatkezelő modulok: Felhasználók és Címek

A felület ezen szakasza a rendszer alapvető entitásainak (felhasználók és szállítási címek) listázásáért és kezeléséért felel. A kód jól szemlélteti a Blazor dinamikus adatkötési képességeit.

Felhasználói lista és műveletek

A felhasználók kezelésére szolgáló táblázat nemcsak adatokat jelenít meg, hanem közvetlen interakciót is lehetővé tesz.



5. ábra: A felhasználói adatok dinamikus listázása és a törlési funkció

- **Dinamikus adatsorok:** A `@foreach` (var user in FilteredUsers) ciklus végigfut a szűrt felhasználói listán, és minden rekordhoz létrehoz egy táblázatsort.
- **Szerepkör alapú jelölés:** A kód vizuálisan is megkülönbözteti a felhasználókat a szerepkörük (pl. Admin, User) alapján.
- **Törlési interakció:** Minden sor végén található egy törlés gomb, amely a `DeleteUser` metódust hívja meg. Fontos megjegyezni a felhasználói élményt javító megoldást: a gomb egy megerősítő kérdést (JavaScript confirm-szerű megoldást vagy belső állapotváltozót) válthat ki, mielőtt a végleges törlés megtörténne.

Cím adatok adminisztrációja (Addresses)

A következő szekció a rendszerben tárolt szállítási és számlázási címek áttekintését biztosítja.

```
private void EditUser(ComputerPartsLibrary.MODEL.Users u)
{
    // ensure admin is authenticated before editing
    if (!AuthService.IsAuthenticated)
    {
        Navigation.NavigateTo("/login");
        return;
    }

    editing = true;
    editUser = new ComputerPartsLibrary.MODEL.Users
    {
        id = u.id,
        username = u.username,
        email = u.email,
        role = u.role,
        created_at = u.created_at,
        password_hash = u.password_hash
    };
    showForm = true;
}

private async Task DeleteUser(int id)
{
    // ensure admin is authenticated before deleting
    if (!AuthService.IsAuthenticated)
    {
        Navigation.NavigateTo("/login");
        return;
    }

    try
    {
        var resp = await AuthService.DeleteAsync($"api/users/{id}");
        if (!resp.IsSuccessStatusCode)
        {
            var txt = await resp.Content.ReadAsStringAsync();
            errorMessage = !string.IsNullOrEmpty(txt) ? txt : "Failed to delete user";
        }
        await LoadItems();
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
    }
}

private System.Globalization.CultureInfo currencyFormat = System.Globalization.CultureInfo.GetCultureInfo("en-US");

protected override async Task OnInitializedAsync()
{
    // Ensure auth state is loaded
    await AuthService.InitializeAsync();
    checkedAuth = true;
    isAdmin = string.Equals(AuthService.Role, "Admin", StringComparison.OrdinalIgnoreCase);
    if (!isAdmin)
    {
        StateHasChanged();
        return;
    }
    await LoadItems();
}

private void SetSection(ManagerSection sec)
{
    currentSection = sec;
    search = string.Empty;
    _ = LoadItems();
}
```

6. ábra: A címek kezelésének HTML szerkezete

- **Keresés és szűrés:** Hasonlóan a felhasználókhoz, a címeknél is található egy keresőmező (searchTermAddresses), amely azonnal szűri a listát a beírt karakterek alapján.
- **Adatszerkezet megjelenítése:** A táblázat tartalmazza a címek összes fontos elemét: Irányítószám, Város, Utca, Házsám.
- **Kapcsolt adatok:** Bár a táblázat a címeket mutatja, logikailag ezek a UserId mezőn keresztül kapcsolódnak a felhasználókhoz, így az adminisztrátor látja, melyik cím kihez tartozik.

Implementációs sajátosság:

A kód ezen részén látszik a "DRY" (Don't Repeat Yourself) elv alkalmazása: a fejlesztők hasonló szerkezeti felépítést alkalmaztak minden egyes admin modulnál (Címsor -> Kereső -> Táblázat), ami egységes és professzionális megjelenést kölcsönöz az egész adminisztrátori felületnek.

Kereskedelmi modulok: Termékek és Kategóriák

Ebben a szekcióban az adminisztrátor a webáruház "digitális polcait" kezelheti. Itt történik a késztermékek paraméterezése és a logikai csoportosításuk (kategóriák) karbantartása.

Termékkatalógus kezelése (Products)

A termékek táblázata komplexebb, mint a korábbiak, mivel több üzleti adatot (ár, kategória azonosító) kell egyszerre megjelenítenie és szerkeszthetővé tennie.

```
private async Task LoadItems()
{
    switch (currentSection)
    {
        case ManagerSection.Components:
            await LoadComponents();
            break;
        case ManagerSection.Products:
            await LoadProducts();
            break;
        case ManagerSection.Deals:
            await LoadDeals();
            break;
        case ManagerSection.Users:
            await LoadUsers();
            break;
    }
}

private async Task LoadComponents()
{
    isLoading = true;
    errorMessage = null;
    try
    {
        var all = await Http.GetFromJsonAsync<List<Computerpartslibrary.MODEL.Components>>("api/components");
        if (all == null)
        {
            components = new List<Computerpartslibrary.MODEL.Components>();
        }
        else
        {
            if (string.IsNullOrEmpty(search)) components = all;
            else components = all.Where(x => !string.IsNullOrEmpty(x.name) && x.name.Contains(search, StringComparison.OrdinalIgnoreCase)).ToList();
        }
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
        components = new List<Computerpartslibrary.MODEL.Components>();
    }
    isLoading = false;
    // Load component types for name resolution
    try
    {
        await LoadComponentTypes();
    }
    catch { }
    StateHasChanged();
}

private List<CP.Component_type> componentTypes = new();
private Dictionary<int, string> componentTypeLookup = new();

private async Task LoadComponentTypes()
{
    try
    {
        var all = await Http.GetFromJsonAsync<List<CP.Component_type>>("api/componenttypes");
        componentTypes = all ?? new List<CP.Component_type>();
        componentTypeLookup = componentTypes.Where(t => t != null).ToDictionary(t => t.id, t => t.name ?? string.Empty);
    }
    catch
    {
        componentTypes = new List<CP.Component_type>();
        componentTypeLookup = new Dictionary<int, string>();
    }
}
```

7. ábra: A termékek listázása és a szerkesztési/törlési opciók

- **Összetett adatmegjelenítés:** A táblázat nemcsak a termék nevét és ID-ját, hanem a kategória azonosítóját (CategoryId), a leírást és az árat is tartalmazza.
- **Műveleti gombok:** Minden terméksor végén két fontos gomb található:
 - **Szerkesztés (Edit):** Meghívja a PrepareProductEdit metódust, amely betölti az adott termék adatait egy szerkesztő űrlapba.
 - **Törlés (Delete):** A DeleteProduct metóduson keresztül véglegesen eltávolítja a terméket a kínálatból.

- **Keresési funkció:** A `searchTermProducts` változó segítségével az adminisztrátor pillanatok alatt megtalálhatja a módosítani kívánt árucikket a neve alapján.

Kategóriarendszer adminisztrációja (Categories)

A kategóriák kezelése biztosítja a webshop strukturált felépítését. Ez a modul felelős a rendszerezési szintek (pl. Monitorok, Billentyűzetek) fenntartásáért.

```
private string GetTypeName(int id)
{
    if (componentTypeLookup != null && componentTypeLookup.TryGetValue(id, out var name) && !string.IsNullOrEmpty(name))
        return name;
    return "ismeretlen";
}

private async Task LoadProducts()
{
    isLoading = true;
    errorMessage = null;
    try
    {
        var all = await Http.GetFromJsonAsync<List<Computerpartslibrary.MODEL.Products>>("api/products");
        if (all == null) products = new List<Computerpartslibrary.MODEL.Products>();
        else products = string.IsNullOrEmpty(search) ? all : all.Where(x => !string.IsNullOrEmpty(x.name) && x.name.Contains(search, StringComparison.OrdinalIgnoreCase)).ToList();
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
        products = new List<Computerpartslibrary.MODEL.Products>();
    }
    isLoading = false;
    StateHasChanged();
}

private async Task LoadDeals()
{
    isLoading = true;
    errorMessage = null;
    try
    {
        var all = await Http.GetFromJsonAsync<List<Computerpartslibrary.MODEL.Deals>>("api/deals");
        if (all == null) deals = new List<Computerpartslibrary.MODEL.Deals>();
        else deals = string.IsNullOrEmpty(search) ? all : all.Where(x => !string.IsNullOrEmpty(x.name) && x.name.Contains(search, StringComparison.OrdinalIgnoreCase)).ToList();
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
        deals = new List<Computerpartslibrary.MODEL.Deals>();
    }
    isLoading = false;
    StateHasChanged();
}

private void ShowAddForm()
{
    // ensure admin is authenticated before adding
    if (!AuthService.IsAuthenticated)
    {
        Navigation.NavigateTo("/login");
        return;
    }

    editing = false;
    editModel = new Computerpartslibrary.MODEL.Components();
    editProduct = new Computerpartslibrary.MODEL.Products();
    editDeal = new Computerpartslibrary.MODEL.Deals();
    editUser = new Computerpartslibrary.MODEL.Users();

    // ensure required non-null fields have defaults to avoid server-side model/db errors
    editProduct.image_url = editProduct.image_url ?? string.Empty;
    editProduct.description = editProduct.description ?? string.Empty;
    editUser.password_hash = editUser.password_hash ?? string.Empty;
    editUser.created_at = DateTimeOffset.UtcNow;
    showForm = true;
}
```

8. ábra: A kategóriák listázása és a hozzájuk tartozó műveletek

- **Egyszerűsített kezelőfelület:** Mivel a kategóriák kevesebb attribútummal rendelkeznek, a táblázat átláthatóbb, elsősorban a kategória nevére fókuszál.
- **Keresés és szűrés:** Itt is megjelenik a standard adminisztrátori mintázat: egy dedikált keresőmező (`searchTermCategories`) és a dinamikusan frissülő táblázat.
- **Karbantartás:** Lehetőséget biztosít új kategóriák létrehozására vagy a régiek átnevezésére/törlésére, ami közvetlen hatással van a frontend oldali navigációs menüre.

Fejlesztői megfigyelés:

A kód ezen részein látható, hogy a törlési funkciók (`DeleteProduct`, `DeleteCategory`) aszinkron módon, a háttérben futnak le, miközben a UI azonnal frissül. Ez a reaktív viselkedés a Blazor

State Management egyik nagy előnye: az adminisztrátor azonnali visszajelzést kap a művelet sikeréről a sor eltűnésével.

Technikai modulok: Alkatrészek és Típusok

Ez a szekció az egyedi PC-építés alapköveit kezeli. Itt az adminisztrátorok nemcsak a termékeket, hanem a mélyebb technikai specifikációkkal rendelkező alkatrészeket is paraméterezhetik.

Alkatrész-katalógus kezelése (Components)

Az alkatrészek kezelése az egyik legrészletesebb táblázatot igényli, mivel itt a technikai jellemzők (fogyasztás, teljesítmény, kompatibilitás) is megjelennek.

```
private void EditComponent(ComputerpartsLibrary.MODEL.Components c)
{
    editing = true;
    editModel = new ComputerpartsLibrary.MODEL.Components
    {
        id = c.id,
        type_id = c.type_id,
        sku = c.sku,
        name = c.name,
        description = c.description,
        price = c.price
    };
    showForm = true;
}

private void EditProduct(ComputerpartsLibrary.MODEL.Products p)
{
    editing = true;
    editProduct = new ComputerpartsLibrary.MODEL.Products
    {
        id = p.id,
        category_id = p.category_id,
        name = p.name,
        sku = p.sku,
        base_price = p.base_price,
        image_url = p.image_url,
        description = p.description
    };
    showForm = true;
}

private void EditDeal(ComputerpartsLibrary.MODEL.Deals d)
{
    editing = true;
    editDeal = new ComputerpartsLibrary.MODEL.Deals
    {
        id = d.id,
        name = d.name,
        description = d.description,
        price = d.price
    };
    showForm = true;
}

private void CancelForm()
{
    showForm = false;
}

private async Task SaveComponent()
{
    try
    {
        if (editing)
        {
            var resp = await AuthService.PutAsJsonAsync($"api/components/{editModel.id}", editModel);
            if (!resp.IsSuccessStatusCode)
            {
                errorMessage = await resp.Content.ReadAsStringAsync();
            }
        }
        else
        {
            var resp = await AuthService.PostAsJsonAsync("api/components", editModel);
            if (!resp.IsSuccessStatusCode)
            {
                errorMessage = await resp.Content.ReadAsStringAsync();
            }
        }
        showForm = false;
        await LoadComponents();
    }
}
```

9. ábra: Az alkatrészek listázása és a technikai adatok áttekintése

- **Részletes adatmegjelenítés:** A táblázat a név mellett tartalmazza a gyártót (Manufacturer), a típust (TypeId), az árat és a leírást is.
- **Keresési hatékonyság:** A searchTermComponents segítségével az adminisztrátor szűrhet gyártóra vagy konkrét modellre is, ami elengedhetetlen egy több száz darabos alkatrész-adatbázis esetén.
- **Műveletek:** Megmarad a konzisztens adminisztrációs minta: a szerkesztés (PrepareComponentEdit) és a törlés (DeleteComponent) gombok minden sor végén elérhetőek.

Alkatrésztípusok adminisztrációja (Component Types)

Ez a modul a kategóriarendszerhez hasonlóan működik, de kifejezetten a hardverek technikai besorolására (pl. Processzor, Alaplap, Videókártya) szolgál.

```
        catch (Exception ex)
        {
            errorMessage = ex.Message;
        }
    }

    private async Task SaveItem()
    {
        try
        {
            if (currentSection == ManagerSection.Components)
            {
                if (editing)
                {
                    var resp = await AuthService.PutAsJsonAsync($"api/components/{editModel.id}", editModel);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
                else
                {
                    var resp = await AuthService.PostAsJsonAsync("api/components", editModel);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
            }
            else if (currentSection == ManagerSection.Products)
            {
                if (editing)
                {
                    var resp = await AuthService.PutAsJsonAsync($"api/products/{editProduct.id}", editProduct);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
                else
                {
                    var resp = await AuthService.PostAsJsonAsync("api/products", editProduct);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
            }
            else if (currentSection == ManagerSection.Deals)
            {
                if (editing)
                {
                    var resp = await AuthService.PutAsJsonAsync($"api/deals/{editDeal.id}", editDeal);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
                else
                {
                    var resp = await AuthService.PostAsJsonAsync("api/deals", editDeal);
                    lastResponseStatus = (int)resp.StatusCode;
                    lastResponseBody = await resp.Content.ReadAsStringAsync();
                    if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
                }
            }

            showForm = false;
            await LoadItems();
        }
        catch (Exception ex)
        {
            errorMessage = ex.Message;
        }
    }
}
```

10. ábra: Az alkatrésztípusok listázása és karbantartása

- **Strukturális alapok:** Itt határozható meg, hogy a PC-építőben milyen típusú komponensek közül választhat a vásárló.
- **Egyszerűsített interfész:** A kategóriákhoz hasonlóan itt is a típus neve a mérvadó. A keresőmező (searchTermComponentTypes) biztosítja a gyors navigációt.
- **Rendszerintegritás:** Az itt végzett módosítások közvetlenül befolyásolják a PC-építő logika működését, hiszen a rendszer a típusok alapján válogatja szét a kompatibilis elemeket.

Technikai megvalósítási részletek:

A kód ezen szakaszaiban is megfigyelhető a Blazor @foreach ciklusainak hatékony használata. A rendszer a háttérben injektált IComponentService és IComponentTypeService segítségével

frissíti az adatokat, biztosítva, hogy a módosítások azonnal érvénybe lépjenek mind az admin felületen, mind a publikus áruházban.

Rendeléskezelés és a komponens logikája

Az Admin.razor utolsó szakaszában a kereskedelmi folyamatok lezárása (rendelések) és az egész oldalt mozgató C# kód inicializálása kapott helyet.

Rendelések felügyelete (Orders)

A rendelések modul az adminisztrátor számára kritikus, hiszen itt követhető nyomon az üzleti forgalom és a vásárlói igények.

```
// Save user from users section
private async Task SaveUser()
{
    try
    {
        // basic client-side validation
        if (string.IsNullOrWhiteSpace(editUser.username))
        {
            try { await JS.InvokeVoidAsync("showToast", "Felhasználónév megadása kötelező.", "warning", 4000, "bottom-center"); } catch { }
            return;
        }
        if (string.IsNullOrWhiteSpace(editUser.email))
        {
            try { await JS.InvokeVoidAsync("showToast", "Email megadása kötelező.", "warning", 4000, "bottom-center"); } catch { }
            return;
        }
        // if creating a new user, require a password and use register endpoint
        if (!editing)
        {
            // client-side duplicate check to provide immediate feedback
            if (users != null && users.Any(u => string.Equals(u.username, editUser.username, StringComparison.OrdinalIgnoreCase)))
            {
                try { await JS.InvokeVoidAsync("showToast", "Ez a felhasználónév már foglalt.", "warning", 4000, "bottom-center"); } catch { }
                return;
            }
            if (users != null && users.Any(u => string.Equals(u.email, editUser.email, StringComparison.OrdinalIgnoreCase)))
            {
                try { await JS.InvokeVoidAsync("showToast", "Ez az email cím már regisztrálva van.", "warning", 4000, "bottom-center"); } catch { }
                return;
            }
            if (string.IsNullOrWhiteSpace(newUserPassword) || newUserPassword.Length < 8)
            {
                try { await JS.InvokeVoidAsync("showToast", "Jelszó megadása kötelező (min. 8 karakter).", "warning", 4000, "bottom-center"); } catch { }
                return;
            }
        }
        // call registration endpoint to ensure proper hashing and token generation
        var payload = new
        {
            Username = editUser.username,
            Email = editUser.email,
            Password = newUserPassword
        };
        var resp = await Http.PostAsJsonAsync("api/auth/register", payload);
        LastResponseStatus = (int)resp.StatusCode;
        LastResponseBody = await resp.Content.ReadAsStringAsync();
        if (resp.IsSuccessStatusCode)
        {
            // success toast
            try { await JS.InvokeVoidAsync("showToast", "Felhasználó sikeresen hozzáadva.", "success", 4000, "bottom-center"); } catch { }
            showForm = false;
            newUserPassword = string.Empty;
            editUser = new ComputerPartsLibrary.MODEL.Users();
            await LoadItems();
            return;
        }
        else
        {
            try { await JS.InvokeVoidAsync("showToast", string.IsNullOrEmpty(LastResponseBody) ? "Hiba a regisztráció során." : LastResponseBody, "danger", 6000, "bottom-center"); } catch { }
            return;
        }
    }
}
```

11. ábra: A beérkezett rendelések listázása

- **Tranzakciós adatok:** A táblázat megjeleníti a rendelés azonosítóját, a vásárló ID-ját, a rendelés pontos dátumát és a végösszeget.
- **Átláthatóság:** A korábbi modulokhoz hasonlóan itt is működik a keresési funkció (searchTermOrders), így egy konkrét rendelésszámmra vagy dátumra is könnyen rá lehet keresni.
- **Műveletek:** Lehetőség van a rendelések törlésére vagy állapotuk ellenőrzésére, ami elengedhetetlen a logisztikai folyamatokhoz.

A háttérlogika kezdete (C# Code Block)

A fájl végén található a `@code` blokk, ahol a komponens belső állapota és a működésért felelős metódusok definiálásra kerültek.

```

    if ([editing])
    {
        var resp = await AuthService.PutAsJsonAsync($"api/users/{editUser.id}", editUser);
        lastResponseStatus = (int)resp.StatusCode;
        lastResponseBody = await resp.Content.ReadAsStringAsync();
        if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
    }
    else
    {
        var resp = await AuthService.PostAsJsonAsync("api/users", editUser);
        lastResponseStatus = (int)resp.StatusCode;
        lastResponseBody = await resp.Content.ReadAsStringAsync();
        if (!resp.IsSuccessStatusCode) errorMessage = lastResponseBody;
    }
    showForm = false;
    await LoadItems();
}
catch (Exception ex)
{
    errorMessage = ex.Message;
}
}

private async Task DeleteItem(int id)
{
    try
    {
        HttpResponseMessage resp = null;
        if (currentSection == ManagerSection.Components) resp = await AuthService.DeleteAsync($"api/components/{id}");
        else if (currentSection == ManagerSection.Products) resp = await AuthService.DeleteAsync($"api/products/{id}");
        else if (currentSection == ManagerSection.Deals) resp = await AuthService.DeleteAsync($"api/deals/{id}");
        else if (currentSection == ManagerSection.Users) resp = await AuthService.DeleteAsync($"api/users/{id}");
        if (resp != null && !resp.IsSuccessStatusCode) errorMessage = await resp.Content.ReadAsStringAsync();
        await LoadItems();
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
    }
}

// delete confirmation modal state
private bool showDeleteConfirm = false;
private int? pendingDeleteId = null;

private void ShowDeleteConfirm(int id)
{
    pendingDeleteId = id;
    showDeleteConfirm = true;
}

private void CancelDelete()
{
    pendingDeleteId = null;
    showDeleteConfirm = false;
}

private async Task ConfirmDeleteAsync()
{
    if (!pendingDeleteId.HasValue) return;
    var id = pendingDeleteId.Value;
    pendingDeleteId = null;
    showDeleteConfirm = false;
    await DeleteItem(id);
}

private async Task DeleteComponent(int id)
{
    try
    {
        await Http.DeleteAsync($"api/components/{id}");
        await LoadComponents();
    }
    catch (Exception ex)
    {
        errorMessage = ex.Message;
    }
}
}

```

12. ábra: A változók deklarálása és a szervizek használatának előkészítése

- **Állapotjelzők (Tabs):** Itt látható a `currentTab` változó, amely alapértelmezett értéként a "users" szekciót kapja meg, így az oldal betöltésekor mindig a felhasználói lista jelenik meg először.

- **Adatlisták:** A kód ezen része definiálja azokat a gyűjteményeket (pl. users, addresses, products, orders), amelyek az API-ból érkező adatokat tárolják a memóriában a megjelenítéshez.
- **Keresési változók:** Itt kaptak helyet a különböző modulokhoz tartozó searchTerm stringek is, amelyek a kétirányú adatkötés (two-way binding) segítségével azonnal reagálnak a felhasználó gépelésére.

Összegzés:

Ezzel végére értünk az Admin.razor szerkezeti bemutatásának. A fájl monumentalitása ellenére a felépítése szigorúan követi a Blazor komponens-alapú fejlesztési elveit. A HTML rész felel a reszponzív megjelenítésért, míg a C# blokk biztosítja az aszinkron kommunikációt a backend API-val, lehetővé téve a valós idejű adatmódosítást.

Felhasználói dokumentáció

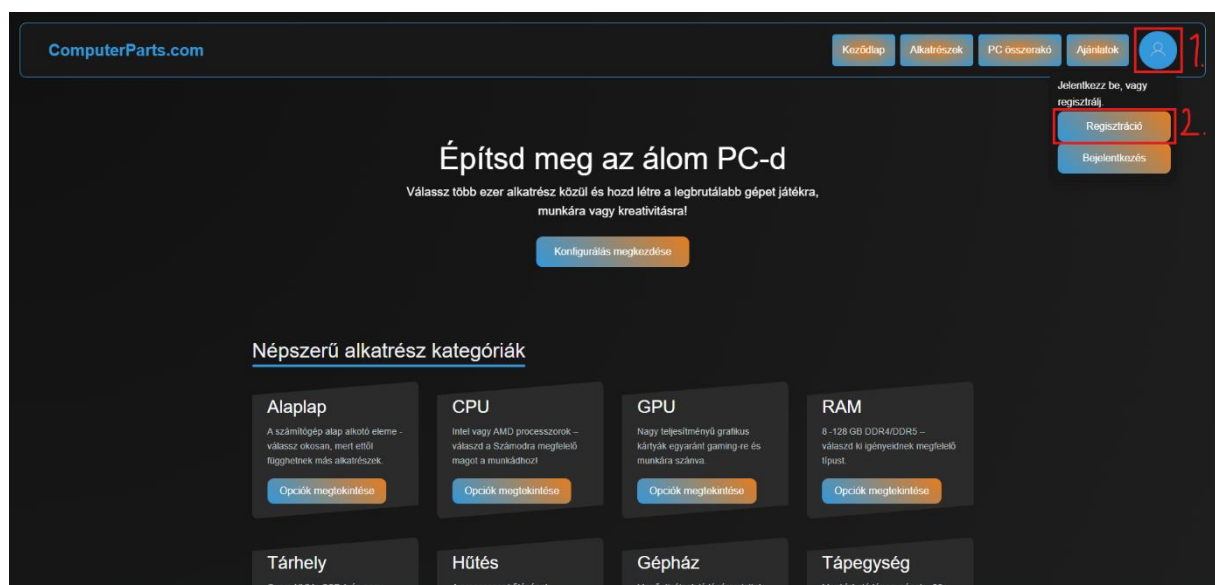
Felhasználói felületek

Biztosított Felhasználói bejelentkezési adatok:

Felhasználói fiók felh.név és jelszó páros: TesztElek123, TesztElek123

Oldalunk fő funkciója, a saját konfiguráció összeállítása és mentése a lehető legegyszerűbben zajlik le a kódolás oldalán is, de mi a helyzet a felhasználói oldalon? Nos, itt sem különb a helyzet: a felhasználónak egyáltalán nincsen szüksége házi atomerőművekre, Nasa-s technológiákra a használatához, ráadásul sok esetben, főként mobil környezetben, még csak telepítésre sincs szüksége ahhoz, hogy használni tudja az oldalunk nyújtotta előnyöket, funkciókat. A felhasználónak nincs más dolga, mint hogy megkeresi a „www.computerpartstest.com„ oldalat a telefonjára már gyárilag telepített böngészőben (főként Google Chrome lehet, de elképzelhető „Samsung Internet” a koreai cég telefonjainál, vagy ha a felhasználó a saját igényeinek megfelelő böngészőt telepített, akkor valószínűleg „Firefox”, „Brave”, „Opera” vagy „DuckDuckGo” lehető fel eszközén, iOS esetén pedig a „Safari” elérhető opció), és már is a főoldalunkon találja magát a felhasználó. Innentől pedig a felhasználó két dolgot tehet: használhatja már az oldal szinte összes elérhető funkcióját (leszámítva azokat, amik regisztrációt és bejelentkezést igényelnek, a többi funkció leírása lentebb olvasható), vagy pedig megteszi a szükséges regisztrációt, ha még nincs fiókja, vagy bejelentkezik a már meglévőbe (általában előbbi szükséges utóbbihoz, ha ezen sorok olvasása előtt még sosem járt oldalunkon és nem regisztrált be). A regisztrációhoz és onnan a bejelentkezéshez az alábbi útvonalat kell követni:

A webhely betöltődése után jobb fent a profilkép ikonra kattintva megjelenik egy lenyíló menü, itt a „Regisztráció” gombra kattintsunk.



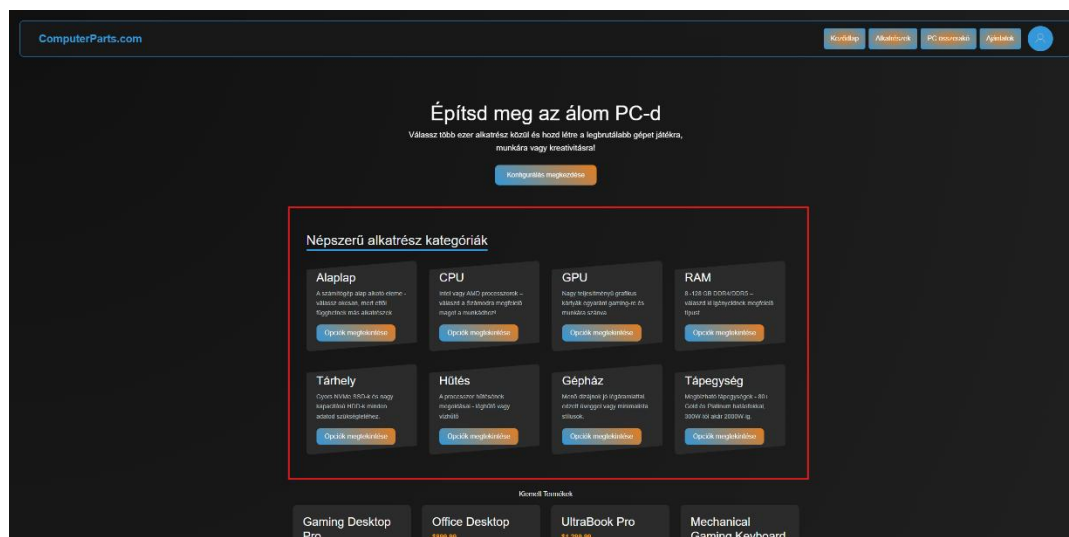
Miután megjelenik a regisztrációs oldal, a felhasználónak meg kell adnia egy kitalált felhasználó nevet *(ha szeretné, lehet a valódi neve is)*, ehhez egy email címet *(tesztelés esetén érdemes a @example.com véget használni)*, és a jelszavát kétszer *(a rendszer ellenőrzi, hogy jól írta-e be a felhasználó mindkétyszer a jelszavát, más különben hibát fog dobni)*, végül pedig a lenti nagy „Regisztrálás” gombra kell kattintania.

The screenshot shows the 'Regisztráció' (Registration) form. It has a dark background with white text. The form fields are: 'Felhasználónév' (Username), 'Email', 'Jelszó' (Password), and 'Jelszó megerősítése' (Confirm Password). A large orange 'Regisztrálás' (Register) button is at the bottom. Red boxes and numbers highlight specific elements: 3. points to the Username field, 4. points to the Email field, 5. points to the Password field, 6. points to the Confirm Password field, and 7. points to the Register button. Below the button, there is a link 'Térj vissza a bejelentkezéshez...' and a link 'Vissza a kezdőlapra Kezdőlap'.

Miután a felhasználó rá kattintott a „Regisztrálás” gombra, a rendszer automatikusan át fogja irányítani a bejelentkező oldalra *(ha nem történik meg, akkor a felhasználónak rá kell kattintani a fentebbi képen látható „Térj vissza a bejelentkezéshez...” szövegre, ami szintén átirányítja a bejelentkező oldalra)*, ahol már csak az imént beregisztrált felhasználónevet és jelszót kell beírnia *(ezt érdemes valahova felírni, ne hogy elfelejtsük! :P)*, majd a „Bejelentkezés” gombra kattintva a rendszer átirányítja a felhasználót ismét a főoldalra.

The screenshot shows the 'Bejelentkezés' (Login) form. It has a dark background with white text. The form fields are: 'Email vagy Felhasználónév' (Email or Username) and 'Jelszó' (Password). A large orange 'Bejelentkezés' (Login) button is at the bottom. Red boxes and numbers highlight specific elements: 8. points to the Email or Username field, 9. points to the Password field, and 10. points to the Login button. Below the button, there is a link 'Készíts egyet itt...' and a link 'Vissza a kezdőlapra Kezdőlap'.

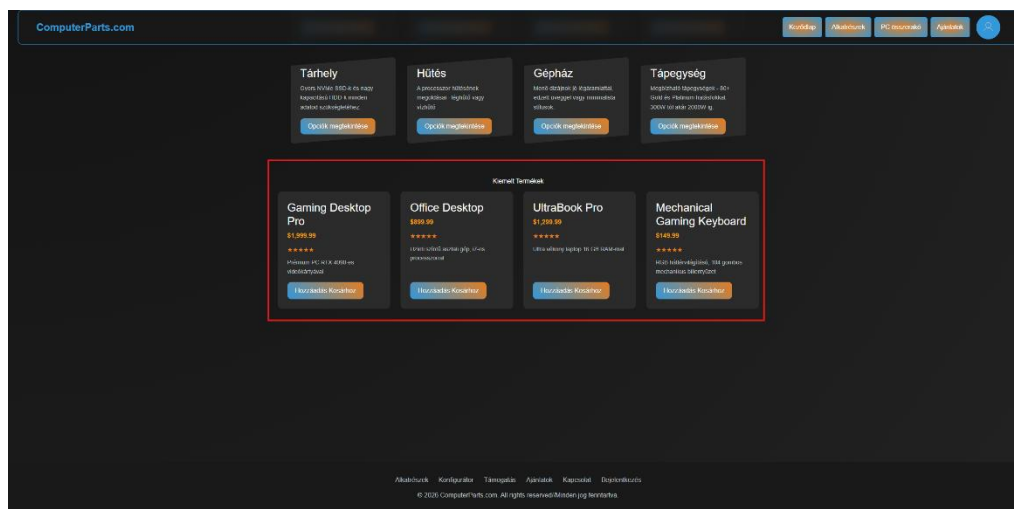
És ezzel a felhasználó létre hozta fiókját és be is jelentkezett abba, így innentől már az oldal összes funkcióját használhatja a felhasználó *(azokat is, amelyek regisztrációhoz kötöttek, ezeket lentebb szintén említettük, mely funkciók kötöttek regisztrációhoz)*. Windows környezetben legfeljebb annyi a különbség, hogy vagy az előre telepített Microsoft Edge nevű böngészőt használja a felhasználó, vagy pedig a saját igényeinek megfelelőt, mint Google Chrome, Brave, stb. A felhasználónak innentől, hogy vissza tért a főoldalra, több lehetősége is van: akár egyből maradhat is a főoldalon, ahhol körül tud nézni a különböző kategóriáink aktuálisan elérhető termékei között. Processzorra van szükség? Esetleg megunta a jelenlegi gépház kinézetét vagy nem fér bele valami? Netán nincs már megelégedve a játék vagy munka közben tapasztalható videókártya teljesítménnyel? Bármelyik is legyen, a főoldalon egyetlen egy kattintással megtekintheti valamelyik kategória elérhető darabjait! A felhasználónak nincs más dolga, csak valamelyik kisebb ablak gombjára kell kattintania a „Népszerű alkatrész kategóriák” alatt és már el is kezdhet böngészni a termékek között. Ha már egy adott kategóriában, például RAM-ok között már megtalálta az ideális választást, a felugró ablakon, amin az adott kategória termékei megjelennek, csak a jobb alsó sarokban megjelenő „Bezár” gombra kell kattintania és pontosan ugyan onnan eléri az összes többi kategóriát a főoldalon, mint a példában említettét.



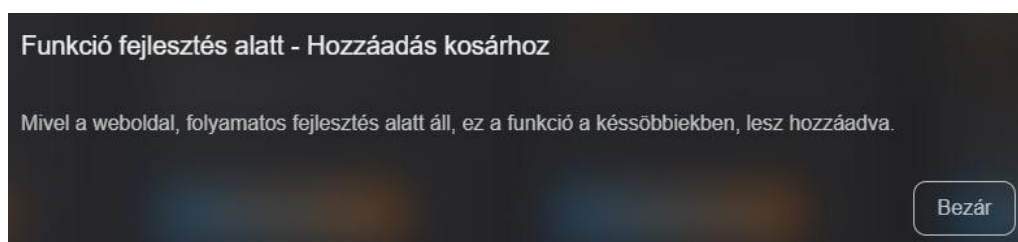
Jelen esetben a példa ablak a processzoré.



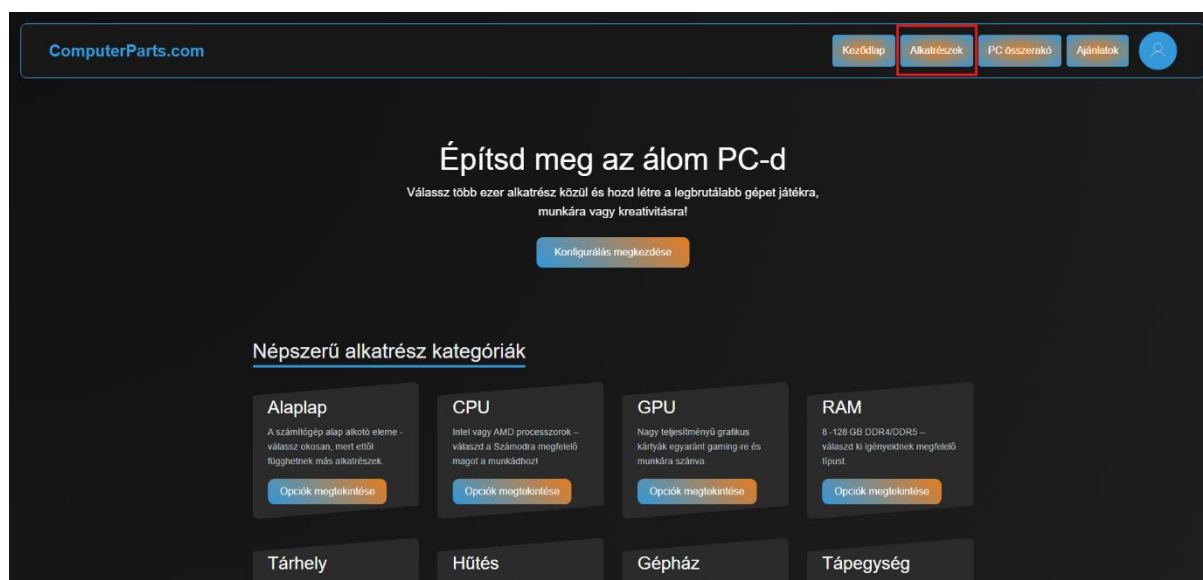
Ha pedig a felhasználó lejjebb görget, néhány aktuálisan elérhető promóciót találhat meg, azonban fontos megjegyezni, hogy itt még csak az ajánlat egy rövidített leírását olvashatja el a felhasználó, ha a „Hozzáadás Kosárhoz” gombra kattint a felhasználó, a felugró ablakon csak egy üzenetet fog tudni olvasni, ami azt írja, hogy jelenleg ez a funkció még nem készült el, viszont ez már fel lett véve a korábban is írt **Fejlesztési Ötletek** fejezetébe. A főoldal, emellett pedig minden oldal alján *(leszámitva a regisztrációs- és bejelentkező oldalakat)* megjelenik a lábléc, ahol a felhasználó szintén elérheti a bejelentkező oldalt, az „Alkatrészek”, a „PC Összerakó” és az ajánlatok oldalt, amik már a navigációs sávon is jelen vannak, valamint itt, a „Támogatás” gombra kattintva kérhet a felhasználó közvetlenül az oldalon segítséget, illetve itt találja a „Kapcsolat” gombot is, amire kattintva oldalunk kapcsolattartási módjait éri el. *(Megjegyzés: a footer-en található gombokról szintén készül használati leírás, ez alól a „Bejelentkezés” képez kivételt, mivel ez már fentebb el lett magyarázva. Azon gombok, amelyek szerepelnek mind a navigációs sávon, mind pedig a footer-en, csak egyszer lesznek elmagyarázva, ezek közé tartozik az „Alkatrészek”, a „PC Összerakó” és az „Ajánlatok” gomb.)*



Példa ablak az „Office Desktop”-hoz tartozó „Hozzáadás Kosárhoz” gombra kattintva, viszont bármelyikre kattint a felhasználó, ugyanezt az üzenetet olvashatja.

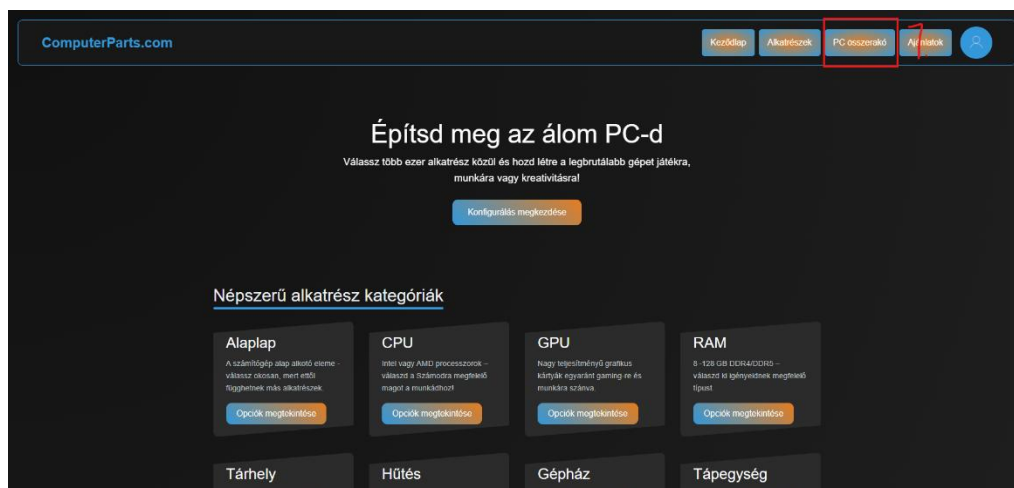


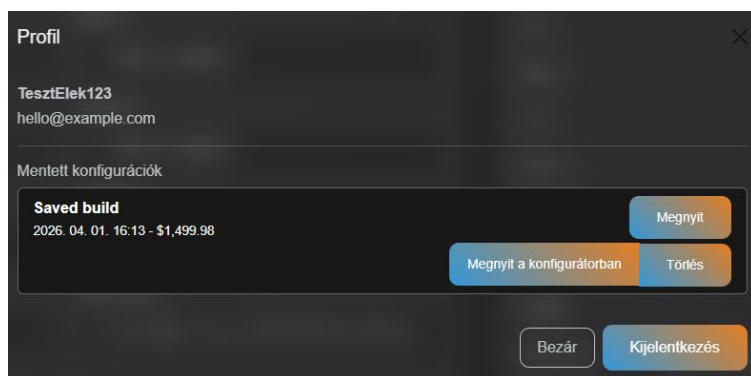
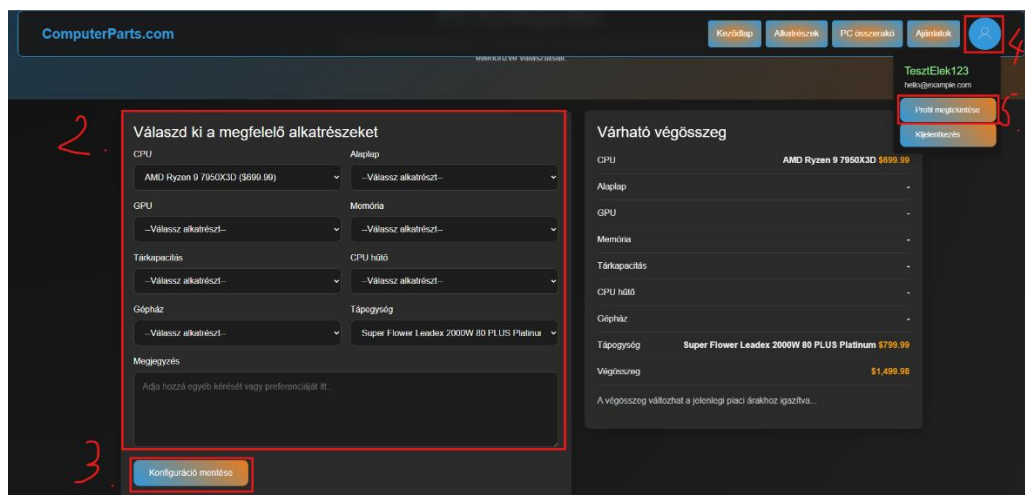
Viszont rátérve az „Alkatrészek” oldalra, ami jelen pillanatban sokkal inkább egy leírásként szolgál a különböző, elérhető kategóriákhoz, egyes egyedül a lap alsó felén látható két, nagyobb teret magáénak tevő ablaknak vannak saját, teljes mértékben működő gombjaik. Az első, az „Építés-kész szűrők” alján található gomb egyből átirányítja a felhasználót a navigációs sávban is látható „PC Összerakó” oldalra, míg a „Segítségre van szükség?” gombja a láblécezen látható „Támogatás” oldalra fogja átvezetni a felhasználót. Az alábbi képeken vizualizáltuk, hogyan juthat el a felhasználó **bármely** oldalról az „Alkatrészek” oldalra. *(Megjegyzés: minden esetben illusztrációként a főoldalról készült képet szúrtuk be, mivel a navigációs sáv mindegyik oldalon megegyezik, így megítélésünk szerint nincs szükség több, különböző kép beszúrására. Az „Alkatrészek oldalon” látható két, működő gombja, a „Konfigurátor megnyitása” és a „Segítségkérés” később kerül bemutatásra, amikor épp annál az oldalnál járunk, viszont ezekre a gombokra kattintva ugyan arra az oldalra jutunk el, mintha a navigációs sávon nyomjuk meg a „PC Összerakó” gombot, vagy a footer-en a „Konfigurátor” vagy „Támogatás” gombot.)*



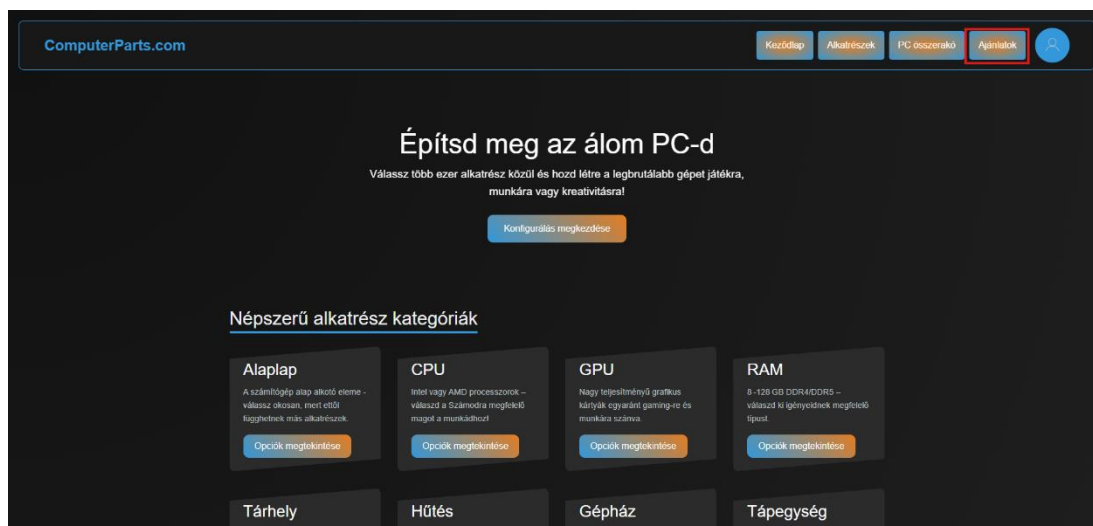
Oldalunk fő funkciója pedig itt kerül bemutatásra. A „PC Összerakó”, vagy szakmaibb nyelven „Konfigurátor” („*Configurator*”) szolgál arra, hogy a felhasználó egy átlátható, egyszerű, letisztult kezelőfelületen tudja megépíteni az álom számítógépét, amin nem csak az alkatrészeket válogathatja össze, hanem a konfigurációja árát is pontosan láthatja (*továbbra is átlagárakat használunk!*), valamint a saját fiókjába akár el is mentheti a kész álmogépet. Itt jön igazán jól a korábban bemutatott regisztráció és saját fiók használata, ugyan is regisztrált fiók nélkül a felhasználó nem tudja elmenteni az összeállított konfigot. A „PC Összerakó” oldal elérésének, a konfig összeállításának és mentésének, valamint a mentett konfig megtekintésének a menete így néz ki: a felhasználónak a navigációs sávon a „PC Összerakó” gombra kell kattintania (*ha nem a footer „Konfigurátor” gombjára kattintva jutott el ide*). Ekkor elé tárul a konfigurátor oldala, ahhol két, nagy helyet igénylő ablak fogja várni: a bal oldalin („*Válaszd ki a megfelelő alkatrészeket*”) nyolc (8) lenyíló menü található, a hozzájuk kapcsolódó termék kategória nevével felette. A felhasználónak nincsen egy adott összeállítási sorrend megszabva, tetszőleges sorrendben válogathatja ki a neki megfelelő alkatrészeket. Minden egyes komponens kiválasztása után a jobb oldali ablakon („*Várható végösszeg*”)

lista szerűen megjelenik az összes választott termék, a legalsó sorban pedig kiemelt színnel a listán szereplő összes termék ára összeadva. A felhasználó nincsen arra kötelezve, hogy mindegyik alkatrésznél válasszon egyet, akár egyetlen egy alkatrész kiválasztásával el tudja menteni az összeállított konfigurációt. Ha megtörtént a mentés, miután a felhasználó ellenőrizte, mindenhol annak az alkatrésznek a neve szerepel, amit szeretne és egyiknél sem kattintott mellé, a bal oldali ablakon („Válaszd ki a...”) a lent található „Konfiguráció mentése” gombra kattintva mentésre kerül fiókjába a megálmodott konfiguráció. Innen már csak a megtekintéshez annyit kell tennie, hogy a jobb felső sarokban található profil ikonra kattint, majd a lenyíló menüben a „Profil megtekintése” gombra kattint. A felugró ablakon megjelenik néhány profil adat, így a neve és email címe, valamint a mentett konfigurációk *(ha vannak)*. Itt a felhasználónak három lehetősége van a mentett konfigurációkkal kapcsolatban: ha a „Megnyit” gombra kattint, akkor a mentés dátuma és a konfiguráció ára mellett részletesebben jelenik meg a konfiguráció, tartalmazva a hozzá adott alkatrészeket. Ha a „Megnyit a konfigurátorban” gombra kattint, akkor a rendszer vissza vezeti a felhasználót a „PC Összerakó” oldalra annyi különbséggel, hogy a mentett konfigurációban szereplő alkatrészek már előre ki lesznek választva az adott kategóriának megfelelően *(például kiválasztja a processzort és tápegységet, majd menti a konfigurációt és megnyitja újra a konfigurátorban, a processzor és tápegység már ki lesz választva)*, így a felhasználónak már csak hozzá kell tennie azokat az alkatrészeket, amire szüksége van. A harmadik gomb, „Törlés” pedig értelem szerűen törölni fogja a mentett konfigurációt. Emellett a felugró ablakon még három gomb szerepel: a jobb alul lévő „Bezár”-ra vagy jobb fent található „X”-re kattintva a felugró ablakot zárhatja be a felhasználó, a „Kijelentkezés” gombbal pedig értelem szerűen kijelentkezhet fiókjából. Az alábbi képek segítenek tájékozódni ebben.

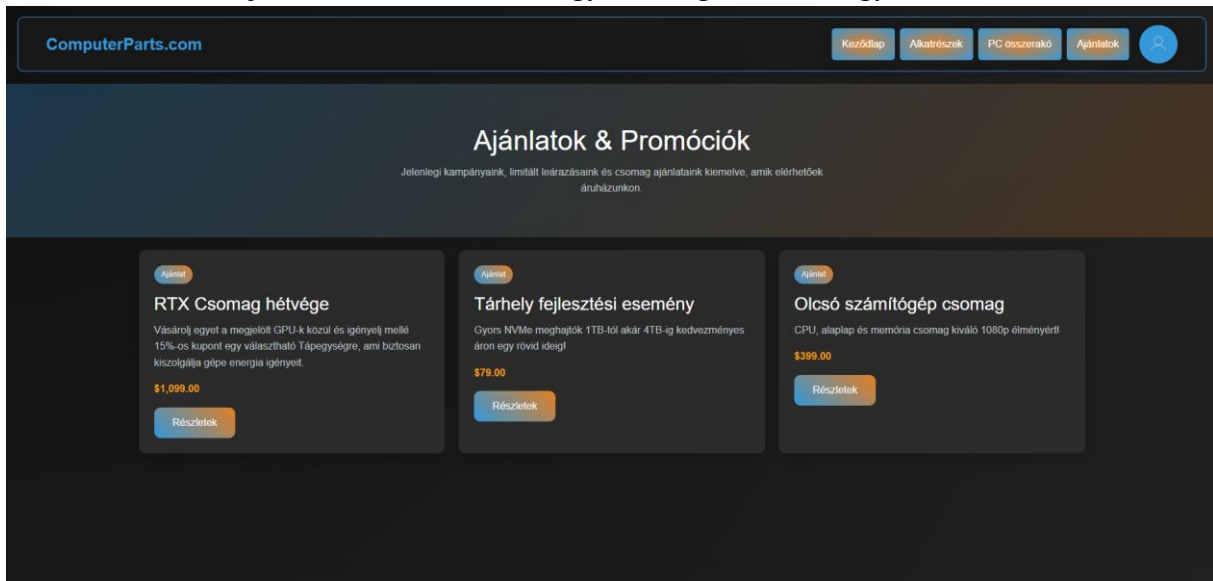




Az utolsó főbb oldal pedig, ami mind a navigációs sávon, mind pedig a láblécen megtalálható, az „Ajánlatok” gomb és oldal. Ezen az oldalon az éppen megtalálható promóciók, termék csomagok és akciók láthatók. Három kisebb ablak foglal helyet, ezek viszont csak illusztrációs céllal készültek el, a gombjaiknak egyelőre nincsen funkciója. Az oldal a képen látható módon érhető el.



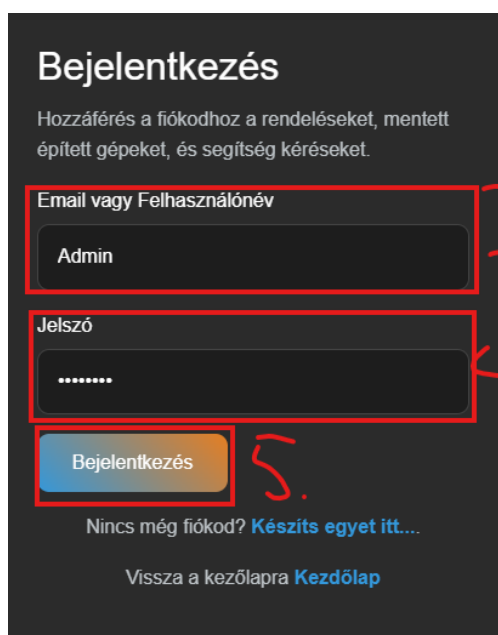
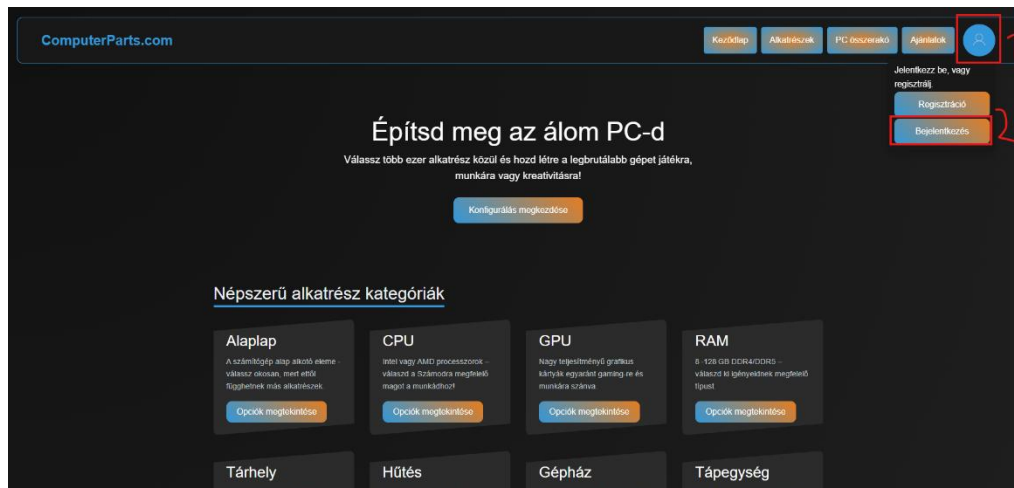
A gombokra kattintva egyelőre semmi nem történik, viszont ezeknek a beprogramozása szintén fel lett írva a fejlesztési tervek közé, így a megvalósítás egyáltalán nincsen messze!



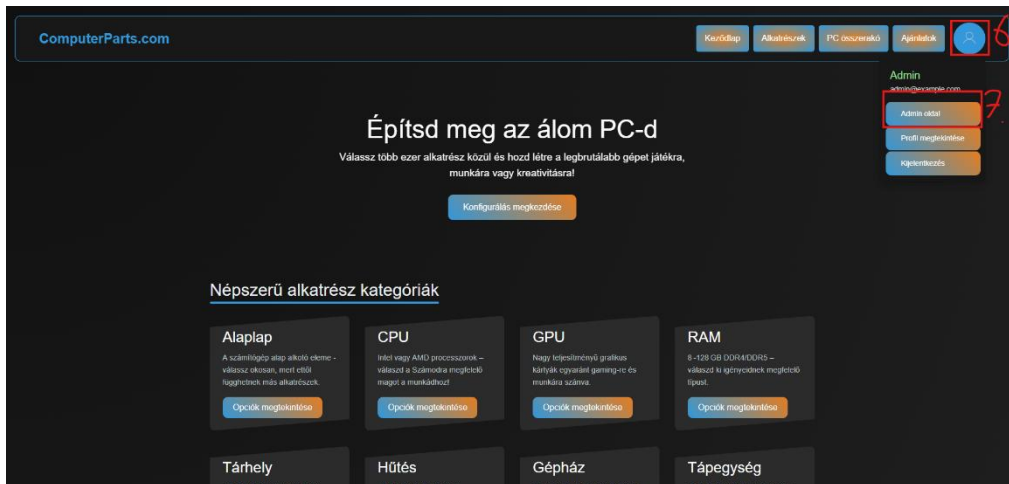
Adminisztrátor felületek

Adminisztrátori fiók felh.név és jelszó páros: Admin, 12345678

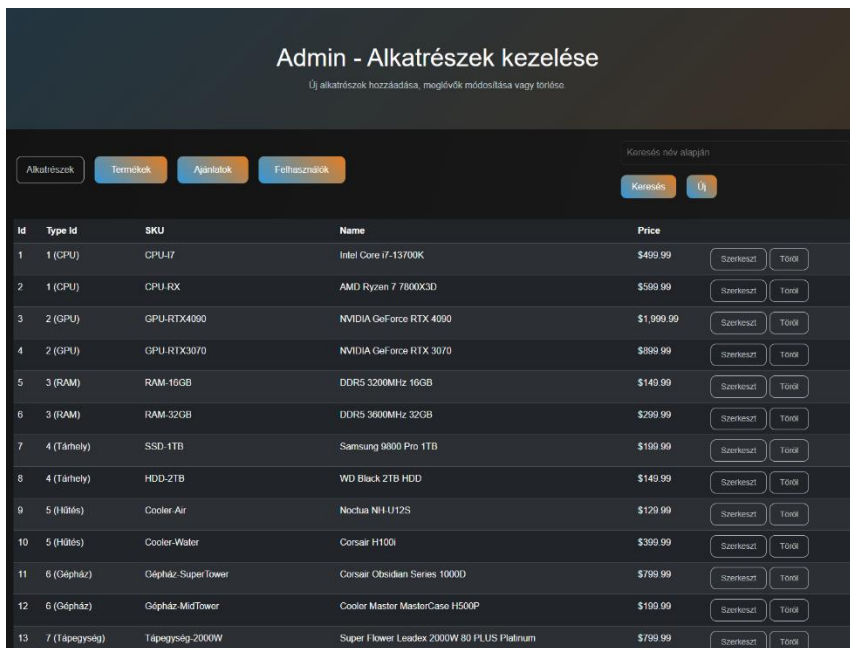
Adminisztrátori felülettel is rendelkezik oldalunk, ez viszont kizárólag a megfelelő besoroltsággal rendelkező személyek érhetik el (ehhez hoztunk létre egy külön adminisztrátor fiókot, amivel megtekinthető az Admin oldal, a hozzá szükséges bejelentkezési adatok megtalálhatóak a „Felhasználói dokumentáció” ezen oldalának tetején), itt pedig több különböző feladatot láthat el egy adminisztrátori jogokkal rendelkező felhasználó. A bejelentkezés pontosan ugyan úgy zajlik, mint egy normál felhasználó esetén: a jobb felső sarokban található profil ikonra kattintva lenyílik a már korábban is említett menü, itt viszont a „Regisztráció” gomb helyett egyből a „Bejelentkezés” gombra kell kattintani. Itt az adminnak meg kell adnia az admin fiók felhasználónevet és jelszavát, majd amint a rendszer visszairányította a főoldalra, egyből kintinthat is az admin ismét a profil ikonra, ahol a normál felhasználói fiók opciói mellett megjelenik az „Admin oldal” gomb is, erre kell kattintani. A képek vizuálisan is bemutatják, mit kell tenni.



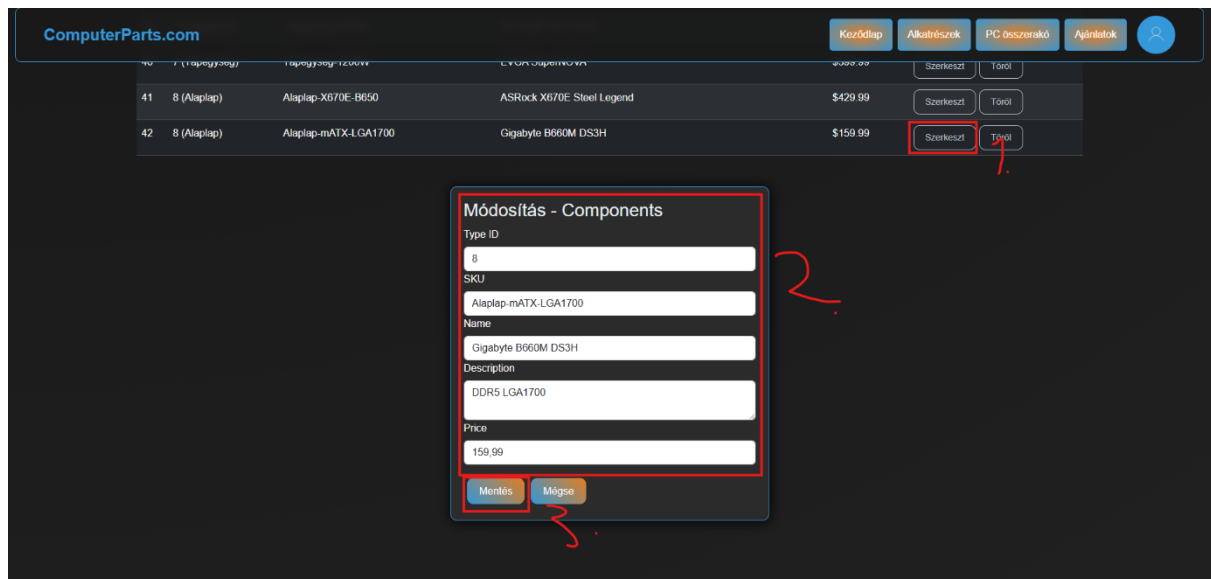
Megjegyzés: ez a kép még az előző oldalhoz tartozik.



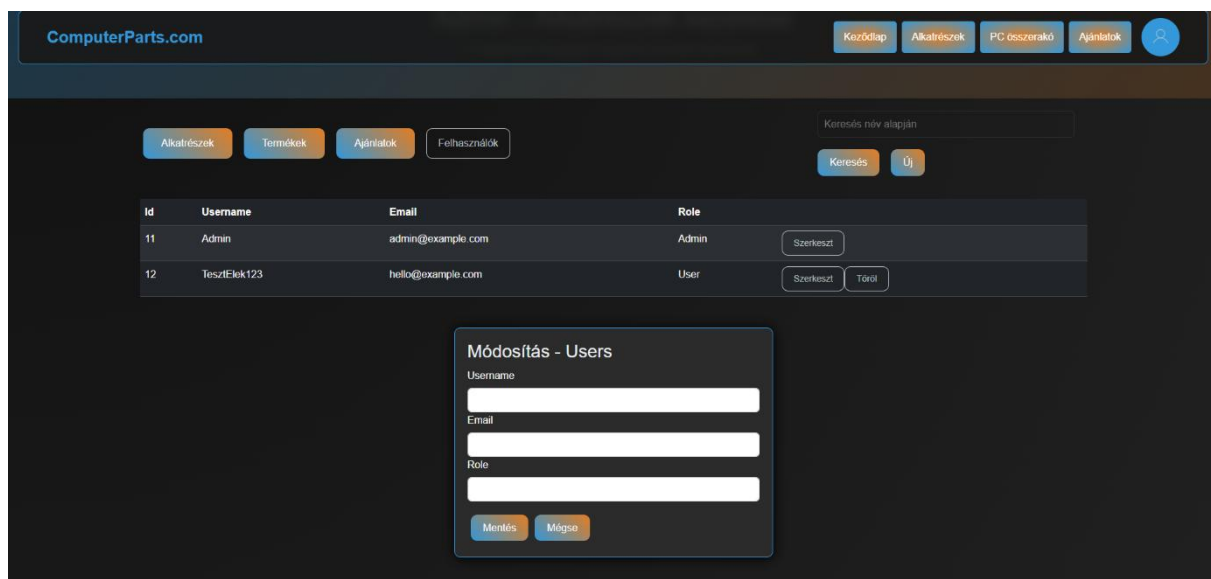
Itt pedig az Admin több mindent tud csinálni: először az „Alkatrészek” oldalra fogja dobni a rendszer, ahhol adatbázisunkban megtalálható összes, alkatrészként besorolt termék található meg. Itt az Admin tud újat hozzá adni, meglévőt szerkeszteni vagy törölni, valamint a kereső sávot használva név szerint rá tud keresni egy adott termékre. A felület így néz ki:



Ha az adminisztrátor valamelyik alkatrészt módosítani szeretné, akkor a „Módosítás” gombra kell először kattintania, majd a lap alján megjelenő ablakon megadnia a termék új adatait. *(Megjegyzés: az összes többi adminisztrátori felület leköveti ugyan ezt a sémát, design-t és működési elvet, emiatt úgy gondoljuk, felesleges lenne egyesével bemutatni mindegyiket, mivel a különböző felületek leírása pontosan ugyan az lenne, maximum eltérő megnevezésekkel és esetlegesen eltérő megfogalmazással. Most csak az „Alkatrészek” lap lesz bemutatva, de mindegyik admin szerkesztői oldal ugyan így fog kinézni és ugyan így is fog működni. Kép a következő oldalon.)*



Az adminisztrátoroknak meg van a joguk, hogy egy regisztrált felhasználói fiókot is módosítsanak, ezek közé kizárólag a felhasználónév, email cím és jogkör („Role”) tartozik bele, jelszó változtatást nem végezhet adminisztrátor. *(Megjegyzés: egyszerű felhasználói fiók adminisztrátorivá módosítása esetén az egyetlen megváltozó funkció, hogy eltűnik mellőle a „Töröl” gomb. Ha egy olyan fiókot szeretnénk eltávolítani, ami Admin jogokkal rendelkezik, először meg kell változtatni a jogkörét, „Role”-jét „User”-re, utána lesz csak törölhető ez a fiók.*



Köszönet nyilvánítás és Projekt értékelése

Köszönetnyilvánítás

Köszönet nyilvánításunkat úgy igazán nem is tudjuk, hol kezdhetnénk, viszont úgy igazából minden szeretünknek, ismerősünknek, családtagunknak, tanárunknak szeretnénk megköszönni, hogy a projekt során végig támogató, megértő magatartást tanúsítottak annak érdekében, hogy a lehető legjobb végeredményt tehessük le az asztalra a mi vizsgaremekünként, viszont azért még is vannak néhányan, akiknek külön meg szeretnénk köszönni az egész éves, vagy akár hat éves, vagy még annál is hosszabb közreműködését a projektben vagy életünkben.

Külön köszönet **szüleinknek** a folyamatos támogatásért a nehezebb időkben, amikor valamiért nem volt hajlandó egy nagyon apró hiba miatt működni egy apróbb, alacsonyabb metódus, amit már több órája próbálunk megoldani, de végül kiderült, hogy rossz helyen van egy nyitó- vagy záró tag, valamint a projekt minden egyes pillanatában, amíg dolgozhattunk rajta.

Külön köszönet **Török Edit** tanárnőnek, aki nem csak, mint projekt-vezető volt jelen a projekt fejlesztése során, hanem mint tanár rengeteget segített nekünk a projekt témájának részletes kidolgozásában, az órán tanított, hasznos példákra, mind a vizsgaremek projektben, mind pedig az elméleti vizsga modulra, valamint a folyamatos motiváló, ösztönző szavaiért!

Szintén külön köszönet **Méhes József** tanár úrnak is, aki számunkra nem csak egy egyszerű tanár volt az elmúlt egy tanévben, aki csak leadja a tananyagot és csak valamit éppen, hogy magyaráz, hanem sokkal inkább szívügyének tekintette, hogy segíthessen minket a projekt, főként Backend oldali kódok tökéletesre csiszolásában, egy bonyolultnak tűnő adatbázis tökéletes átlátásában, valamint a Frontend és a Backend összekötésében.

Egyúttal külön köszönet **Szabó Rózsa** tanárnőnek is, akit ugyan jól megszívattak azzal a heti egyetlen szerencsétlen órával, amivel megáldották és velünk kellett töltenie, de az igyekezete, hogy mindenben segítsen nekünk, amiben csak tud, számunkra felbecsülhetetlen értékkel bír.

Köszönjük **Csukárdi Rajmund** tanár úrnak is a segítséget, aki fiatalos, így hozzánk közel álló lendületével egyértelműen nem csak tanítani próbált, hanem meg is értetni velünk a talán túlságosan is túlbonyolított Javascript-Node párost, amit bár nagy részünk nem szeret, de a tanár úr igyekezetével egy igen is példát mutathat az előtte, valamint utána járó tanároknak is.

És természetesen hogyan is feledkezhetnénk meg kedvelt osztályfőnökünkéről, **Dr. Újj Anna** tanárnőről, aki nem csak az ebben az évben, de a teljes hat évében értünk tett munkájáért, amíg a mi osztályunkat terelte a jó útra és végig kitartott mellettünk, habár sokszor nem is érdemeltük volna meg.

*További jó egészséget kívánunk **Dr. Újj Anna** tanárnőnek, reméljük, még láthatjuk a közeljövőben, újra ugyan annak az élettel teli, vidám embernek, amelyennek megismertük 2020. szeptemberében az elmúlt, nehézségekkel teli időszak után!*

Projekt értékelése

Úgy gondoljuk, projektünkkel elértük azt, amit szerettünk volna. Ugyan az igazi tervünk az lett volna, hogy egy letisztult, egyszerűen használható, könnyű kezelő felülettel rendelkező webshopot készítsünk, helyette végül egy önmagában sokkal hasznosabb programot sikerült felépítenünk, amire az elektronikai eszközök értékesítésével foglalkozó webshopok túlnyomó többségének igen is hatalmas szüksége lehet a jövőben, nem pusztán azért, mert sikerült egy bárki számára könnyen alkalmazható PC Konfig összeállító weboldalt létrehoznunk a saját, önálló adatbázisával (*természetesen csak példa adatokkal*), hanem a felhasználók érdekében is, akik egy helyen szeretnék elvégezni az álom gépük alkatrészei összeválogatását és a rendelés leadását. A Webshoppá alakítási fejlesztői ötlet, a kód könnyű újrafelhasználhatósága, az átlátható, nem túlbonyolított programnyelvek és keretrendszerek és természetesen a már meglévő, szerintünk minőségi alap pedig számunkra pont ezt teszi lehetővé: egy olyan webshop létrehozása, ahol a felhasználó egy helyen tud mindent elvégezni jövő beli gamer vagy munkás gépével kapcsolatban.

Projektünket sikeresnek tekintjük, több okból is:

- Az oldal készítése során nem csak egy adott programnyelvet tanultunk meg jól használni, hanem több, különböző programnyelvet tudunk most már egymással működtetni, ami nem csak ezen projekt készítése, kódolása, design-olása végett egy hatalmas plusz pont, hanem a jövőre nézve is, amikor már hivatásos munkaként végezhetjük a szoftver fejlesztést és -tesztelést.
- Megtanultunk igazán csapatban dolgozni, megtudtuk, milyen az, amikor két embernek össze hangoltan kell dolgoznia ahhoz, hogy a végtermék ne csak elfogadható legyen, hanem törekedjünk arra, hogy a nagyvilágban is megállja a helyét, mint hivatásos programozók által készített szoftver vagy weboldal.

Számunkra egy élvezetes, kalandos, sokszor talán nehézségekkel teli utazás volt projektünket fejleszteni, viszont mi úgy gondoljuk, sikerült pontosan azt létrehoznunk, amit megálmodtunk akkor, amikor még azt sem tudtuk, mi is pontosan az „EntityFrameworkCore”, vagy hogy miért szükséges „Swagger” az adatbázis lekérdezéséhez C-Sharp programnyelvben, vagy amikor egy borzasztó egyszerű, néhány billentyű lenyomásával orvosolható hiba miatt szinte az egész rendszer az összeomlás szélén táncolt, de végül sikerült megjavítanunk és sok hónappal később elérni a végeredményt.

Kép források

C-Sharp Logo: https://commons.wikimedia.org/wiki/File:Csharp_Logo.png

Blazor Logo: <https://commons.wikimedia.org/wiki/File:Blazor.png>

Bootstrap logo: <https://getbootstrap.com/docs/5.0/about/brand/>

Swagger Logo: <https://swagger.io/>

MSSQL Logo: <https://www.svgrepo.com/svg/303229/microsoft-sql-server-logo>